

Státnicové okruhy

Bakalářská státní závěrečná zkouška

Aplikovaná informatika

Verze	Datum	Popis změn
1.0	2026-06-04	Počáteční verze
1.1	2026-06-04	Rozšíření Random Forest (hlasování vs průměr); obrázky K-Means a hierarchické shlukování; rozšíření IDF; vysvětlení facilitátorů COBIT; vztah COBIT a ITIL; úpravy COBIT/ITIL sekcí
1.2	2026-06-04	Rozšíření 4 perspektiv SAM; SOAP/WSDL; OPEX/CAPEX; typy MES; vysvětlení Zachmanova rámce; artefakty; procesní vs datový pohled; sequence flow vs message flow; paralelní zpracování BPMN; OMG a CIM
1.3	2026-06-04	Bezpečnost: příklady kryptografických primitiv; příklad proudové šifry (XOR); princip RSA/DH/EC; ukázka AES-GCM/AEAD; uznávaný el. podpis; L3/L4 packet filter; NAT+PAT s příklady; TLS 1.3 RTT a handshake; PKI

Obsah

1	Základy odborné práce	7
1.1	Vědecká práce a výzkumné metody	7
1.1.1	Základní pojmy	7
1.1.2	Kvantitativní vs. kvalitativní výzkum	7
1.1.3	Výzkumný proces	8
1.2	Struktura odborného textu – IMRaD	8
1.2.1	IMRaD struktura	8
1.2.2	Struktura diplomové práce vs. IMRaD	9
1.3	Bibliografické citace a odkazování	10
1.3.1	Principy citování	10
1.3.2	Citační norma ČSN ISO 690	10
1.3.3	Primární a sekundární zdroje	12
1.3.4	Recenzní řízení (Peer Review)	12
1.3.5	Etika vědecké práce	13
1.4	Informační etika	13
1.4.1	Masonův model – PAPA	13
1.4.2	Digitální propast (Digital Divide)	14
1.4.3	Etické problémy informační společnosti	15
1.5	Žánry odborného textu a citační nástroje	15
1.5.1	Žánry odborného textu	15
1.5.2	Citační manažery	16
2	Informační a komunikační technologie (IKT)	18
2.1	Síťové protokoly – IP, TCP, UDP, ICMP	18
2.1.1	Model TCP/IP a ISO/OSI	18
2.1.2	IP (Internet Protocol)	19
2.1.3	TCP (Transmission Control Protocol)	20
2.1.4	UDP (User Datagram Protocol)	22
2.1.5	ICMP (Internet Control Message Protocol)	22
2.1.6	DNS (Domain Name System)	22
2.2	Aplikační protokoly – SSH, E-mail, WWW	24
2.2.1	SSH (Secure Shell)	24
2.2.2	Architektura e-mailové komunikace	24
2.2.3	E-mail – SMTP, POP3, IMAP	24
2.2.4	Zabezpečení e-mailu	25
2.3	Webové technologie – HTTP, cookies, session	26
2.3.1	HTTP (Hypertext Transfer Protocol)	26
2.3.2	HTTPS a TLS	27
2.3.3	Cookies	27
2.3.4	Session (relace)	28
2.3.5	Client-side vs. Server-side zpracování	28
3	Programování v Javě a objektově-orientované paradigma	30
3.1	Objektově-orientované programování (OOP)	30
3.1.1	Základní principy OOP	30
3.1.2	Třída vs. instance	33

3.1.3	Abstraktní třídy vs. rozhraní	33
3.1.4	Principy SOLID	34
3.2	Test Driven Development (TDD)	35
3.2.1	Principy TDD	35
3.2.2	Typy testů	35
3.2.3	JUnit 5 v Javě	35
3.2.4	Mockování závislostí	36
3.3	Lambda výrazy a Stream API	36
3.3.1	Funkcionální rozhraní a lambda výrazy	36
3.3.2	Stream API	37
3.3.3	Optional	40
4	Softwarové inženýrství	41
4.1	UML diagramy	41
4.1.1	Diagram případů užití (Use Case Diagram)	41
4.1.2	Diagram tříd (Class Diagram)	41
4.1.3	Sekvenční diagram (Sequence Diagram)	42
4.1.4	Diagram aktivit (Activity Diagram)	43
4.2	Návrhové vzory (GoF Design Patterns)	44
4.2.1	Tvořivé vzory (Creational Patterns)	44
4.2.2	Strukturální vzory (Structural Patterns)	45
4.2.3	Vzory chování (Behavioral Patterns)	46
4.3	Metodiky vývoje IS	46
4.3.1	Tradiční (rigorózní) metodiky	46
4.3.2	Agilní metodiky	47
4.3.3	Srovnání přístupů	50
4.4	Architektonické vzory a CMMI	50
4.4.1	Architektonické vzory	50
4.4.2	CMMI (Capability Maturity Model Integration)	51
4.5	Software Quality Assurance (SQA)	52
4.5.1	Co je SQA a proč na ní záleží	52
4.5.2	Dimenze kvality softwaru – ISO/IEC 25010	52
4.5.3	Procesy SQA – s příklady z praxe	53
5	Databáze	56
5.1	Analýza, návrh a realizace databáze	57
5.1.1	Databázový systém a jeho vlastnosti	57
5.1.2	Metodologie analýzy a návrhu databáze – P3A	58
5.1.3	Normalizace databáze	60
5.1.4	Integrita a transakce	64
5.1.5	Indexy	66
5.2	Datové modelování	66
5.2.1	Základní pojmy	68
5.2.2	ER (Entity Relationship) modelování	68
5.2.3	Transformace do relačního modelu dat	69
5.2.4	Modely databázových systémů	69
5.3	Databázové jazyky	70
5.3.1	Dělení databázových jazyků	70
5.3.2	Vyhledávací interface pro relační databáze	71

5.3.3	SQL (Structured Query Language)	73
5.3.4	PL/SQL	74
5.3.5	T-SQL (Transact-SQL)	74
5.3.6	OQL (Object Query Language)	76
5.3.7	JPQL (Java Persistence Query Language)	76
5.3.8	XQuery	77
5.3.9	Datové formáty pro výměnu dat	77
5.3.10	SQL Injection	77
6	Datová analytika	79
6.1	Přidaná hodnota dat pro business	79
6.1.1	Data jako strategický aktiv	79
6.2	Kvalita dat (Data Quality)	80
6.2.1	Dimenze kvality dat	80
6.2.2	Příčiny špatné kvality dat	80
6.2.3	Data Quality Management	81
6.3	Datový sklad, OLAP a Business Intelligence	81
6.3.1	Datový sklad (Data Warehouse)	81
6.3.2	ETL (Extract, Transform, Load)	83
6.3.3	Dimenzionální modelování	85
6.3.4	OLAP (Online Analytical Processing)	87
6.3.5	KPI, Dimenze, Granularita	88
6.3.6	Business Intelligence (BI)	88
6.3.7	Balanced Scorecard (BSC)	90
7	Zpracování informací	92
7.1	Strojové učení a dolování z dat (Data Mining)	92
7.1.1	Základní pojmy	92
7.1.2	CRISP-DM	93
7.1.3	Rozhodovací stromy (Decision Trees)	95
7.1.4	Clustering (Shlukování)	96
7.1.5	Asociační pravidla	97
7.1.6	Hodnocení klasifikačních modelů	99
7.2	Ukládání a vyhledávání textových informací	100
7.2.1	Information Retrieval (IR)	100
7.2.2	Booleovský model vyhledávání	101
7.2.3	Vektorový model (Vector Space Model)	101
7.2.4	PageRank	103
7.3	Regulární výrazy	103
7.3.1	Základy regulárních výrazů	103
8	IT Governance	106
8.1	Governance vs. Management	106
8.1.1	Základní rozdíl	106
8.1.2	Klíčové oblasti EGIT	106
8.2	COBIT (Control Objectives for Information Technology)	107
8.2.1	Klíčové novinky COBIT 2019	107
8.2.2	Pět principů COBIT 5	108
8.2.3	Domény procesů COBIT 5	109

8.2.4	Model procesní vyspělosti COBIT (PAM)	109
8.2.5	COBIT v praxi – příklad z bankovníctví	110
8.3	ITIL (IT Infrastructure Library)	111
8.3.1	ITIL 3 (v3, 2007/2011)	111
8.3.2	ITIL v praxi – klíčové procesy s příklady	111
8.3.3	ITIL 4 (2019)	112
8.3.4	Srovnání ITIL 3 vs ITIL 4	115
9	Podnikové informační systémy	116
9.1	Typy podnikových IS	116
9.1.1	ERP (Enterprise Resource Planning)	116
9.1.2	CRM (Customer Relationship Management)	116
9.1.3	BI (Business Intelligence)	117
9.1.4	SCM (Supply Chain Management)	117
9.1.5	MES (Manufacturing Execution System)	117
9.2	Implementace podnikových IS	117
9.2.1	4 etapy implementace podnikového IS	117
9.2.2	IASW vs. TASW	118
9.3	Trendy a přínosy podnikových IS	119
9.3.1	Cloud Computing	119
9.3.2	SOA (Service-Oriented Architecture)	119
9.3.3	Přínosy a rizika podnikových IS	121
9.3.4	Ekonomické hodnocení IS investic	122
10	Analýza a návrh informačních systémů	123
10.1	Propojení IT a businessu	123
10.1.1	Proč je propojení IT a businessu důležité	123
10.1.2	SAM – Strategic Alignment Model	123
10.1.3	IS/IT Strategie	125
10.1.4	Podniková architektura (Enterprise Architecture)	125
10.1.5	Digitální transformace	128
10.2	MMDIS – Multidimenzionální metoda pro IS	130
10.2.1	11 principů MMDIS	130
10.2.2	9 fází MMDIS (skupiny GST-VY)	130
10.2.3	3 skupiny pohledů na IS	131
10.3	Procesní modelování	131
10.3.1	Základní pojmy	131
10.3.2	EPC (Event-driven Process Chain)	132
10.3.3	BPMN (Business Process Model and Notation)	132
10.3.4	KBPR (Key Business Process Requirements)	135
10.4	Architektura IS a způsoby zpracování	136
10.4.1	MDA (Model Driven Architecture)	136
10.4.2	Způsoby zpracování dat v IS	136
10.4.3	Typy IS/ICT služeb	137
11	Bezpečnost informačních systémů	138
11.1	Kryptografie a kryptografická primitiva	138
11.1.1	Základní pojmy	138
11.1.2	Kryptografická primitiva	138

11.1.3	Symetrická kryptografie	138
11.1.4	Asymetrická kryptografie	140
11.2	Hash, MAC, Digitální podpis	141
11.2.1	Hašovací funkce (Hash)	141
11.2.2	MAC (Message Authentication Code)	141
11.2.3	AEAD – Authenticated Encryption with Associated Data	141
11.2.4	Digitální podpis	142
11.3	Certifikáty, eIDAS	142
11.3.1	Digitální certifikát	142
11.3.2	eIDAS – Elektronická identifikace a podpisy	143
11.4	Identifikace, Autentizace, Vícefaktorová autentizace	143
11.4.1	Identifikace vs. Autentizace vs. Autorizace	143
11.4.2	Hesla	143
11.4.3	Vícefaktorová autentizace (MFA)	144
11.5	Segmentace sítě a bezpečnostní technologie	144
11.5.1	Segmentace sítě	144
11.5.2	Firewall	144
11.5.3	Proxy servery	145
11.5.4	NAT (Network Address Translation)	145
11.5.5	IDS a IPS	146
11.5.6	TLS (Transport Layer Security)	146
11.5.7	VPN (Virtual Private Network)	147
11.5.8	Honeypot a Web Application Firewall	147

1 Základy odborné práce

1.1 Vědecká práce a výzkumné metody

1.1.1 Základní pojmy

- **Věda** – systematická snaha budovat a organizovat znalosti ve formě testovatelných vysvětlení a predikcí o světě
- **Výzkum** – cílená, systematická a pečlivě prováděná investigace zaměřená na nalezení nových faktů nebo principů
- **Výzkumná metoda** – soubor technik a procedur používaných k identifikaci a analýze informací
- **Hypotéza** – testovatelné tvrzení, které může být potvrzeno nebo vyvráceno empirickými daty
- **Teorie** – ověřené vysvětlení nebo model přijatý vědeckou komunitou

1.1.2 Kvantitativní vs. kvalitativní výzkum

Kvantitativní výzkum	Kvalitativní výzkum
Numerická data, statistická analýza	Textová/narativní data, interpretace
Ověřuje hypotézy	Generuje hypotézy
Velký výzkumný vzorek	Malý vzorek, hloubkový pohled
Dotazníky, experimenty	Rozhovory, pozorování, případové studie
Výsledky lze zobecnit	Výsledky kontextuálně specifické
Objektivita	Subjektivita (role výzkumníka)
Deduktivní přístup (od teorie k datům)	Induktivní přístup (od dat k teorii)

Proč kvantitativní ověřuje a kvalitativní generuje hypotézy? Rozdíl plyne z opačného směru uvažování – **deduktivního** (od teorie k datům) vs. **induktivního** (od dat k teorii).

Kvantitativní výzkum vychází z existující teorie nebo dosavadního poznání. Výzkumník nejprve formuluje konkrétní, měřitelnou hypotézu (např. „*Studenti, kteří denně stráví více než 3 hodiny na sociálních sítích, mají nižší průměr známek*“), poté sesbírá numerická data a statisticky otestuje, zda je hypotéza pravdivá. Hypotéza tedy **existuje před sběrem dat** – cílem je ji potvrdit nebo vyvrátit.

Kvalitativní výzkum naopak vstupuje do terénu bez pevně dané hypotézy. Výzkumník provádí hloubkové rozhovory, pozoruje nebo analyzuje texty a z dat induktivně **odhaluje vzorce a témata**. Tato témata pak vedou ke vzniku nových hypotéz nebo teorií. Hypotéza tedy **vzniká až po sběru dat** jako výstup procesu.

Příklad – výzkum vlivu sociálních sítí na studijní výsledky:

- *Kvalitativní fáze (generuje hypotézu):* Výzkumník provede 20 hloubkových rozhovorů se studenty a zjistí, že mnozí zmiňují „přerušování soustředění notifikacemi“ jako hlavní problém. Na základě toho formuluje hypotézu: „*Frekvence notifikací negativně koreluje se studijním výkonem.*“
- *Kvantitativní fáze (ověřuje hypotézu):* Výzkumník sesbírá data od 500 studentů (počet notifikací/den, GPA) a provede korelační analýzu. Výsledek hypotézu potvrdí nebo vyvrátí.

Kvalitativní přístup je tedy typický pro **explorativní výzkum** – kdy téma není dobře zmapované a nevíme, co hledat. Kvantitativní přístup nastupuje, když již víme, co testovat, a potřebujeme výsledek zobecnit na větší populaci.

Smíšený výzkum (Mixed Methods) Kombinace kvantitativního a kvalitativního přístupu – využívá výhody obou přístupů.

1.1.3 Výzkumný proces

1. Identifikace výzkumného problému
2. Přehled literatury (literature review)
3. Formulace výzkumných otázek nebo hypotéz
4. Volba výzkumné metodologie
5. Sběr dat
6. Analýza dat
7. Interpretace výsledků
8. Prezentace a publikace výsledků

1.2 Struktura odborného textu – IMRaD

1.2.1 IMRaD struktura

IMRaD (Introduction, Methods, Results and Discussion) je standardní struktura vědeckého článku, zejména v přírodních a sociálních vědách. Odpovídá na čtyři klíčové otázky:

1. **Introduction (Úvod)** – *Proč byl výzkum proveden?*
 - Kontext a motivace výzkumu
 - Přehled existující literatury (stav poznání)
 - Identifikace výzkumné mezery (research gap)
 - Cíl práce a výzkumné otázky/hypotézy
 - Stručný přehled struktury práce

2. **Methods (Metody)** – *Jak byl výzkum proveden?*

- Výzkumný design a přístup
- Popis vzorku a způsob sběru dat
- Použité nástroje, techniky a analytické metody
- Postup replikovatelný jinými výzkumníky
- Etické aspekty výzkumu

3. **Results (Výsledky)** – *Co bylo zjištěno?*

- Objektivní prezentace zjištění bez interpretace
- Tabulky, grafy a obrázky s popisky
- Statistická data (pokud je výzkum kvantitativní)
- Odpovědi na výzkumné otázky

4. **Discussion (Diskuse)** – *Co výsledky znamenají?*

- Interpretace výsledků v kontextu existující literatury
- Potvrzení nebo vyvrácení hypotéz
- Praktické implikace zjištění
- Omezení výzkumu (limitations)
- Doporučení pro budoucí výzkum

Doplňující části odborného textu

- **Abstract (Abstrakt)** – stručné shrnutí celé práce (150–250 slov); píše se jako poslední
- **Klíčová slova** – 4–8 termínů pro indexování
- **Závěr (Conclusion)** – syntéza hlavních zjištění; někdy sloučen s Discussion
- **Literatura (References)** – seznam použitých zdrojů
- **Přílohy (Appendices)** – doplňující materiály (dotazníky, rozsáhlá data apod.)

1.2.2 Struktura diplomové práce vs. IMRaD

IMRaD je formát **vědeckých a akademických článků** (journal paper, conference paper apod.), zatímco diplomová práce (DP) má rozsáhlejší strukturu odpovídající akademickým požadavkům školy.

Diplomová práce (VŠE)	IMRaD (vědecký článek)
Titulní strana + prohlášení	–
Poděkování (volitelné)	–
Abstrakt CZ + EN (150–300 slov)	Abstract (100–250 slov)
Klíčová slova	Keywords
Obsah, seznam ob- rázků/tabulek	–
Úvod – cíl, motivace, struk- tura	Introduction
Teoretická část – přehled li- teratury (rešerše)	(zahrnut v Introduction)
Metodika	Methods
Analytická/empirická část	Results
Výsledky a diskuse	Discussion
Závěr + doporučení	(součást Discussion)
Seznam literatury	References
Přílohy	Appendices (volitelné)

Klíčové rozdíly:

- DP obsahuje povinnou **titulní stranu a prohlášení o samostatném zpracování**
- DP má rozsáhlou samostatnou **teoretickou část** (rešerše literatury); v IMRaD je přehled literatury pouze součástí Úvodu
- DP je obvykle **50–100+ stran**; vědecký článek 8–20 stran
- DP musí být schválena vedoucím, IMRaD prochází externím recenzním řízením

1.3 Bibliografické citace a odkazování

1.3.1 Principy citování

Citování slouží k:

- Vzdání uznání autorům citovaných prací
- Podpoření vlastních tvrzení autoritativními zdroji
- Umožnění čtenáři dohledat zdrojové informace
- Prevenci plagiátorství

Plagiátorství – použití cizích myšlenek, textu nebo dat bez řádného odkazu na původního autora. Zahrnuje: doslovné kopírování, parafrázi bez odkazu, sebeplagiátorství.

1.3.2 Citační norma ČSN ISO 690

Mezinárodní norma pro bibliografické citace platná v ČR. Definuje povinné a nepovinné prvky bibliografického záznamu a způsob jejich uspořádání.

Typy citačních stylů

- **Harvardský styl (autor-rok)** – odkaz v textu: (Novák, 2020); seřazení dle autora
- **Číselný styl (Vancouver/IEEE)** – odkaz v textu: [1]; seřazení dle pořadí výskytu; IEEE standard hojně využíván v informatice a elektrotechnice
- **APA 7 (American Psychological Association)** – preferovaný styl na VŠE Praha; sociální a behaviorální vědy; podrobně níže
- **MLA (Modern Language Association)** – hojně používaný v humanitních vědách; v textu: (Novák 45) – příjmení + strana bez čárky
- **Chicago** – dvě varianty: *Author-Date* (podobná APA) nebo *Notes and Bibliography* (poznámky pod čarou); hojně v historii a humanitních vědách
- **Oxford (OSCOLA)** – právní vědy; poznámky pod čarou s čísly

APA 7 – detailní přehled APA 7. vydání (2020) je aktuální standard. Klíčové prvky:

- **Odkaz v textu:** (Příjmení, rok) nebo Příjmení (rok) uvádí...
 - Dva autoři: (Novák & Svoboda, 2020)
 - Tři a více: (Novák et al., 2020) – od 1. citace
- **Bibliografický záznam – monografie:**
Příjmení, I. (Rok). *Název knihy*. Nakladatel. <https://doi.org/xxx>
- **Bibliografický záznam – článek v časopise:**
Příjmení, I. (Rok). Název článku. *Název časopisu, ročník* (číslo), strany. <https://doi.org/xxx>
- **Novinky APA 7 oproti APA 6:**
 - Uvádí se až 20 autorů (dříve 6, pak “et al.”)
 - DOI jako hyperlink (<https://doi.org/...>) povinný, pokud existuje
 - Místo vydání se **neuvádí** (dříve povinné)
 - Přidány pokyny pro citace podcastů, TikTok, Twitter aj.
- **Seznam literatury:** abecedně dle příjmení, visutý odsek (hanging indent) 1,27 cm

Struktura bibliografického záznamu (ČSN ISO 690) Základní prvky záznamu pro monografii:

1. Příjmení a jméno autora
2. Název díla
3. Vydání (pokud není první)
4. Místo vydání
5. Nakladatel
6. Rok vydání

7. ISBN/ISSN

Příklady záznamů:

- *Monografie*: NOVÁK, Jan. *Název knihy*. 2. vyd. Praha: Nakladatelství, 2020. ISBN 978-80-000-0000-0.
- *Článek v časopise*: NOVÁK, Jan. Název článku. *Název časopisu*. 2020, **15**(3), 45–67. ISSN 1234-5678.
- *WWW zdroj*: NOVÁK, Jan. Název dokumentu [online]. 2020 [cit. 2024-01-15]. Dostupné z: <https://www.example.com>

1.3.3 Primární a sekundární zdroje

- **Primární zdroje** – původní výzkum, originální data; vědecké články (peer-reviewed), disertace, zprávy z výzkumu
- **Sekundární zdroje** – interpretace nebo analýza primárních zdrojů; přehledové články, učebnice, encyklopedie
- **Terciární zdroje** – kompilace sekundárních zdrojů; bibliografie, databáze

1.3.4 Recenzní řízení (Peer Review)

Peer review (recenzní řízení) je proces, při němž nezávislí odborníci ze stejného oboru posuzují vědeckou práci **před** jejím přijetím k publikaci. Cílem je zajistit kvalitu, vědeckou správnost a relevanci výsledků.

Jak recenzní řízení probíhá – krok za krokem

1. **Podání práce** – autor odešle rukopis redakci časopisu
2. **Desk rejection** – editor rozhodne, zda je práce vhodná pro daný časopis (tematicky i kvalitativně); nevhodné práce jsou okamžitě odmítnuty
3. **Výběr recenzentů** – editor osloví 2–3 odborníky; recenzenti mohou odmítnout
4. **Recenze** – recenzenti obdrží rukopis a do 4–8 týdnů vypracují posudek
5. **Rozhodnutí editora**:
 - *Accept* – přijato bez úprav (vzácné u prvního podání)
 - *Minor revision* – drobné úpravy, autor reaguje písemně
 - *Major revision* – zásadní úpravy, po revizi znovu recenzováno
 - *Reject* – odmítnuto; autor může práci přepracovat a podat jinde
6. **Revize a odpověď** – autor odevzdá opravenou verzi + dokument odpovědí na každý komentář recenzenta
7. **Finální přijetí a publikace**

Typy peer review

- **Single-blind** – recenzenti znají jméno autora, autor nezná recenzenty
- **Double-blind** – ani recenzenti neznají autora, ani autor recenzenty (nejčastější v sociálních vědách)
- **Open review** – identity jsou veřejné; transparentnější, ale může odradit recenzenty
- **Post-publication review** – recenze probíhá po publikaci (např. preprints na arXiv)

Konkrétní příklad Autor napsal článek o efektivnosti nové metody výuky programování (kvantitativní studie, $n=120$ studentů). Podá jej do časopisu *Computers & Education*. Editor osloví dvě recenzenty z oblasti informatiky a pedagogiky. Po 6 týdnech přijde rozhodnutí *Major revision*: recenzent 1 požaduje rozšíření sekce o metodice a doplnění kontrolní skupiny do experimentu; recenzent 2 navrhuje doplnit přehled literatury o 5 klíčových prací. Autor práci přepracuje a napíše detailní odpovědi (“Response to Reviewers”), kde bodově reaguje na každý komentář. Po druhém kole recenzí je práce přijata.

1.3.5 Etika vědecké práce

- **Integrita** – poctivost v hlášení dat a výsledků
- **Objektivita** – minimalizace předsudků ve výzkumném designu a interpretaci
- **Transparentnost** – zveřejňování metodologie a dat
- **Ochrana subjektů výzkumu** – informovaný souhlas, anonymizace dat
- **Konflikt zájmů** – deklarování potenciálních konfliktů

1.4 Informační etika

1.4.1 Masonův model – PAPA

Richard Mason (1986) identifikoval čtyři základní etické problémy informačního věku (model PAPA). Model slouží jako **checklist pro etické hodnocení IS/IT projektů** – pomáhá zjistit, kde mohou vznikat etické problémy při návrhu, nasazení nebo používání informačního systému.

- **Privacy (Soukromí)** – *Jaké informace smíme o lidech sbírat a ukládat?*
 - Otázka: Sbíráme data, která opravdu potřebujeme? Informoval jsme uživatele? Mohou ho odmítnout?
 - Příklady v IS: HR systém ukládající záznamy o zdravotním stavu zaměstnanců; e-shop sledující chování uživatele bez souhlasu; Facebook profiling
 - Relevantní předpisy: GDPR, zákon o ochraně osobních údajů
- **Accuracy (Přesnost)** – *Kdo odpovídá za správnost informací a jejich opravu?*

- Otázka: Jsou data v systému aktuální a správná? Co se stane, když jsou chybná?
- Příklady v IS: chybná kreditní zpráva znemožní získání půjčky; chyba v registru pacientů vede ke špatnému léčení; dezinformace na zpravodajském portálu
- Kdo nese odpovědnost za opravu? Systém, provozovatel, nebo uživatel?
- **Property (Vlastnictví)** – *Kdo vlastní data a SW, a jaká jsou práva k jejich použití?*
 - Otázka: Patří data zákazníkovi nebo firmě, která je sesbírala? Platím za SW nebo ho mohu kopírovat?
 - Příklady v IS: uživatelský obsah nahraný na YouTube vlastní YouTube (dle podmínek); softwarové licence (proprietární vs. open source); patenty na algoritmy
 - Klíčové koncepty: autorské právo, licence (GPL, MIT, proprietární), duševní vlastnictví
- **Accessibility (Přístupnost)** – *Kdo má právo přistupovat k jakým informacím a za jakých podmínek?*
 - Otázka: Je přístup k informacím omezený technologicky, ekonomicky, nebo politicky?
 - Příklady v IS: placené vědecké databáze (Elsevier) vs. open access; internetová cenzura; informace dostupné jen majitelům drahých zařízení (digitální propast)
 - Relevantní: právo na informace, e-government, přístupnost pro ZTP (WCAG)

Jak PAPA použít v praxi Při návrhu IS (např. personálního systému, mobilní aplikace, e-shopu) projdete každou oblast:

1. **P:** Která osobní data sbíráme? Máme souhlas? Je to minimální nutné množství?
2. **A:** Jak zajistíme přesnost dat? Kdo ověřuje a opravuje chyby?
3. **P:** Kdo je vlastníkem dat a kódu? Jaká licencování potřebujeme?
4. **A:** Kdo může k systému přistupovat? Není přístup nespravedlivě omezený?

1.4.2 Digitální propast (Digital Divide)

Nerovnoměrný přístup k ICT technologiím a internetu:

- **Horizontální propast** – rozdíly ve vlastnictví zařízení a přístupu k internetu
- **Vertikální propast** – rozdíly v dovednostech a schopnosti využívat ICT efektivně
- Dimenze: ekonomická, geografická (venkov vs. město), generační, genderová

1.4.3 Etické problémy informační společnosti

Mikroetické problémy (úroveň jednotlivce)

- Kyberšikana a obtěžování online
- Závislost na technologiích
- Narušení soukromí sociálními sítěmi
- Neetické využívání AI (deepfakes, manipulace)

Makroetické problémy (úroveň společnosti)

- Dohledová společnost (surveillance capitalism) – sběr a monetizace osobních dat
- Algoritmická diskriminace – automatizovaná rozhodnutí znevýhodňující skupiny
- Digitální monopoly (GAFAM – Google, Amazon, Facebook, Apple, Microsoft)
- Dopady automatizace na trh práce

1.5 Žánry odborného textu a citační nástroje

1.5.1 Žánry odborného textu

- **Abstrakt** – nejkratší odborný text; shrnutí celé práce (150–250 slov); vždy v češtině i angličtině
- **Anotace** – několik řádků; autor vysvětlí, co se za názvem tématu skrývá
- **Resumé** – delší shrnutí práce (celá kapitola nebo část); někdy v jiném jazyce
- **Recenze** – zhodnocení vědecké práce; subjektivní pohled; zpravidla do 7 stran
- **Rešerše** – soupis subjektivně nejdůležitějších bodů a myšlenek odborného textu; slouží k rychlé orientaci v tématu
- **Konspekt** – písemný záznam obsahu a podstatných myšlenek odborného díla; výpis z práce, často doplněný komentáři autora
- **Teze** – zhuštěný výklad hlavních myšlenek *teprve připravované* publikace; cca 10 stran
- **Odborný esej** – slohově vybroušené úvahy; cílem je navrhnout řešení odborné otázky; svobodný útvar s vysokými nároky na odbornost
- **Polemika** – vědecký spor; vyargumentované hájení vlastního názoru vůči jinému textu
- **Referát** – zpráva o odborném problému a jeho výsledcích; určena k prezentaci
- **Koreferát** – navazuje na referát; zaměřen pouze na jeden aspekt problému z předchozího referátu

- **Vědecká stať** – autor koncipuje vlastní nové řešení problému; plnohodnotný odborný žánr
- **Odborný vědecký článek** – kratší stať představující výsledky k danému problému
- **Příspěvek do sborníku** – hlavní myšlenka a závěry jsou autorem předneseny na vědecké konferenci
- **Monografie** – vědecká kniha věnovaná omezenému tématu do hloubky
- **Učebnice** – odborné texty určené ke vzdělávání čtenářů
- **Seminární / ročníková práce** – zadávána studentům SŠ i VŠ; nácvik výzkumných a písemných dovedností
- **Závěrečná práce** – bakalářská, diplomová, rigorózní (po složení magisterské zkoušky)
- **Disertační práce** – doktorská práce s původním vědeckým přínosem

1.5.2 Citační manažery

Software pro správu bibliografických záznamů a automatické generování citací:

- **Zotero** – open source, zdarma; plugin pro prohlížeč; integrace s Word/LibreOffice
- **Citace.com / Citace PRO** – česky; oblíbený v českém akademickém prostředí
- **EndNote** – komerční; rozšířený v přírodních a lékařských vědách
- **Mendeley** – od Elseviera; sociální síť pro vědce, PDF anotace
- **RefWorks** – cloudový; oblíbený na amerických univerzitách

Funkce citačních manažerů: automatický import metadat, generování citací v různých stylech, správa PDF, spolupráce.

APA styl na VŠE APA (American Psychological Association) je preferovaný citační styl na VŠE Praha. Formát: *Příjmení, Iniciála. (Rok). Název. Vydavatel. Odkaz v textu: (Novák, 2020) nebo Novák (2020) uvádí...*

Otázky profesorů k tématu: Základy odborné práce

Pour Jan, doc. Ing., CSc.

- Jaká je obecná struktura odborného textu?
- Co je IMRaD a jak se liší od standardní struktury diplomové práce?
- Jak rozlišit primární a sekundární zdroje?

Řepa Václav, prof. Ing., CSc.

- Jaký je rozdíl mezi kvantitativním a kvalitativním výzkumem?
- Jak probíhá recenzní řízení vědeckého článku?

Maryška Miloš, doc. Ing., Ph.D.

- Co je plagiátorství a jak mu předcházet?
- Jaké citační normy znáte?

Chudán David, Ing., Ph.D.

- Jaký je postup při tvorbě odborné práce?
- Co tvoří abstrakt odborného článku?

2 Informační a komunikační technologie (IKT)

2.1 Síťové protokoly – IP, TCP, UDP, ICMP

2.1.1 Model TCP/IP a ISO/OSI

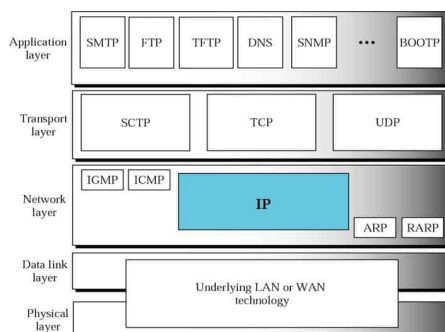
Síťová komunikace je popsána vrstevnými modely. **ISO/OSI** má 7 vrstev (fyzická, linková, síťová, transportní, relační, prezentační, aplikační). **TCP/IP** má 4 vrstvy:

TCP/IP vrstva	Funkce	Protokoly
Aplikační	Síťové aplikace	HTTP, FTP, SMTP, DNS, SSH
Transportní	Přenos dat end-to-end	TCP, UDP
Síťová (Internet)	Adresování a směrování	IP, ICMP, ARP
Přístup k síti	Fyzický přenos	Ethernet, Wi-Fi

ISO/OSI x TCP/IP		
Aplikační vrstva	Aplikační vrstva	
Prezentační vrstva		
Relační vrstva		
Transportní vrstva	Transportní vrstva	
Síťová vrstva	Síťová vrstva	
Linková vrstva	Vrstva síťového přístupu	
Fyzická vrstva		

Obrázek 1: Porovnání modelů ISO/OSI a TCP/IP

Position of IP in TCP/IP protocol suite



Obrázek 2: Síťové protokoly v modelu TCP/IP

2.1.2 IP (Internet Protocol)

Vrstva: Síťová (TCP/IP: Internet layer, OSI: vrstva 3)

IP je základní protokol síťové vrstvy zajišťující směrování paketů.

IPv4

- 32-bitová adresa (4 oktety, např. 192.168.1.1)
- **Třídy adres** – historické dělení adresního prostoru (classful networking); každá třída má výchozí masku a rozsah:
 - **A** – rozsah 0.0.0.0 – 127.255.255.255, maska /8; velké sítě (např. 10.0.0.0)
 - **B** – rozsah 128.0.0.0 – 191.255.255.255, maska /16; střední sítě (např. 172.16.0.0)
 - **C** – rozsah 192.0.0.0 – 223.255.255.255, maska /24; malé sítě (např. 192.168.1.0)
 - **D** – rozsah 224.0.0.0 – 239.255.255.255; multicast (skupinové vysílání)
 - **E** – rozsah 240.0.0.0 – 255.255.255.255; rezervováno pro experimentální účely

Dnes se místo tříd používá **CIDR** (Classless Inter-Domain Routing) – maska se zapisuje jako prefix (např. /24).

- **Privátní adresy** (RFC 1918) – adresy, které nejsou směrovány na internetu; slouží pro interní sítě (LAN, domácnosti, firmy):
 - 10.0.0.0/8 – třída A (velké podnikové sítě)
 - 172.16.0.0/12 – třída B (střední sítě)
 - 192.168.0.0/16 – třída C (domácí sítě, typicky 192.168.1.x)

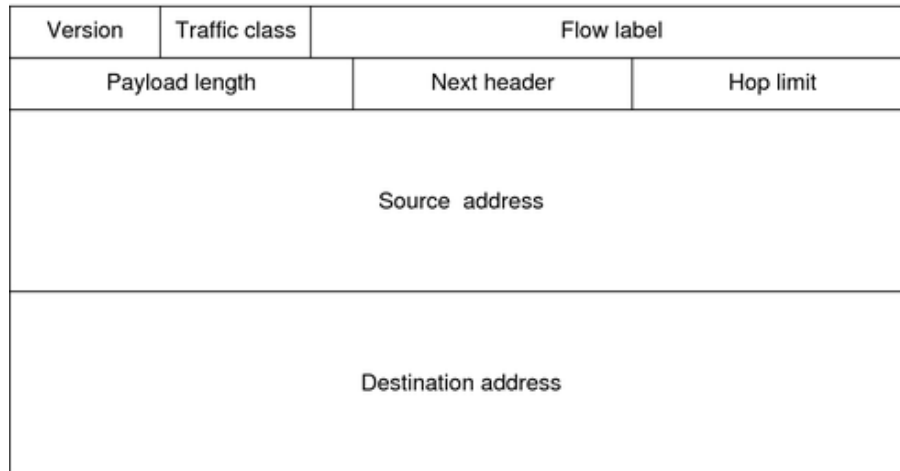
Na internet se dostanou přes **NAT** – router privátní adresu přeloží na svou veřejnou IP.

- **Přidělování adres:** statické nebo DHCP (Dynamic Host Configuration Protocol)
- **NAT** (Network Address Translation) – překlad privátních adres na veřejné; řeší nedostatek adres

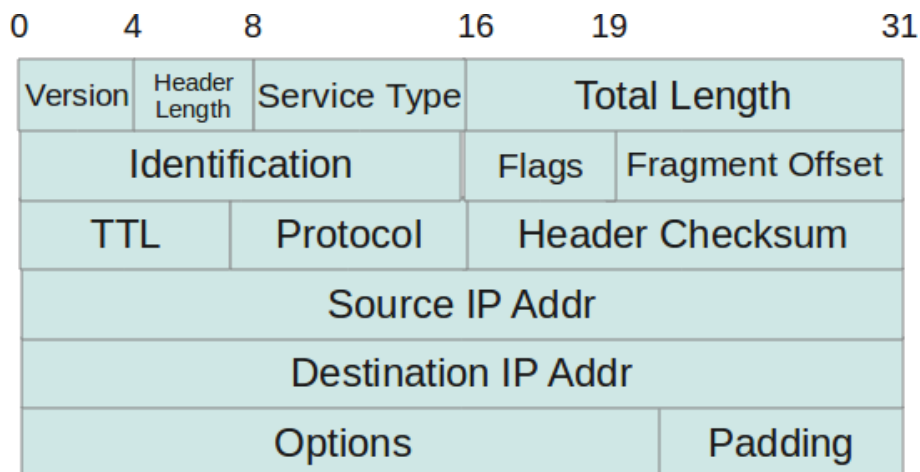
IPv6

- 128-bitová adresa (8 skupin po 4 hex číslicích, např. 2001:0db8:85a3::8a2e:0370:7334)
- 2^{128} adres – řeší problém nedostatku adres
- **SLAAC** (Stateless Address Autoconfiguration) – zařízení si samo nakonfiguruje IPv6 adresu *bez DHCP serveru*; router rozešle Router Advertisement (RA) se síťovým prefixem, zařízení k němu připojí vlastní identifikátor odvozený z MAC adresy (EUI-64) nebo náhodně vygenerovaný
- **IPsec** – sada protokolů pro zabezpečení komunikace na síťové vrstvě (šifrování + autentizace paketů); v IPv4 je volitelný, v IPv6 je *nativně integrován*; využívá se např. ve VPN; dva režimy:

- **Transport mode** – šifruje pouze payload (obsah paketu)
- **Tunnel mode** – šifruje celý IP paket (typické pro VPN)
- Záhlaví zjednodušeno oproti IPv4 – pevná délka 40 bajtů, volitelná rozšíření přes Extension Headers



Obrázek 3: Struktura IPv6 hlavičky



Obrázek 4: Struktura IPv4 hlavičky

2.1.3 TCP (Transmission Control Protocol)

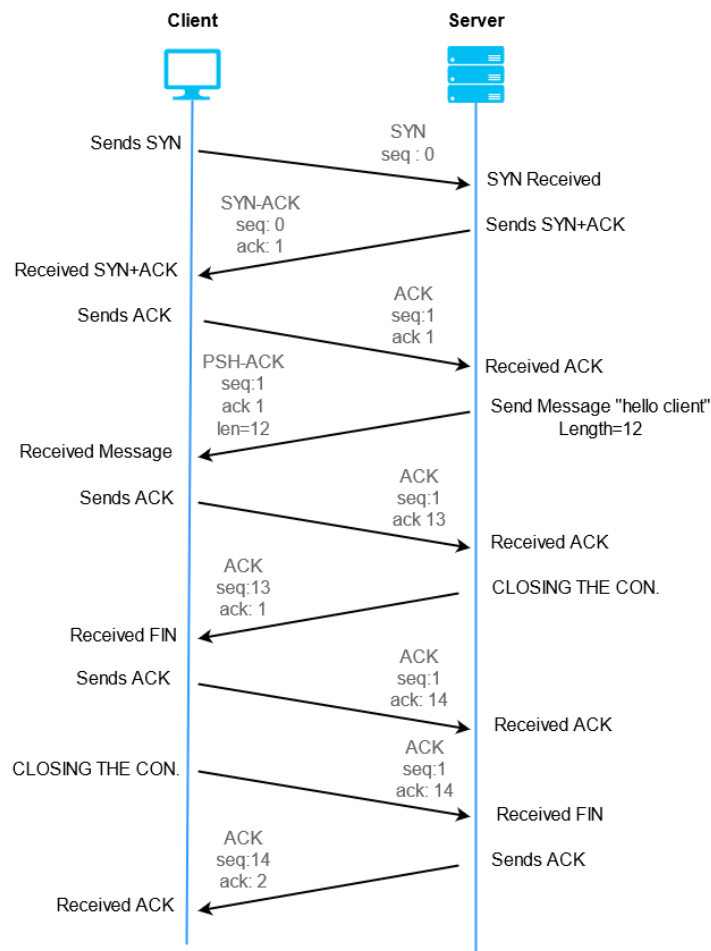
Vrstva: Transportní (TCP/IP: Transport layer, OSI: vrstva 4)

TCP je spolehlivý, spojovaný protokol transportní vrstvy. Garantuje doručení dat ve správném pořadí.

Navázání spojení – 3-Way Handshake

1. **SYN** – klient pošle segment s příznakem SYN a náhodným sekvenčním číslem (ISN)
2. **SYN-ACK** – server potvrdí ($ACK = ISN + 1$) a pošle vlastní SYN se svým ISN

3. **ACK** – klient potvrdí serverovo ISN ($ACK = serverISN + 1$); spojení navázáno



Obrázek 5: TCP 3-Way Handshake

Ukončení spojení – 4-Way Termination

1. Klient → FIN
2. Server → ACK
3. Server → FIN
4. Klient → ACK

Vlastnosti TCP

- **Spolehlivost** – potvrzování (ACK), opakování ztracených segmentů
- **Řízení toku** – sliding window; zabraňuje zahlcení příjemce. Princip: příjemce řekne odesílateli, kolik dat najednou zvládne přijmout (*velikost okna*). Odesílatel může poslat právě tolik bajtů, aniž čeká na potvrzení každého paketu zvlášť – okno se „posouvá“ dopředu vždy, když přijde ACK. Větší okno = vyšší propustnost; pokud příjemce nestíhá, okno zmenší a odesílatel zpomalí.
- **Řízení přetížení** – slow start, congestion avoidance; zabraňuje zahlcení sítě

- **Multiplexování** – portová čísla (0–65535); dobře známé porty 0–1023

2.1.4 UDP (User Datagram Protocol)

Vrstva: Transportní (TCP/IP: Transport layer, OSI: vrstva 4)

UDP je nespolehlivý, nespojovaný protokol. Nezaručuje doručení ani pořadí paketů. Vhodný tam, kde je rychlost důležitější než spolehlivost:

- Streamování videa/audia (ztráta paketu = jen výpadek obrazu)
- DNS dotazy (rychlé, krátké)
- Online hry
- VoIP

2.1.5 ICMP (Internet Control Message Protocol)

Vrstva: Síťová (TCP/IP: Internet layer, OSI: vrstva 3)

ICMP je protokol síťové vrstvy pro zasílání chybových zpráv a diagnostiku sítě.

- **ping** – testuje dosažitelnost hosta (ICMP Echo Request/Reply)
- **traceroute** – sleduje cestu paketu přes síť (TTL expiry zprávy)
- Chybové zprávy: Destination Unreachable, Time Exceeded, Redirect

2.1.6 DNS (Domain Name System)

Vrstva: Aplikační (TCP/IP: Application layer, OSI: vrstva 7)

DNS překládá doménová jména na IP adresy (a zpět – reverzní DNS).

Hierarchická struktura DNS

- **Root DNS** – kořenové servery (13 skupin, označeny A–M)
- **TLD (Top Level Domain)** – .cz, .com, .org, .net, .edu
- **Autoritativní DNS server** – zná záznamy pro danou doménu
- **Rekurzivní resolver** – ptá se jménem klienta (typicky DNS poskytovatele); funguje jako „prostředník“: klient mu pošle dotaz a on za něj celý řetězec dotazů (root → TLD → autoritativní server) vyřeší sám a vrátí klientovi hotovou odpověď. Klient tedy nemusí vědět nic o hierarchii DNS – resolver udělá všechnu práci za něj.

Typy DNS záznamů

- **A** – doménové jméno → IPv4 adresa
- **AAAA** – doménové jméno → IPv6 adresa
- **CNAME** – alias pro jiné jméno
- **MX** – poštovní server pro doménu
- **NS** (Name Server) – určuje, který DNS server je autoritativní pro danou doménu; při resoluci TLD server vrátí NS záznam, který říká „pro tuto doménu se ptejte tohoto serveru“
- **PTR** – reverzní lookup (IP → jméno)
- **TXT** – libovolný textový záznam; nejčastěji pro e-mailové zabezpečení:
 - **SPF** (Sender Policy Framework) – definuje, které IP adresy smějí odesílat e-maily za danou doménu; přijímající server ověří, zda je odesílatel v seznamu
 - **DKIM** (DomainKeys Identified Mail) – e-mail je podepsán privátním klíčem domény; přijímající server ověří podpis pomocí veřejného klíče uloženého právě v TXT záznamu

DNS resoluce

1. Klient → lokální cache / hosts soubor
2. Klient → rekurzivní resolver (ISP)
3. Resolver → root server (odkaz na TLD server)
4. Resolver → TLD server (odkaz na autoritativní server)
5. Resolver → autoritativní server (IP adresa)
6. Resolver → klient (odpověď s IP)

DNS Cache Poisoning a DNSSEC

- **DNS Cache Poisoning** – útočník vloží falešný záznam do cache resolveru; oběti jsou přesměrovány na podvodné stránky (pharming)
- **DNSSEC (DNS Security Extensions)** – digitálně podepisuje DNS záznamy; resolver ověří podpis autoritativního serveru; brání cache poisoningu
- DNSSEC přidává záznamy: RRSIG (podpis), DNSKEY (veřejný klíč), DS (delegation signer)

2.2 Aplikační protokoly – SSH, E-mail, WWW

2.2.1 SSH (Secure Shell)

SSH je kryptografický síťový protokol pro bezpečný vzdálený přístup a přenos dat. Port 22.

Funkce SSH

- **Vzdálené přihlášení** – bezpečná náhrada za telnet a rlogin
- **SCP/SFTP** – bezpečný přenos souborů
- **Port forwarding (tunelování)** – přesměrování TCP portů přes šifrovaný kanál
- **X11 forwarding** – vzdálené spouštění grafických aplikací

Autentizace v SSH

- **Heslem** – nejjednodušší, náchylné na brute-force
- **Veřejným klíčem** – asymetrická kryptografie; klient má privátní klíč, server veřejný
- **Certifikátem** – vydán certifikační autoritou

2.2.2 Architektura e-mailové komunikace

Přenos e-mailu zahrnuje tři typy komponent:

- **MUA (Mail User Agent)** – poštovní klient (Outlook, Thunderbird, Gmail webové rozhraní); uživatel pomocí MUA píše a čte e-maily
- **MTA (Mail Transfer Agent)** – poštovní server zajišťující přenos zpráv mezi servery (Postfix, Sendmail, Exchange); přijímá od MUA a směřuje k cílovému MTA
- **MDA (Mail Delivery Agent)** – doručí zprávu do schránky příjemce na serveru; MUA si zprávu stáhne přes POP3 nebo IMAP

Tok zprávy: MUA (odesílatel) → MTA (odchozí) $\xrightarrow{\text{SMTP}}$ MTA (příchozí) → MDA → schránka $\xrightarrow{\text{POP3/IMAP}}$ MUA (příjemce).

2.2.3 E-mail – SMTP, POP3, IMAP

SMTP (Simple Mail Transfer Protocol)

- Port 25 (server-to-server), 587 (klient-server s autentizací), 465 (SMTPS)
- Zajišťuje **odesílání** e-mailů z klienta na server a přenos mezi servery
- Příkazy:

- EHLO – pozdrav a identifikace odesílajícího serveru (Extended HELO)
- MAIL FROM – adresa odesílatele (obálka zprávy)
- RCPT TO – adresa příjemce; opakuje se pro více příjemců
- DATA – zahájí přenos těla e-mailu; ukončí se tečkou na samostatném řádku
- QUIT – ukončení SMTP spojení

POP3 (Post Office Protocol 3)

- Port 110 (nešifrovaný), 995 (POP3S)
- Stahuje e-maily ze serveru **do lokálního klienta** a standardně je ze serveru maže
- Vhodné pro jeden zařízení, offline přístup

IMAP (Internet Message Access Protocol)

- Port 143 (nešifrovaný), 993 (IMAPS)
- E-maily zůstávají na serveru; klient pracuje s e-maily online
- Synchronizace stavu (přečteno, označeno) napříč zařízeními
- Vhodné pro přístup z více zařízení (preferované)

Vlastnost	POP3	IMAP
Uložení zpráv	Lokálně	Na serveru
Více zařízení	Komplikované	Podporováno nativně
Synchronizace	Žádná	Plná
Offline přístup	Ano	Možný (s cachingem)
Nároky na server	Nízké	Vyšší

2.2.4 Zabezpečení e-mailu

- **SPF (Sender Policy Framework)** – TXT DNS záznam; definuje povolené odesílající IP servery
- **DKIM (DomainKeys Identified Mail)** – digitální podpis e-mailu klíčem domény
- **DMARC** – politika kombinující SPF a DKIM; určuje akci při selhání (none, quarantine, reject)
- **S/MIME, PGP** – end-to-end šifrování obsahu e-mailu

2.3 Webové technologie – HTTP, cookies, session

2.3.1 HTTP (Hypertext Transfer Protocol)

HTTP je aplikační protokol pro přenos hypertextových dokumentů. Základ webu. Port 80 (HTTP), 443 (HTTPS).

HTTP metody

- **GET** – získání zdroje; bez těla; idempotentní; lze cachovat
- **POST** – odeslání dat na server (vytvoření záznamu); má tělo; není idempotentní
- **PUT** – vytvoření nebo nahrazení zdroje; idempotentní
- **PATCH** – částečná aktualizace zdroje
- **DELETE** – smazání zdroje; idempotentní
- **HEAD** – jako GET, ale bez těla odpovědi
- **OPTIONS** – zjištění povolených metod pro daný zdroj

Idempotentní metoda = opakované volání se stejnými parametry vždy vrátí stejný výsledek a nezmění stav serveru opakovaně. Např. **DELETE /user/5** – ať zavoláš jednou nebo desetkrát, výsledek je stejný (uživatel je smazán). Naproti tomu **POST /objednavky** každým voláním vytvoří novou objednávku – není idempotentní.

HTTP stavové kódy

- **1xx** – informační (100 Continue, 101 Switching Protocols)
- **2xx** – úspěch (200 OK, 201 Created, 204 No Content)
- **3xx** – přesměrování (301 Moved Permanently, 302 Found, 304 Not Modified)
- **4xx** – chyba klienta (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found)
- **5xx** – chyba serveru (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable)

HTTP verze

- **HTTP/1.0** – každý požadavek = nové TCP spojení
- **HTTP/1.1** – persistentní spojení (keep-alive); **chunked transfer encoding** – server může posílat odpověď po částech (chuncích), aniž by předem znal celkovou velikost; vhodné pro dynamicky generovaný obsah nebo streaming
- **HTTP/2** – jedno TCP spojení, ale přes něj běží více požadavků současně (*multiplexing*) – odstraňuje problém „head-of-line blocking“ z HTTP/1.1; hlavičky jsou komprimovány (HPACK), takže opakující se hlavičky nezabírají zbytečně místo;

server může proaktivně poslat zdroje klientovi dříve, než si o ně požádá (*server push*)

- **HTTP/3** – místo TCP používá **QUIC** (protokol nad UDP); QUIC řeší hlavní slabinu TCP: při ztrátě paketu v TCP čeká celé spojení, v QUIC je každý stream nezávislý, takže ztráta jednoho paketu nezastaví ostatní; výsledkem je nižší latence, zejména na nestabilních sítích (mobilní připojení)

2.3.2 HTTPS a TLS

HTTPS = HTTP přes TLS (Transport Layer Security).

- Zajišťuje: důvěrnost, integritu dat, autentizaci serveru (certifikátem)
- **TLS handshake**: výměna certifikátů, dohodnutí šifrovacích algoritmů, výměna klíčů

2.3.3 Cookies

HTTP cookie je malý kus dat uložený v prohlížeči a odesílaný s každým HTTP požadavkem na daný server.

Vlastnosti cookies

- **Expires/Max-Age** – expirace; bez nich = session cookie (smazán po zavření prohlížeče)
- **Secure** – odesílán pouze přes HTTPS
- **HttpOnly** – nepřístupný JavaScriptu (ochrana před **XSS** – Cross-Site Scripting: útočník vloží škodlivý JS kód do stránky, který by jinak mohl cookie přečíst a odeslat na cizí server)
- **SameSite** – ochrana před **CSRF** (Cross-Site Request Forgery: útočník přiměje přihlášeného uživatele, aby nevědomky odeslal požadavek na cizí web – prohlížeč by jinak cookie automaticky přiložil; SameSite=Strict/Lax cookie nepřiloží u cross-site požadavků); hodnoty: Strict, Lax, None
- **Domain/Path** – omezení platnosti na doménu/cestu

Použití cookies

- Správa sezení (session management) – identifikátor přihlášeného uživatele
- Personalizace – uložení preferencí uživatele
- Sledování (tracking) – analýza chování uživatele

2.3.4 Session (relace)

HTTP je bezstavový protokol – každý požadavek je nezávislý. Session řeší udržování stavu:

1. Po přihlášení server vytvoří session s unikátním ID
2. Session ID se uloží do cookie (nebo URL parametru)
3. Při každém požadavku se session ID posílá serveru
4. Server dohledá session data (uložena v paměti nebo DB)

Úložiště session dat na serveru

- **Paměť serveru** – rychlé, ale ztráta při restartu; problém u více serverů (load balancing)
- **Sdílená cache** – Redis, Memcached; vhodné pro distribuované systémy
- **Databáze** – perzistentní, ale pomalejší

2.3.5 Client-side vs. Server-side zpracování

Server-side rendering (SSR)

- HTML generováno na serveru (PHP, Java/Spring, Python/Django, ASP.NET)
- Prohlížeč dostane hotové HTML
- **Výhody:** SEO-friendly, rychlé načtení první stránky, vhodné pro starší prohlížeče
- **Nevýhody:** vyšší zátěž serveru, pomalejší interakce

Client-side rendering (CSR)

- JavaScript stáhne data přes API (REST, GraphQL) a dynamicky vytvoří HTML v prohlížeči
- Frameworky: React, Vue.js, Angular
- **Výhody:** bohatá interaktivita, nižší zátěž serveru po načtení
- **Nevýhody:** pomalejší první načtení, složitější SEO

REST API Architekturní styl pro návrh distribuovaných systémů – systémů složených z více samostatných komponent (serverů, služeb), které spolu komunikují přes síť. Každá komponenta může běžet nezávisle na jiném stroji; REST definuje, jak spolu komunikují přes HTTP.

- Bezstavovost (statelessness) – server neukládá stav klienta; každý požadavek musí obsahovat vše potřebné (autentizační token apod.)
- Identifikace zdrojů přes URI (např. `/users/42`)

- **Využití HTTP metod sémanticky** – každá metoda má jasný význam odpovídající operaci nad zdrojem: GET = čti, POST = vytvoř, PUT = nahraď, PATCH = uprav, DELETE = smaž; díky tomu je API předvídatelné a samodokumentující – z metody a URI je ihned jasné, co volání dělá
- JSON nebo XML jako formát dat

Otázky profesorů k tématu: IKT

Komise: Svátek, Chlapek, Zbořil

- Jak funguje překlad doménového jména na IP adresu (DNS)?
- Jaký je rozdíl mezi TCP a UDP?

Zeman Václav, Ing., Ph.D.

- Vysvětlíte 3-way handshake v TCP.
- Jaký je rozdíl mezi POP3 a IMAP?
- Jak fungují cookies a k čemu slouží?

Chudán David, Ing., Ph.D.

- Co je HTTP a jaké jsou jeho hlavní metody?
- Jak probíhá komunikace mezi webovým prohlížečem a serverem?
- Co je session a jak se implementuje v HTTP?

Sedláček Jiří, Ing., Ph.D.

- Jaké jsou bezpečnostní mechanismy pro e-mailovou komunikaci?
- Jak SSH zajišťuje autentizaci?

3 Programování v Javě a objektově-orientované paradigma

3.1 Objektově-orientované programování (OOP)

3.1.1 Základní principy OOP

Objektově-orientované programování je programovací paradigma organizující software kolem objektů, které kombinují data (atributy/pole) a chování (metody).

Lidsky řečeno: Místo toho, abys psal program jako jeden dlouhý seznam příkazů, OOP tě nutí přemýšlet o světě jako o věcech (objektech), které mají vlastnosti a umějí něco dělat. Auto má značku, barvu a počet dveří (data) – a umí nastartovat, zastavit a zatočit (metody). V programu pak vytvoříš třídu `Auto` jako šablonu a z ní libovolně mnoho konkrétních aut (objektů). Každé auto si pamatuje své vlastní hodnoty, ale sdílí stejné chování. Díky tomu je kód přehledný, znovupoužitelný a snáz se v něm orientuje víc lidí.

1. Abstrakce (Abstraction) Proces skrytí složitých implementačních detailů a zobrazení pouze nezbytné funkcionality.

- Zaměřuje se na *co* objekt dělá, ne *jak* to dělá
- Realizuje se pomocí abstraktních tříd (`abstract class`) a rozhraní (`interface`)
- Příklad: rozhraní `List` v Javě – programátor pracuje s metodami `add()`, `get()`, `remove()`, aniž ví, zda jde o `ArrayList` nebo `LinkedList`

Listing 1: Abstrakce – rozhraní `Tvar`

```
// Abstrakce: definujeme CO umí tvar, ne JAK to dělá
interface Tvar {
    double obsah();           // každý tvar musí umět spočítat obsah
    double obvod();
    String nazev();
}

// Konkrétní implementace -- SKRYTÉ od uživatele
class Kruh implements Tvar {
    private double polomer;
    public Kruh(double r) { this.polomer = r; }

    @Override public double obsah() { return Math.PI * polomer *
        polomer; }
    @Override public double obvod() { return 2 * Math.PI * polomer;
    }
    @Override public String nazev() { return "Kruh"; }
}

class Obdelnik implements Tvar {
    private double a, b;
```

```

    public Obdelnik(double a, double b) { this.a = a; this.b = b; }

    @Override public double obsah() { return a * b; }
    @Override public double obvod() { return 2 * (a + b); }
    @Override public String nazev() { return "Obdelnik"; }
}

// Klientský kód pracuje POUZE s abstrakcí Tvar
// nezajímá ho, co je uvnitř
public static void vypisTvary(List<Tvar> tvar) {
    for (Tvar t : tvar) {
        System.out.printf("%s: □ obsah=%.2f%n", t.nazev(), t.obsah());
    }
}
}

```

Proč je to abstrakce? Funkce `vypisTvary` neví, zda pracuje s kruhem nebo obdélníkem – pracuje pouze s rozhraním `Tvar`. Implementační detaily (vzorec pro obsah kruhu) jsou skryté za rozhraním. Lze přidat nový tvar (Trojuhelník) **bez jakékoliv změny** v `vypisTvary`.

2. Zapouzdření (Encapsulation) Sdružení dat a metod, které s nimi pracují, do jednoho celku (třídy) a omezení přístupu zvenčí.

- Data (atributy) by měla být `private`, přístup přes `public` gettery/settery
- Chrání interní stav objektu před neplatným nastavením
- **Modifikátory přístupu v Javě:**
 - `private` – přístup pouze z dané třídy
 - `protected` – přístup z třídy a podtříd (**podtřída** = třída, která dědí z jiné třídy; v Javě `class Pes extends Zvire`, v C# přesně `class Pes : Zvire` – syntaxe se liší, princip je stejný)
 - `package-private` (výchozí) – přístup v rámci balíčku
 - `public` – přístup odkudkoliv

Listing 2: Zapouzdření v Javě

```

public class BankovniUcet {
    private double zustatek;    // soukromé pole

    public double getZustatek() {
        return zustatek;
    }

    public void vlozit(double castka) {
        if (castka > 0) zustatek += castka;
        // validace vstupu pred zmenou stavu
    }
}

```

3. Dědičnost (Inheritance) Mechanismus, který umožňuje nové třídě přebírat vlastnosti a metody existující (nadřazené) třídy.

- Klíčové slovo **extends** (třída) nebo **implements** (rozhraní) – rozdíl: **extends** použiješ, když dědiš z **konkrétní nebo abstraktní třídy** (přebíráš hotový kód, atributy, chování); **implements** použiješ, když třída plní **rozhraní** (interface) – rozhraní je jen „smlouva“ co třída musí umět, bez jakékoliv implementace. V C# se obojí píše jako :, Java to rozlišuje syntakticky.
- Java podporuje jednoduchou dědičnost tříd (jen jeden **extends**), ale vícenásobnou implementaci rozhraní (**implements A, B, C**)
- **super** – odkaz na nadřazenou třídu
- **IS-A** vztah (pes IS-A zvíře) – říká, že podtřída *je druhem* nadtřídy; dědičnost je správná právě tehdy, když platí IS-A: Pes **extends** Zvire dává smysl, protože pes opravdu *je* zvíře. Pokud IS-A neplatí, dědičnost nepoužívej.
- **HAS-A** vztah (auto HAS-A motor) – kompozice; třída *obsahuje* jinou třídu jako atribut, místo aby z ní dědila. Auto nemá *být* motor, ale *mít* ho (**private Motor motor**;). Kompozice je obecně preferována před dědičností – méně těsné propojení.
- **Nevýhody dědičnosti:** těsné propojení (coupling) tříd; změna v rodičovské třídě může rozbít potomky

4. Polymorfismus (Polymorphism) Schopnost různých objektů reagovat různým způsobem na stejné zprávy (volání metod).

- **Compile-time polymorfismus** – přetížení metod (method overloading); stejný název metody, různé parametry
- **Runtime polymorfismus** – přepsání metody (method overriding); podtřída překryje metodu nadtřídy; rozhodnutí za běhu programu
- **Liskovův substituční princip (LSP):** objekty podtřídy musejí být zaměnitelné za objekty nadtřídy

Listing 3: Polymorfismus v Javě

```
abstract class Zvire {
    abstract void vydejZvuk();
}

class Pes extends Zvire {
    @Override
    public void vydejZvuk() { System.out.println("Haf!"); }
}

class Kocka extends Zvire {
    @Override
    public void vydejZvuk() { System.out.println("Mnau!"); }
}

// Runtime polymorfismus
```



```
Zvire z = new Pes();  
z.vydejZvuk(); // vypise "Haf!" -- rozhodnutí za běhu
```

3.1.2 Třída vs. instance

- **Třída (Class)** – šablona/plán; definuje strukturu a chování; na disku jako `.class` soubor
- **Instance (Object)** – konkrétní výskyt třídy vytvořený v paměti pomocí `new`
- **Konstruktor** – speciální metoda volaná při vytváření instance; stejný název jako třída, bez návratového typu

3.1.3 Abstraktní třídy vs. rozhraní

Co je abstraktní třída? Abstraktní třída je třída, od které **nelze přímo vytvořit instanci** (`new` nad ní selže). Slouží jako **neúplná šablona** – definuje společné chování potomků, ale části nechává k implementaci v podtřídách. Používá se, když třídy sdílejí **implementační kód i stav** (atributy).

Listing 4: Abstraktní třída

```
abstract class Vozidlo {  
    protected String znacka;        // sdílený stav  
    protected int rokVyroby;  
  
    public Vozidlo(String znacka, int rok) {  
        this.znacka = znacka;  
        this.rokVyroby = rok;  
    }  
  
    // Konkrétní metoda -- sdílena všemi potomky  
    public int stariVozu() {  
        return 2024 - rokVyroby;  
    }  
  
    // Abstraktní metoda -- každý potomek musí implementovat  
    public abstract String typPaliva();  
}  
  
class Auto extends Vozidlo {  
    public Auto(String znacka, int rok) { super(znacka, rok); }  
    @Override public String typPaliva() { return "Benzin/nafta/  
        elektro"; }  
}
```

Co je rozhraní (interface)? Rozhraní je **čistý kontrakt** – říká “tato třída UMĚT dělat X”, ale vůbec neříká jak. Neobsahuje stav (žádné instanční proměnné). Třída může

implementovat **libovolný počet** rozhraní. Používá se pro definování schopností nezávislých na hierarchii tříd.

Listing 5: Rozhraní

```
interface Elektricke {
    void nabijBaterii();
    int getKapacitaBaterie();    // kWh
}

interface GPS {
    double getSirka();
    double getDelka();
}

// Třída může implementovat více rozhraní zároveň
class ElektrickeAuto extends Auto implements Elektricke, GPS {
    private int kapacita;
    private double lat, lon;

    @Override public void nabijBaterii() { /* ... */ }
    @Override public int getKapacitaBaterie() { return kapacita; }
    @Override public double getSirka() { return lat; }
    @Override public double getDelka() { return lon; }
}
```

Kdy použít co?

Abstraktní třída	Rozhraní
Třídy jsou stejným typem (IS-A): Pes IS-A Zvire	Třídy umí totéž (CAN-DO): Auto CAN-DO Serializable
Sdílí kód i stav (atributy)	Definují pouze kontrakt (co umí)
Potřebujeme konstruktor nebo protected pole	Přidáváme schopnosti napříč hierarchiemi
Pouze jedna abstraktní třída	Více rozhraní najednou

3.1.4 Principy SOLID

- **S** – Single Responsibility Principle: třída má mít pouze jeden důvod ke změně. Každá třída dělá jednu věc a dělá ji dobře. Třída `Invoice` počítá cenu faktury – ale tisk faktury do PDF má mít vlastní třídu `InvoicePrinter`. Pokud by se změnil formát PDF, nemusíš sahat do logiky výpočtu ceny.
- **O** – Open/Closed Principle: třída je otevřena pro rozšíření, ale uzavřena pro modifikaci. Přidáš nové chování přes nové třídy (dědičností nebo kompozicí), ne tím, že

přepisuješ stávající kód. Příklad: místo přidání `if (typ == "kruh")` do existující metody vytvoříš novou třídu `Kruh`, která implementuje rozhraní `Tvar`.

- **L** – Liskov Substitution Principle: podtřídy musejí být plně zaměnitelné za nadtřídy – kód, který pracuje s `Zvire`, musí fungovat stejně správně i když mu předáš `Pes` nebo `Kocka`. Pokud podtřída mění nebo ruší chování rodičovské třídy (háže výjimku tam, kde rodič nic nedělá), porušuje LSP.
- **I** – Interface Segregation Principle: raději víc malých specifických rozhraní než jedno velké „all-in-one“. Třída by neměla být nucena implementovat metody, které nepotřebuje. Místo jednoho rozhraní `Stroj` s metodami `tiskni()`, `skenuj()`, `faxuj()` udělej tři samostatná – tiskárna pak implementuje jen `Tiskatelny`, ne zbylé dvě.
- **D** – Dependency Inversion Principle: třídy vysoké úrovně by neměly záviset na třídách nízké úrovně – oboje by mělo záviset na abstrakcích (rozhraních). Příklad: `ObjednavkoveService` nemá přímo vytvářet `new MySQLDatabase()`, ale dostat rozhraní `Database` zvenčí (injekce závislosti). Snadno tak vyměníš `MySQL` za jinou DB bez dotyku servisní logiky.

3.2 Test Driven Development (TDD)

3.2.1 Principy TDD

TDD je vývojová metodika, při níž se testy píšou **před** samotným produkčním kódem. Cyklus TDD: **Red** – **Green** – **Refactor**

1. **Red** – napsat test, který selže (testuje chování, které ještě neexistuje)
2. **Green** – napsat minimální kód tak, aby test prošel
3. **Refactor** – vylepšit kód bez změny chování (testy stále musí procházet)

3.2.2 Typy testů

- **Unit testy** – testují izolovanou jednotku (třídu, metodu); mock objekty pro závislosti
- **Integrační testy** – testují spolupráci více komponent (např. DB + služba)
- **End-to-end (E2E) testy** – testují celý systém z pohledu uživatele

Testovací pyramida: mnoho unit testů, méně integračních, nejméně E2E.

3.2.3 JUnit 5 v Javě

Listing 6: Unit test s JUnit 5

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class KalkulackaTest {
```

```

private Kalkulacka calc;

@BeforeEach
void setUp() {
    calc = new Kalkulacka();
}

@Test
void testScitani() {
    assertEquals(5, calc.secti(2, 3));
}

@Test
void testDeleniNulou() {
    assertThrows(ArithmeticException.class,
        () -> calc.vydel(10, 0));
}
}

```

3.2.4 Mockování závislostí

Mock objekty simulují chování závislostí (DB, externích API) při unit testech. Knihovna **Mockito** v Javě:

Listing 7: Mockování s Mockito

```

@Mock
private UzivatelskyRepository repo;

@Test
void testNajdiUzivatele() {
    when(repo.findById(1L))
        .thenReturn(Optional.of(new Uzivatel("Jan")));
    Uzivatel u = service.najdi(1L);
    assertEquals("Jan", u.getJmeno());
    verify(repo).findById(1L);
}

```

3.3 Lambda výrazy a Stream API

3.3.1 Funkcionální rozhraní a lambda výrazy

Lambda výraz je anonymní funkce – blok kódu, který lze předat jako argument nebo uložit do proměnné. Zavedeny v Javě 8. Syntaxe: `(parametry) -> {tělo}`

Lidsky: Před Javou 8 jsi musel pro každou „malou akci“ psát celou anonymní třídu (viz příklad níže). Lambda je zkratka – místo tří řádků boilerplate napíšeš jednu řádku. Jde vlastně o způsob, jak předat *chování* (funkci) jako hodnotu, stejně jako předáváš číslo nebo řetězec.

Funkcionální rozhraní – rozhraní s právě jednou abstraktní metodou (označuje se `@FunctionalInterface`). Lambda výraz *je* implementace takového rozhraní – Java ví, kterou jedinou metodu má lambda naplnit.

Zabudovaná funkcionální rozhraní – Java 8 dodává sadu hotových rozhraní pro nejčastější případy, abys nemusel psát vlastní. Tabulka níže ukazuje co každé „očekává“ a co „vrací“:

Rozhraní	Vstup → Výstup	Použití	Příklad
<code>Function<T,R></code>	$T \rightarrow R$	Transformace hodnoty	<code>s -> s.length()</code>
<code>Predicate<T></code>	$T \rightarrow \text{boolean}$	Filtrování (ano/ne)	<code>s -> s.isEmpty()</code>
<code>Consumer<T></code>	$T \rightarrow \text{nic}$	Spotřebování, vedlejší efekt	<code>s -> System.out.println</code>
<code>Supplier<T></code>	$\text{nic} \rightarrow T$	Produkce hodnoty	<code>() -> new ArrayList<>()</code>
<code>BiFunction<T,U,R></code>	$T, U \rightarrow R$	Transformace dvou hodnot	<code>(a,b) -> a + b</code>

Jak číst tabulku: `Function` vezme jednu věc a vrátí jinou (např. ze `String` udělá `Integer` – délku). `Predicate` vezme věc a řekne pravda/nepravda – proto se používá ve `filter()`. `Consumer` věc vezme, ale nic nevrátí – jen ji “spotřebuje” (vypíše, uloží...). `Supplier` naopak nic nebere, ale hodnotu *vyrobí*. `BiFunction` je jako `Function`, ale bere dva vstupy místo jednoho.

Listing 8: Lambda výrazy

```
// Starý způsob - anonymní třída
Comparator<String> comp1 = new Comparator<String>() {
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
};

// Lambda výraz
Comparator<String> comp2 = (a, b) -> a.compareTo(b);

// Method reference
Comparator<String> comp3 = String::compareTo;
```

3.3.2 Stream API

Stream je *pipeline* pro zpracování kolekce dat funkcionálním stylem – místo imperativního cyklu `for` popisujeme, co chceme s daty udělat jako řetěz operací.

Klíčové vlastnosti:

- **Lazy evaluation** – intermediární operace (`filter`, `map` ...) se *nevyhodnocují hned* – jen popíší, co se má stát. Skutečné zpracování proběhne až ve chvíli, kdy přijde terminální operace (`collect`, `findFirst` ...). **Proč je to dobré?** Představ si seznam milionu čísel a chceš první sudé větší než 100. Bez lazy evaluation bys profiltroval všech milion prvků a pak vzal první výsledek. S lazy evaluation stream okamžitě zastaví, jakmile najde první shodu – zbytek vůbec nezpracuje.

- **Spotřeba** – stream lze projít pouze jednou; po terminální operaci je “vyčerpaný”
- **Nemodifikuje zdroj** – stream nikdy nemění původní kolekci, ze které vznikl; `filter()` nebo `map()` vždy vrátí *nový* stream s novými daty. Původní `List` zůstane nedotčený. **Příklad:** zavolaš `seznam.stream().filter(...).collect(...)` – seznam má pořád stejné prvky jako předtím, výsledek filtrování je v nové proměnné.
- **Paralelizovatelný** – `parallelStream()` automaticky rozdělí práci mezi vlákna (Fork/Join pool)

Operace na streamech

- **Intermediární (lazy)** – vracejí nový stream: `filter()`, `map()`, `flatMap()`, `sorted()`, `distinct()`, `limit()`, `peek()`

Rozdíl map vs flatMap: `map()` transformuje každý prvek na jeden výsledek – ze streamu `N` prvků vznikne stream `N` prvků. `flatMap()` transformuje každý prvek na *stream* výsledků a pak všechny tyto streamy „zploští“ do jednoho – použiješ ho, když by `map()` vrátil stream streamů. Příklad: máš seznam vět a chceš stream jednotlivých slov – `map` by vrátil `Stream<String[]>` (každá věta jako pole), `flatMap` vrátí rovnou `Stream<String>` (všechna slova v jednom streamu):

```
List<String> vety = List.of("ahoj_svete", "jak_se_mas");

// map -- Stream<String[]>, vnořené pole
vety.stream().map(v -> v.split("_"));

// flatMap -- Stream<String>, plochý seznam slov
vety.stream()
    .flatMap(v -> Arrays.stream(v.split("_")))
    .collect(Collectors.toList());
// ["ahoj", "svete", "jak", "se", "mas"]
```

- **Terminální (eager)** – vrátí výsledek nebo vedlejší efekt: `collect()`, `forEach()`, `reduce()`, `count()`, `findFirst()`, `anyMatch()`, `allMatch()`

Listing 9: Stream API příklady

```
List<String> jmena = Arrays.asList("Alice", "Bob", "Charlie", "David");

// Filtrování a transformace
List<String> vysledek = jmena.stream()
    .filter(j -> j.length() > 3)           // Alice, Charlie, David
    .map(String::toUpperCase)             // ALICE, CHARLIE, DAVID
    .sorted()                             // řazení
    .collect(Collectors.toList());

// Redukce
int soucet = IntStream.rangeClosed(1, 10)
    .reduce(0, Integer::sum); // 55
```

```
// Seskupení
Map<Integer, List<String>> podleDlky = jmena.stream()
    .collect(Collectors.groupingBy(String::length));

// Paralelní stream (pro velká data)
long pocet = jmena.parallelStream()
    .filter(j -> j.startsWith("A"))
    .count();
```

Srovnání Java Stream API s C# LINQ Pokud znáte .NET, Stream API je ekvivalentem **LINQ** (Language Integrated Query). Koncepty jsou totožné, liší se pouze syntaxí:

Listing 10: Java Stream API vs. C# LINQ – srovnání

```
// === JAVA (Stream API) ===
List<String> jmena = List.of("Alice", "Bob", "Charlie", "David");

List<String> vysledek = jmena.stream()
    .filter(j -> j.length() > 3)      // jako Where() v C#
    .map(String::toUpperCase)        // jako Select() v C#
    .sorted()
    .collect(Collectors.toList());    // jako ToList() v C#

// Průměrný věk dospělých
double prumer = osoby.stream()
    .filter(o -> o.getVek() >= 18)
    .mapToInt(Osoba::getVek)          // speciální IntStream pro int
    .average()
    .orElse(0.0);

// === C# LINQ (ekvivalent) ===
// var vysledek = jmena
//     .Where(j => j.Length > 3)
//     .Select(j => j.ToUpper())
//     .OrderBy(j => j)
//     .ToList();
//
// double prumer = osoby
//     .Where(o => o.Vek >= 18)
//     .Average(o => o.Vek);
```

Hlavní rozdíly Java Stream vs. C# LINQ:

- V Javě je `stream()` explicitní volání; v C# je LINQ přímo na `IEnumerable<T>`
- Java rozlišuje `Stream<T>`, `IntStream`, `LongStream`, `DoubleStream` (kvůli boxing); C# to řeší automaticky
- Terminální operace Javy: `collect(Collectors.toList())`; v C# jednoduše `.ToList()`
- Obě platformy podporují paralelní zpracování: Java `parallelStream()`, C# `AsParallel()` (PLINQ)

3.3.3 Optional

`Optional<T>` je kontejner pro hodnotu, která může být null – eliminuje `NullPointerException`.

Listing 11: Optional v Javě

```
Optional<String> opt = Optional.ofNullable(getJmeno());

// Bezpečná práce s hodnotou
String jmeno = opt.orElse("Neznámý");
opt.ifPresent(j -> System.out.println("Ahoj, " + j));
String velke = opt.map(String::toUpperCase).orElse("");
```

Otázky profesorů k tématu: Programování v Javě

Komise: Svátek, Chlapek, Zbořil

- Vysvětlete principy OOP – abstrakce, zapouzdření, dědičnost, polymorfismus.
- Jaký je rozdíl mezi abstraktní třídou a rozhraním?

Vojtěch Stanislav, Ing., Ph.D.

- Co je TDD a jaký je jeho postup?
- Jak byste testoval metodu, která závisí na externím API?

Chudán David, Ing., Ph.D.

- Co jsou lambda výrazy a jaké přinášejí výhody?
- Vysvětlete rozdíl mezi `map()` a `flatMap()` v Stream API.

Sládek Pavel, Ing., Ph.D.

- Jaké jsou principy SOLID a proč jsou důležité?
- Co je polymorfismus a jak se projevuje v Javě?

Zeman Václav, Ing., Ph.D.

- Jak se v Javě zajišťuje zapouzdření? Jaká jsou pravidla pro modifikátory přístupu?
- Proč je dědičnost považována za nebezpečnou a jaká je alternativa?

4 Softwarové inženýrství

4.1 UML diagramy

UML (**Unified Modeling Language**) je standardizovaný jazyk pro vizuální modelování softwarových systémů. Diagramy UML se dělí do dvou skupin:

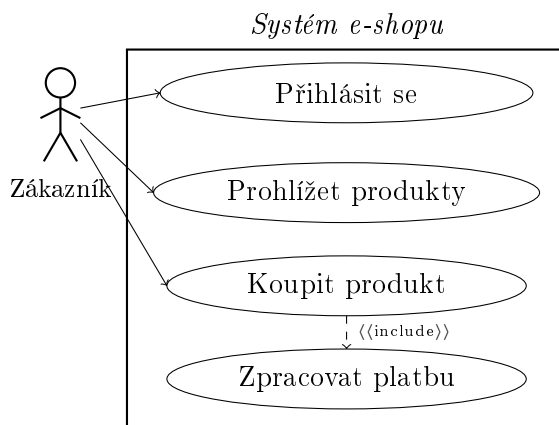
- **Strukturální diagramy** – statická struktura systému: class, object, component, deployment, package
- **Behaviorální diagramy** – dynamické chování: use case, sequence, activity, state machine, communication

4.1.1 Diagram případů užití (Use Case Diagram)

Zobrazuje interakce mezi systémem a jeho aktéry. Odpovídá na otázku: *Co systém dělá?*

Elementy:

- **Aktér (Actor)** – role uživatele nebo externího systému; reprezentován panáčkem
- **Případ užití (Use Case)** – funkce systému z pohledu aktéra; reprezentován elipsou
- **Vztahy:**
 - *Include* (<<include>>) – vždy se zahrne základní UC při provádění tohoto
 - *Extend* (<<extend>>) – volitelně rozšiřuje základní UC za určitých podmínek
 - *Generalizace* – jeden aktér/UC je specializací jiného
- **Hranice systému** – obdélník oddělující systém od aktérů



Obrázek 6: Ukázka diagramu případů užití – e-shop

4.1.2 Diagram tříd (Class Diagram)

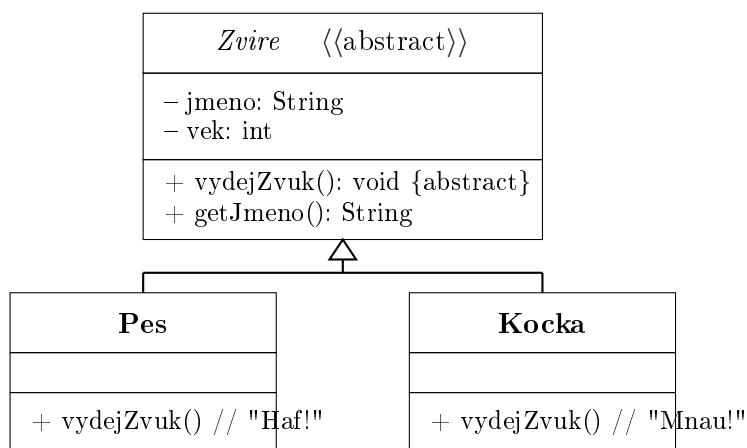
Zobrazuje třídy, jejich atributy, metody a vztahy mezi nimi. Základ strukturální analýzy.

Typy vztahů:

- **Asociace** – obecná vazba; čára mezi třídami; může mít multiplicitu (1, *, 0..1, 1..*)
Příklad: Student — Kurz (student se zapíše na kurz; oba existují nezávisle na sobě)
- **Agregace** – slabé vlastnictví (celek/část); část může existovat bez celku; prázdný diamant
Příklad: Tým ◇— Hráč (hráč patří do týmu, ale když se tým rozpadne, hráč stále existuje)
- **Kompozice** – silné vlastnictví; část nemůže existovat bez celku; plný diamant
Příklad: Objednávka ◆— PoložkaObjednávky (položka bez objednávky nedává smysl; zaniká s ní)
- **Dědičnost (Generalizace)** – IS-A vztah; prázdná trojúhelníková šipka na nadtřídu
Příklad: Pes → Zvíře (pes IS-A zvíře; dědí atributy `jmeno`, `vek`)
- **Realizace** – třída implementuje rozhraní; přerušovaná čára s prázdnou šipkou
Příklad: ArrayList implementuje List (odpovídá `implements` v Javě / `:` v C#)
- **Závislost (Dependency)** – slabá vazba; přerušovaná šipka; `<<use>>`; třída jen krátkodobě *používá* jinou (jako parametr metody, ne jako atribut)
Příklad: FakturaService --> PDFGenerator (service zavolá generátor jednou při tisku; nedrží ho trvale jako atribut)

Viditelnost atributů/metod:

- + public, - private, # protected, ~ package



Obrázek 7: Ukázka diagramu tříd – hierarchie zvířat

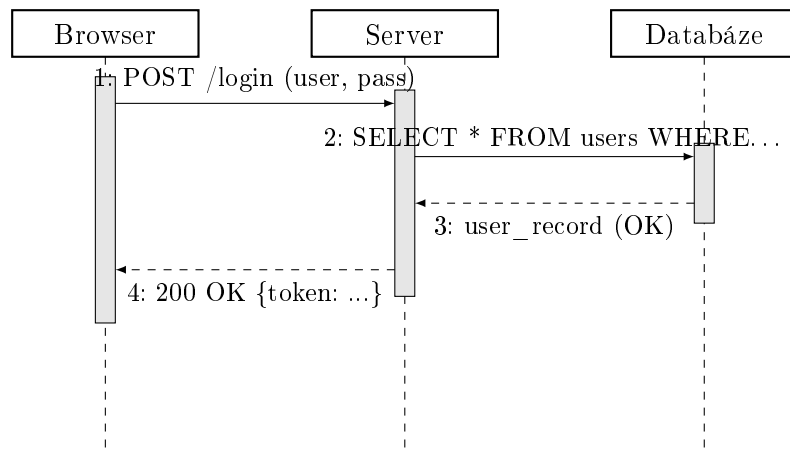
4.1.3 Sekvenční diagram (Sequence Diagram)

Zobrazuje pořadí zpráv mezi objekty v čase. Odpovídá na otázku: *Jak objekty spolupracují?*

Elementy:

- **Lifeline** – svislá přerušovaná čára; reprezentuje objekt/účastníka
- **Aktivační obdélník** – obdélník na lifeline; objekt je aktivní (zpracovává zprávu)

- **Zprávy** – šipky mezi lifelines; synchronní (plná šipka), asynchronní (prázdná šipka), návrat (přerušovaná)
- **Kombinované fragmenty:** *opt* (podmíněné), *alt* (alternativy), *loop* (opakování)



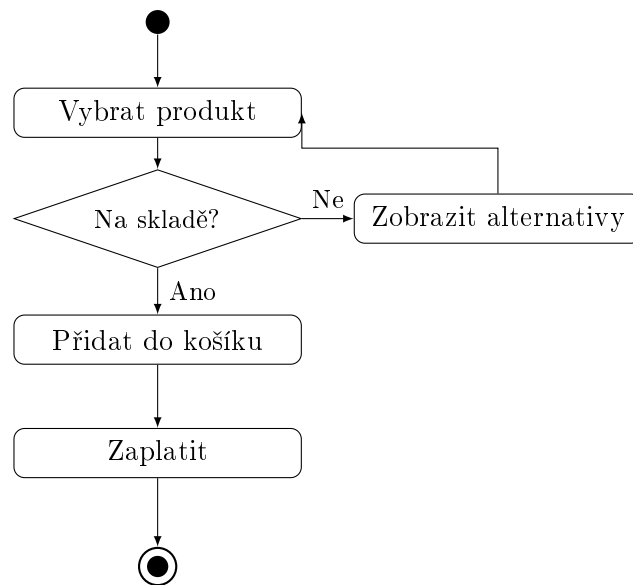
Obrázek 8: Ukázka sekvenčního diagramu – přihlášení uživatele

4.1.4 Diagram aktivit (Activity Diagram)

Zobrazuje tok řízení a dat v procesu nebo algoritmu. Podobný vývojovému diagramu.

Elementy:

- **Počáteční uzel** – plný kruh
- **Aktivita (Action)** – zaoblený obdélník
- **Rozhodovací uzel (Decision)** – diamant; podmíněné větvení
- **Fork/Join** – tučná vodorovná čára; paralelní větve
- **Swimlanes (plavecké dráhy)** – oddělení aktivit podle role/zodpovědnosti; diagram se rozdělí na svislé (nebo vodorovné) pruhy jako plavecký bazén – každý pruh patří jednomu aktérovi (zákazník, systém, banka) a aktivity v daném pruhu jsou zodpovědností tohoto aktéra. Na první pohled vidíš *kdo co dělá* a kde přechází zodpovědnost z jedné role na druhou. *Příklad:* proces objednávky – pruh „Zákazník“ obsahuje „Vybrat produkt“ a „Zaplatit“; pruh „Systém“ obsahuje „Ověřit sklad“ a „Vystavit fakturu“; pruh „Banka“ obsahuje „Autorizovat platbu“
- **Koncový uzel** – plný kruh s kroužkem



Obrázek 9: Ukázka diagramu aktivit – proces objednávky

4.2 Návrhové vzory (GoF Design Patterns)

Návrhové vzory jsou opakovaně použitelná řešení často se vyskytujících problémů při návrhu softwaru. Katalog 23 vzorů (Gang of Four, 1994) je rozdělen do tří skupin.

4.2.1 Tvořivé vzory (Creational Patterns)

Singleton Zajistí, že třída má pouze jednu instanci, a poskytne globální přístupový bod. *Lidsky*: Konfigurace aplikace – chceš jedno místo s nastavením, ne deset kopií, které by se mohly lišit.

Listing 12: Singleton

```

public class Konfigurace {
    private static Konfigurace instance;

    private Konfigurace() { }

    public static synchronized Konfigurace getInstance() {
        if (instance == null) instance = new Konfigurace();
        return instance;
    }
}

```

Factory Method Definuje rozhraní pro vytváření objektů, ale nechá podtřídy rozhodnout, které třídy instancovat. Odstraní závislost kódu na konkrétních třídách. *Lidsky*: Objednáš si „auto“ a továrna rozhodne, zda vyrobí Škodu nebo BMW – ty jen víš, že dostaneš auto, ne jak přesně vzniklo.

Abstract Factory Poskytuje rozhraní pro vytváření rodin příbuzných objektů bez specifikace jejich konkrétních tříd. Příklad: GUI toolkity (WindowsButton, MacButton implementují abstraktní Button). *Lidsky*: IKEA série – objednáš celou kolekci nábytku v jednom stylu; přepneš na jiný styl a dostaneš jinou sérii, ale rozhraní zůstane stejné.

Builder Odděluje konstrukci komplexního objektu od jeho reprezentace. Vhodné pro objekty s mnoha volitelných parametrů. *Lidsky*: Konfigurátor auta na webu – postupně volíš barvu, motor, výbavu; na konci zavoláš `build()` a dostaneš hotový objekt. Místo jednoho konstruktora s 15 parametry.

Prototype Vytváří nové objekty klonováním existujícího. *Lidsky*: Ctrl+C, Ctrl+V – zkopíruješ existující objekt a upravíš jen co potřebuješ, místo vytváření od nuly.

4.2.2 Strukturální vzory (Structural Patterns)

Adapter Umožní spolupráci tříd s nekompatibilními rozhraními. Obalí existující třídu novým rozhraním. Příklad: adaptér mezi evropskou a americkou zástrčkou. *Lidsky*: Cestovní redukce – máš českou zástrčku, potřebuješ ji zapojit do britské zásuvky; adaptér to přeloží, aniž cokoliv měníš na zástrčce nebo zásuvce.

Facade Poskytne zjednodušené rozhraní ke složitému subsystému. Příklad: třída `Computer` s metodou `start()`, která skryje inicializaci CPU, RAM, HDD. *Lidsky*: Dálkový ovladač TV – jedním tlačítkem zapneš TV, zesilovač i přijímač; nemusíš vědět, jak každé zařízení funguje uvnitř.

Decorator Dynamicky přidá objektu nové chování bez modifikace jeho třídy. Obalí objekt dekorátorem. Příklad: `BufferedReader` obalí `FileReader` a přidá bufferování. *Lidsky*: Káva s příchutěmi – začneš s espressem, obalíš ho dekorátorem „+mléko“, pak dekorátorem „+šlehačka“; každé obalení přidá vrstvu chování, originální espresso se nemění.

Composite Složí objekty do stromové struktury; klienti s nimi pracují uniformně (jedna metoda pro list i uzel). *Lidsky*: Složky v souborovém systému – složka i soubor mají stejné operace (smazat, přesunout, zobrazit velikost); složka navíc obsahuje další soubory/složky, ale zvenku se chová stejně.

Proxy Zastupuje jiný objekt; řídí přístup k němu. Typy: vzdálený proxy, virtuální proxy, ochranný proxy. *Lidsky*: Recepce v hotelu – nepůjdeš rovnou do skladu, projdeš přes recepci, která ověří oprávnění, zaloguje přístup, nebo zařídí věc za tebe.

4.2.3 Vzory chování (Behavioral Patterns)

Observer (Pozorovatel) Definuje závislost 1:N; při změně stavu se notifikují všichni závislí pozorovatelé. Příklad: `EventListener` v GUI, MVC architektura (Model notifikuje View).

Listing 13: Observer

```
interface Pozorovatel {
    void update(String zprava);
}
class PredmetPozorovani {
    private List<Pozorovatel> pozorovatele = new ArrayList<>();
    public void pridat(Pozorovatel p) { pozorovatele.add(p); }
    public void notifikovat(String z) {
        pozorovatele.forEach(p -> p.update(z));
    }
}
```

Strategy (Strategie) Definuje rodinu algoritmů, zapouzdří je a umožní jejich změnu. Příklad: různé způsoby řazení (`BubbleSort`, `QuickSort`) implementující rozhraní `SortStrategy`. *Lidsky*: GPS navigace – zadáš cíl a vyberáš strategii: nejrychlejší trasa, nejkratší, bez dálnic; cíl je stejný, algoritmus výpočtu se zamění za běhu.

Command (Příkaz) Zapouzdří požadavek jako objekt; podporuje undo/redo, logování, fronta příkazů. *Lidsky*: `Ctrl+Z` v textovém editoru – každá akce (napsání slova, smazání) je objekt uložený v historii; undo jen vezme poslední příkaz a zavolá `undo()`.

Template Method Definuje kostru algoritmu v základní třídě a nechá podtřídy doplnit konkrétní kroky. *Lidsky*: Recept na pizzu – vždy: připrav těsto → přidej ingredience → upeč; co jsou za ingredience rozhoduje konkrétní podtřída (`Margherita`, `Hawai`). Kostra je v abstraktní třídě, detaily v podtřídách – tedy ano, je to v podstatě abstraktní třída s háčky pro podtřídy.

Iterator Poskytne sekvenční přístup k prvkům kolekce bez odhalení její interní struktury. *Lidsky*: Přesně tak – je to v podstatě `for-each` cyklus. Jedno rozhraní (`hasNext()`, `next()`) funguje stejně pro seznam, strom, frontu nebo databázový kurzor; nemusíš vědět, jak je kolekce uložena uvnitř.

4.3 Metodiky vývoje IS

4.3.1 Tradiční (rigorózní) metodiky

Vodopádový model (Waterfall) Sekvenční fáze: analýza → návrh → implementace → testování → nasazení.

- **Výhody:** jasné fáze, snadná správa, dokumentace
- **Nevýhody:** rigidní, obtížné reagovat na změny požadavků, testování až na konci

RUP (Rational Unified Process) Iterativní rámec; 4 fáze (Inception, Elaboration, Construction, Transition) s opakujícími se iteracemi.

4.3.2 Agilní metodiky

Agile Manifesto (2001) – 4 hodnoty a 12 principů agilního vývoje. Hodnoty: individuality nad procesy, funkční SW nad dokumentací, spolupráce se zákazníkem nad smlouvou, reagování na změny nad dodržováním plánu.

Scrum Nejrozšířenější agilní rámec.

- **Role:** Product Owner (PO), Scrum Master (SM), Development Team (3–9 lidí)
- **Artefakty:** Product Backlog (seznam požadavků), Sprint Backlog, Increment (fungující SW)
- **Events:** Sprint (1–4 týdny), Sprint Planning, Daily Scrum (15 min), Sprint Review, Sprint Retrospective
- Burndown chart sleduje zbývající práci ve Sprintu

Crystal Rodina metodik vytvořená Alistairem Cockburnem. Základní myšlenka: **neexistuje jedna metodika pro všechny** – malý tým na nekritické aplikaci nepotřebuje stejně těžký proces jako 50 lidí vyvíjejících letecký software. Crystal proto nabízí více variant lišících se „tíhou“ procesu podle dvou os:

- **Velikost týmu** – čím víc lidí, tím více formální komunikace a dokumentace je potřeba
- **Kritičnost systému** – co se stane, když selže? (ztráta pohodlí / peněz / zdraví / života)

Varianty podle barvy (barva = přibližný počet členů týmu):

- **Crystal Clear** – 1–6 lidí; nekritické systémy; minimální proces, maximální svoboda
- **Crystal Yellow** – 7–20 lidí
- **Crystal Orange** – 21–40 lidí; více dokumentace a koordinace
- **Crystal Red** – 41–80 lidí; nejtěžší varianta

Klíčové principy Crystal:

- **Osmotická komunikace** – tým sedí fyzicky pohromadě; informace se šíří přirozeně jako osmóza – „uslyšíš“ kolegu řešit problém a automaticky vstřebáš kontext, aniž by se konala formální schůzka. Opak je tým rozptýlený po světě, kde musíš vše aktivně komunikovat.

- **Přímá komunikace** – osobní rozhovor je preferován před e-mailem nebo ticketem
- **Reflexe** – tým se pravidelně zastavuje a přemýšlí, co zlepšit (podobné Scrum Retrospective)
- **Osobní bezpečí** – každý se může vyjádřit bez strachu z postihu; předpoklad pro otevřenou komunikaci
- **Časté dodávky** – pravidelné verze funkčního softwaru reálným uživatelům

Crystal vs. Scrum: Scrum předpisuje konkrétní role (PO, SM) a ceremonie (Daily, Review...); Crystal říká – zjistěte, co funguje pro váš tým, a dělejte to. Je méně strukturovaný, více lidsky orientovaný.

DSDM (Dynamic Systems Development Method) Iterativní a inkrementální metodika; fixuje čas a náklady, variabilní je rozsah (MoSCoW prioritizace).

Iterativní = software se vyvíjí v opakujících se cyklech (iteracích); v každé iteraci se vezme část požadavků, navrhne se, implementuje a otestuje. Na konci iterace se dostane zpětná vazba a příští iterace se podle ní upraví. Nejde rovnou od A do Z – jde se dokola a každým kolem se produkt zpřesňuje.

Inkrementální = každá iterace přidá nový fungující kus (inkrement) do produktu. Po první iteraci máš základní verzi, po druhé o něco více, po třetí ještě více – jako stavění z kostek. Uživatel může používat produkt dřív, než je „hotový“.

Rozdíl od vodopádu: vodopád je ani iterativní ani inkrementální – nejdřív celá analýza, pak celý návrh, pak celá implementace; uživatel vidí výsledek až úplně na konci. DSDM dodává funkční software průběžně.

- **MoSCoW:** Must have, Should have, Could have, Won't have
- Fáze: Pre-project, Feasibility, Foundation, Exploration, Engineering, Deployment

ASD (Adaptive Software Development)

- Cyklus: Speculate (plánování) → Collaborate (vývoj) → Learn (hodnocení)
- Zaměřen na vysoce turbulentní prostředí – projekty, kde se požadavky mění rychleji, než stihneš plánovat

Příklad – vývoj mobilní aplikace pro startupem měnící strategii:

- **Speculate:** Tým odhadne, co by mohlo fungovat pro příštích 3 týdny – ne pevný plán, ale hypotéza. Startup řekne „zkusme push notifikace a doporučování obsahu“. Záměrně se říká *speculate* (spekulovat), ne *plan* – přiznáváme, že nevíme přesně co bude potřeba.
- **Collaborate:** Tým spolupracuje na implementaci; sdílení znalostí, žádná síla – každý ví co dělá ostatní.
- **Learn:** Po dodávce se vyhodnotí co fungovalo a co ne. Startup zjistí, že notifikace uživatele otravují – toto poznání vstoupí do dalšího kola *Speculate*. Cyklus se opakuje.

Klíčový rozdíl od Scrumu: ASD nepředstírá, že lze přesně plánovat – staví na tom, že **změna je normální stav**, ne výjimka.

Lean Software Development Přenesení principů Lean manufacturingu do SW vývoje (Toyota Production System):

- 7 principů: eliminuj plýtvání, zesiluj učení, rozhoduj pozdě, dodávej rychle, zmocni tým, buduj integritu, viditelnost celku

FDD (Feature Driven Development)

- Vývoj řízený funkcemi (features); iterace 2 týdny
- 5 aktivit: Develop Overall Model, Build Feature List, Plan by Feature, Design by Feature, Build by Feature

Příklad – e-shop:

- **Develop Overall Model:** Tým vytvoří celkový přehled systému – třídy jako Zákazník, Objednávka, Produkt, jejich vztahy. Ne detailní návrh, jen doménový model.
- **Build Feature List:** Ze systému se vypíše seznam konkrétních funkcí ve formátu „akce výsledku objektem“: „zobrazit detail produktu“, „přidat produkt do košíku“, „odeslat potvrzovací e-mail“. Každá funkce = max 2 týdny práce.
- **Plan by Feature:** Funkce se seřadí podle priority a přiřadí vývojářům: „přidat do košíku“ má vyšší prioritu než „zobrazit historii objednávek“.
- **Design by Feature:** Pro každou funkci se navrhne detailní řešení – sekvenční diagram, třídy.
- **Build by Feature:** Implementace, unit testy, code review, integrace. Po dvou týdnech je funkce hotová a fungující.

Výsledkem je, že zákazník vidí průběžně přibývajících konkrétních funkcí, ne abstraktní „procenta hotovosti“.

XP (Extreme Programming) Agilní metodika s důrazem na technické praktiky:

- **Párové programování** – dva vývojáři u jednoho počítače; průběžná revize kódu
- **TDD** – testy před implementací
- **Kontinuální integrace** – časté buildy a testy
- **Refaktoring** – průběžné zlepšování struktury kódu
- **Jednoduché návrhy** – YAGNI (You Aren't Gonna Need It)
- **Kolektivní vlastnictví kódu** – každý může měnit jakýkoli kód
- Krátké iterace (1–2 týdny), zákazník je přímo v týmu

4.3.3 Srovnání přístupů

Vlastnost	Agilní	Tradiční (Waterfall)
Požadavky	Proměnné, iterativní	Fixní, předem definované
Dokumentace	Minimální, funkční SW	Rozsáhlá
Zákazník	Průběžně zapojen	Zapojen na začátku a konci
Dodávka	Inkrementální	Jednorázová
Tým	Malý, self-organizing	Hierarchický
Vhodné pro	Nejasné požadavky	Stabilní, dobře definované projekty

4.4 Architektonické vzory a CMMI

4.4.1 Architektonické vzory

Architektonické vzory popisují strukturu celého systému na vysoké úrovni:

- **Vrstvená architektura (Layers)** – systém rozdělen do horizontálních vrstev; každá vrstva poskytuje služby vrstvě nad ní (UI → Business Logic → Data Access → DB); výhoda: oddělení zodpovědností, snadná výměna vrstvy.
Příklad: Typická webová aplikace – React (UI) volá Spring Boot kontroler (Business Logic), který přes JPA repozitář (Data Access) čte z PostgreSQL (DB). Chceš vyměnit DB za MySQL? Stačí změnit Data Access vrstvu, UI se nedotkneš.
- **Pipes and Filters** – data procházejí sekvencí filtrů (transformací) propojených rourami; výstup jednoho filtru je vstupem dalšího.
Příklad: Unix pipeline `cat log.txt | grep ERROR | sort | uniq -c` – každý příkaz je filtr, roura `|` je pipe. Nebo ETL proces: Extrahuj data z CSV → Odstraň duplicity → Převeď formát dat → Načti do DB.
- **Shared Repository (Blackboard)** – komponenty sdílejí společné datové úložiště; každá čte a zapisuje do společné „tabule“; vhodné pro komplexní problémy řešené více spolupracujícími komponentami.
Příklad: IDE – editor zapíše zdrojový kód do úložiště, kompilátor ho přečte a zapíše AST, debugger přečte AST a nabídne breakpointy; všechny komponenty sdílejí stejná data, ale nevědí o sobě navzájem.
- **MVC (Model-View-Controller)** – oddělení dat (Model), prezentace (View) a logiky (Controller); Model notifikuje View o změnách (Observer pattern).
Příklad: E-shop v ASP.NET – `ProduktModel` obsahuje data o produktu, `ProduktController` zpracuje HTTP požadavek a rozhodne co se stane, `ProduktView` (Razor šablona) zobrazí HTML. Změníš šablonu bez dotyku databázové logiky.
- **Klient-Server** – server poskytuje zdroje/služby; klient o ně žádá přes síť; server obsluhuje více klientů najednou.
Příklad: Prohlížeč (klient) pošle HTTP GET na `google.com`; server zpracuje požadavek a vrátí HTML. Nebo Outlook (klient) se připojí na Exchange server přes IMAP a stáhne e-maily.

- **Peer-to-Peer (P2P)** – každý uzel je současně klient i server; není centrální autorita; odolné vůči výpadkům.

Příklad: BitTorrent – stahující počítač zároveň nabízí již stažené části ostatním; čím více lidí stahuje, tím rychleji to jde. Nebo Bitcoin – každý uzel drží kopii blockchainu a validuje transakce; neexistuje centrální banka.

- **Event-driven** – komponenty komunikují přes události (events); producent událost vydá, konzument na ni reaguje; producent neví kdo poslouchá – volná vazba.

Příklad: E-shop – po dokončení objednávky se vydá událost **ObjednavkaVytvorena**; na ni nezávisle reagují: služba pro e-mail (pošle potvrzení), služba pro sklad (odečte zboží), služba pro účetnictví (vystaví fakturu). Přidáš novou službu bez změny ostatních.

4.4.2 CMMI (Capability Maturity Model Integration)

CMMI je rámec pro hodnocení a zlepšování procesní zralosti organizací vyvíjejících software. Definuje 5 úrovní zralosti:

1. **Initial (Počáteční)** – procesy chaotické, ad-hoc; úspěch závisí na individu
2. **Managed (Řízený)** – projekty jsou plánované a sledované; základní projektové procesy
3. **Defined (Definovaný)** – procesy standardizované na úrovni organizace; popsány v procesním modelu
4. **Quantitatively Managed (Kvantitativně řízený)** – procesy měřeny a statisticky řízeny; předvídatelnost
5. **Optimizing (Optimalizující)** – kontinuální zlepšování procesů; inovace; prevence defektů

Příklad – softwarová agentura vyvíjející zakázkový software:

1. **Level 1 – Initial:** Malá agentura, 5 vývojářů. Každý projekt běží jinak podle toho, kdo ho vede. Jeden senior všechno drží v hlavě, termíny se odhadují od oka, testování se dělá „až bude čas“. Projekt dopadne dobře, když je na něm správný člověk – jinak chaos.
2. **Level 2 – Managed:** Agentura zavede základní projektové řízení. Každý projekt má Jira board, odhady v story pointech, pravidelný status report klientovi. Stále se procesy liší projekt od projektu, ale aspoň jsou řízené.
3. **Level 3 – Defined:** Agentura sepíše firemní „playbook“ – jak vypadá každý projekt: šablona pro analýzu, povinný code review, definovaný deployment proces, onboarding nováčků. Nový vývojář nastoupí a ví přesně co dělat – nezávisí na tom, kdo mu co řekne.
4. **Level 4 – Quantitatively Managed:** Agentura měří: průměrná hustota bugů na 1000 řádků kódu, průměrné překročení odhadů, velocity týmu. Na základě dat předvídá, zda projekt doručí včas, a umí říct klientovi „na základě historických dat dodáme za 6 týdnů $\pm$ 1 týden“.

5. **Level 5 – Optimizing:** Agentura automaticky analyzuje data z projektů, hledá vzorce (např. „bugy vznikají nejvíce v pátek odpoledne před releasem“) a aktivně mění procesy. Zavede například povinné freeze na nové funkce 2 dny před releasem. Procesy se zlepšují samy.

4.5 Software Quality Assurance (SQA)

4.5.1 Co je SQA a proč na ní záleží

Software Quality Assurance (SQA) je systematický soubor činností, jejichž cílem je zajistit, že softwarový produkt splňuje definované požadavky na kvalitu a je dodáván bez kritických chyb.

SQA vs. QC – důležité rozlišení

- **QA (Quality Assurance)** – *preventivní*; zaměřuje se na **procesy** vývoje; cílem je zabránit vzniku defektů (správné code review, správná metodika, testovací strategie)
- **QC (Quality Control)** – *detektivní*; zaměřuje se na **produkt**; hledá defekty které již vznikly (testování, inspekce výstupů)

Proč SQA záleží v praxi:

- Chyba nalezená při **code review** stojí hodiny práce
- Chyba nalezená při **testování** stojí dny práce (oprava + regrese – po opravě chyby musíš znovu otestovat vše ostatní, protože oprava jedné věci může nechtěně rozbít jinou; tomuto zpětnému prověření se říká *regresní testování*)
- Chyba nalezená **zákazníkem v produkci** stojí týdny + poškozená reputace
- Příklad: Boeing 737 MAX – chyba MCAS softwaru; NASA Mars Climate Orbiter – chyba jednotek (imperial vs. metric)

4.5.2 Dimenze kvality softwaru – ISO/IEC 25010

ISO/IEC 25010 (SQuaRE) definuje 8 charakteristik kvality produktu s konkrétními příklady z praxe:

- **Funkčnost** – dělá systém to, co má?
Příklad: platební brána správně odečte částku z účtu a zašle potvrzení
- **Výkonnostní efektivita** – jak rychle a s jakými zdroji?
Příklad: stránka e-shopu se načte do 2 s i při 10 000 souběžných uživatelích
- **Spolehlivost** – funguje systém správně i za nepříznivých podmínek?
Příklad: bankovní systém dosahuje 99,99 % dostupnosti (méně než 1 hod výpadku/rok)
- **Použitelnost** – mohou uživatelé systém efektivně používat?
Příklad: nový zaměstnanec zvládne základní operace v CRM bez školení do 30 minut

- **Bezpečnost** – jsou data a funkce chráněny před neoprávněným přístupem?
Příklad: hesla jsou ukládána jako bcrypt hash, ne plaintext; HTTPS everywhere
- **Udržitelnost** – lze systém snadno měnit a rozšiřovat?
Příklad: přidání nové platební metody nevyžaduje změny ve více než 3 souborech
- **Přenositelnost** – lze systém nasadit v různých prostředích?
Příklad: aplikace běží na Windows, Linux i macOS; Docker kontejner nasaditelný na AWS/Azure
- **Kompatibilita** – funguje systém správně vedle jiných systémů?
Příklad: REST API vrací data ve formátu kompatibilním s mobilní i webovou aplikací

4.5.3 Procesy SQA – s příklady z praxe

1. Revize kódu (Code Review) Inspektor (kolega nebo nástroj) prochází kód před mergem do hlavní větve.

Jak probíhá v praxi (GitHub/GitLab Pull Request flow):

1. Vývojář vytvoří feature branch a napíše kód
2. Otevře Pull Request (PR) – popis změny, jaké testy přidal
3. 1–2 kolegové zkontrolují: logika, bezpečnostní díry, dodržení konvencí, duplicity
4. Reviewer buď approves nebo přidá komentáře s požadovanými změnami
5. Po schválení se branch mergeje do main

Co code review zachytí: překlepy v podmínkách (= místo ==), SQL injection, chybějící null check, porušení SOLID, zbytečná složitost, chybějící testy.

2. Statická analýza kódu Automatizované nástroje analyzují zdrojový kód **bez jeho spuštění**:

- **SonarQube** – identifikuje bugs, code smells, bezpečnostní zranitelnosti, technický dluh; integruje se do CI/CD pipeline; přiřazuje rating A–E
- **Checkstyle / PMD / SpotBugs** – Java; kontrola coding conventions, potenciální chyby
- **ESLint / TSLint** – JavaScript/TypeScript; syntax, konvence, potenciální chyby

Příklad: SonarQube v CI pipeline odmítne merge pokud code coverage klesne pod 80 % nebo přibýde kritická zranitelnost.

3. Testování – testovací pyramida

Typ testu	Co testuje	Rychlost	Příklad
Unit testy	Izolovaná třída/metoda	Sekundy	<code>kalkulacka.secti(2,3) == 5</code>
Integrační	Spolupráce komponent	Minuty	Service + databáze vrátí správný záznam
Systémové	Celý systém end-to-end	Minuty	Uživatel se přihlásí a vidí svůj profil
UAT	Akceptace zákazníkem	Hodiny	Zákazník potvrdí, že objednávka funguje
Zátěžové	Výkon pod zátěží	Hodiny	1000 users/s, response time < 500 ms

Testovací pyramida: základna = mnoho unit testů (levné, rychlé); střed = integrační testy; vrchol = málo E2E testů (drahé, pomalé). Čím výše v pyramidě, tím dražší oprava chyby.

4. CI/CD – Continuous Integration / Continuous Deployment Každý commit spustí automatický řetěz kontrol:

Krok	Nástroj	Co dělá
Checkout kódu	GitHub Actions / GitLab CI	Stáhne větev
Build	Maven, Gradle, npm	Zkompiluje projekt
Unit testy	JUnit, pytest	Spustí testy, musí 100 % projít
Statická analýza	SonarQube	Kontrola kvality kódu
Integrační testy	Testcontainers, DB v Dockeru	Testy s reálnou DB
Security scan	OWASP Dependency-Check	Zranitelné závislosti?
Deploy (staging)	Kubernetes, Helm	Nasazení na testovací prostředí
E2E testy	Selenium, Cypress	Testy UI v prohlížeči
Deploy (produkce)	–	Pouze pokud vše prošlo

Příklad: tým v Spotify provede přes 1000 deployů denně – to je možné jen s plně automatizovaným CI/CD.

5. Metriky kvality kódu

- **Code coverage** – % kódu pokrytého testy; cíl typicky $\geq 80\%$; *pozor: 100 % coverage neznamená 0 bugů – záleží na kvalitě assertů*
- **Technický dluh (Technical Debt)** – množství práce potřebné k uvedení kódu do ideálního stavu; měří ho SonarQube v hodinách/dnech
- **Cyklomatická složitost** – počet nezávislých cest kódem; hodnota >10 signalizuje příliš složitou metodu, která by se měla rozdělit.
Jak se měří: spočítáš větvení v kódu – každý `if`, `else`, `for`, `while`, `case`, `&&`, `||` přidá 1. Výsledek = počet větví + 1. Metoda bez jediného větvení má složitost 1 (jedna cesta). Metoda s pěti `if` bloky má složitost 6.
Příklad: metoda, která ověří přihlášení – zkontroluje zda uživatel existuje, zda není blokový, zda heslo souhlasí, zda nevypršela session – to jsou 4 větve, složitost = 5. Snadno testovatelné. Metoda se složitostí 20 má 20 různých průchodů – musíš napsat 20 unit testů jen pro ni, což signalizuje, že ji máš rozdělit.
- **MTTR (Mean Time to Repair)** – průměrná doba od detekce incidentu po jeho vyřešení; klíčová metrika DevOps týmů

- **Defect density** – počet bugů na 1000 řádků kódu (KLOC); benchmark: <1 bug/KLOC.
KLOC = Kilo Lines of Code = 1000 řádků kódu; slouží jako normalizační jednotka
 – projekt s 50 000 řádky a 30 bugy má defect density $30/50 = 0,6$ bug/KLOC, což je pod benchmarkem a považuje se za dobré. Srovnáváš tak různě velké projekty nebo moduly spravedlivě.

Otázky profesorů k tématu: Softwarové inženýrství

Vojtěch Stanislav, Ing., Ph.D.

- Popište návrhový vzor Observer a jeho použití.
- Jak funguje Scrum? Jaké jsou jeho artefakty a role?
- Jaký je rozdíl mezi agilním a vodopádovým modelem?

Chudán David, Ing., Ph.D.

- Jaké UML diagramy znáte? Který byste použil pro popis interakce mezi objekty?
- Co je návrhový vzor a proč jsou důležité?

Komise: Svátek, Chlapek, Zbořil

- Jaké jsou základní principy agilního vývoje?
- Popište diagram případů užití a jeho elementy.

Sládek Pavel, Ing., Ph.D.

- Co je TDD a jak se liší od tradičního testování?
- Jaký je rozdíl mezi Singleton a Factory Method?

Pour Jan, doc. Ing., CSc.

- Jaké jsou fáze metodiky DSDM?
- Co je SQA a jaké procesy zahrnuje?

5 Databáze

Obecné pojmy

Databáze je organizovaný systém souborů pro ukládání dat. Tyto soubory jsou mezi sebou navzájem propojeny. V širším smyslu jsou součástí databáze i softwarové prostředky, které umožňují manipulaci s uloženými daty a přístup k nim. Tento software se v české odborné literatuře nazývá **systém řízení báze dat (SŘBD)**. Běžně se označením databáze myslí jak uložená data, tak i software.

- **data** – formalizované a fyzicky zaznamenané znalosti, poznatky, zkušenosti, výsledky pozorování
- **informace** – smysluplná interpretace dat
- **databáze** – integrovaná počítačově zpracovaná množina persistentních dat
- **relační model dat** – data jsou uložena v tabulkách (relacích); každý řádek je jedna entita, každý sloupec je atribut. Matematický základ tvoří predikátová logika prvního řádu. Existují dva ekvivalentní způsoby jak se na manipulaci s daty dívat:
 - **Relační algebra** – *procedurální* přístup; popisuje *jak* data získat pomocí operátorů: σ (selekce řádků), π (projekce sloupců), $\bowtie$ (spojení tabulek), $\cup$ (sjednocení), $-$ (rozdíl). Příklad: “z tabulky zaměstnanců vyber sloupce jméno a plat, kde plat > 50 000” = $\pi_{\text{jméno, plat}}(\sigma_{\text{plat} > 50000}(\text{zamestnanci}))$
 - **Relační kalkul** – *deklarativní* přístup; popisuje *co* chceme (ne jak to získat). SQL je v podstatě syntaktický cukr nad relačním kalkulem – napíšete `SELECT jmeno, plat FROM zamestnanci WHERE plat > 50000` a databázový stroj sám rozhodne, jak dotaz vykoná

Oba přístupy jsou ekvivalentní (Coddův teorém) – vše co jde vyjádřit algebrou, jde vyjádřit i kalkulem a naopak

- **dotazovací jazyk** – slouží pro získání dat z databáze; v relační databázi je odpovědí relace, jejíž n-tice splňují nadefinované podmínky
- **systém řízeníází dat (SŘBD / DBMS)** – množina programových prostředků, která umožňuje: vytvoření databáze, použití databáze (manipulace s daty), údržbu a správu databáze
- **databázový systém (DBS)** – SŘBD + databáze

Dělení databází podle typu

1. **Relační databáze** – nejčastěji používaný typ; data modelována pomocí tabulek s fixní strukturou propojených pomocí relací. Příklady: Oracle, MySQL, PostgreSQL, MS SQL Server.
2. **NoSQL databáze** – bez tabulek, nestrukturované; rychlejší a škálovatelnější. Ukládání dat do JSON/BSON. Příklady: MongoDB, CouchDB, Amazon DynamoDB.
3. **Specializované databáze:**

- *Graph Databases* – neo4j; data jsou uložena jako **uzly** (entity) a **hrany** (vztahy mezi nimi); ideální pro situace, kde jsou vztahy stejně důležité jako data samotná.
Příklad: sociální síť – každý uživatel je uzel, přátelství je hrana. Dotaz „najdi všechny přátele přátel Jana, kteří mají zájem o filmy“ je v grafové DB přirozený a rychlý; v relační DB by vyžadoval složité joiny přes vazební tabulky. Další využití: detekce podvodů, doporučovací systémy (Spotify, Netflix).
- *Key-Value* – Redis, Memcached; funguje jako obrovský **slovník (hashmap)** – ke každému klíči patří jedna hodnota; vše je typicky v paměti (RAM) → extrémně rychlé.
Příklad: ukládání session přihlášeného uživatele: klíč = `session:abc123`, hodnota = `{user_id: 42, role: "admin"}`. Při každém HTTP požadavku se session načte z Redis za zlomek milisekundy místo dotazu do DB. Využití: cache, session management, rate limiting, fronty.
- *Time-series* – optimalizované pro **časové řady** – data, která přicházejí průběžně s časovou značkou a typicky se nikdy nemění, jen přibývají. Příklady: InfluxDB, Prometheus, TimescaleDB.
Příklad: monitoring serveru – každou sekundu se zapíše (`timestamp`, `cpu=72%`, `ram=4GB`, `disk_io=120MB/s`). Za den to jsou desetitisíce záznamů. Time-series DB je optimalizována pro dotazy jako „zobraz průměrné CPU za posledních 6 hodin po minutách“ – v relační DB by to bylo pomalé. Další využití: IoT senzory, burzovní kurzy, telemetrie.
- *Sloupcové (Column-family)* – Cassandra, HBase; místo ukládání dat **po řádcích** ukládají data **po sloupcích**; při analytickém dotazu na jeden sloupec přes miliony řádků nemusí číst ostatní sloupce → rychlejší analytika.
Příklad: e-shop s miliony objednávek; dotaz „jaký byl celkový obrát za každý měsíc“ potřebuje jen sloupec `castka` a `datum` – sloupcová DB přečte jen tyto dva sloupce, relační DB by četla celé řádky včetně jména zákazníka, adresy atd. Využití: datové sklady, big data analytika.

5.1 Analýza, návrh a realizace databáze

5.1.1 Databázový systém a jeho vlastnosti

Databázový systém (DBS) = SŘBD + Databáze

Důvody pro vznik databázových systémů:

- **Vyšší datová abstrakce** – SŘDB zahrnují manipulační jazyky pro práci s datovými strukturami vyšší úrovně
- **Nezávislost aplikačních programů** na změnách ve fyzickém uložení dat
- **Ochrana dat** před neoprávněným přístupem a poruchami
- **Odstranění redundance** – každý údaj uložen pouze jedenkrát
- **Sdílení dat** – data mohou být používána několika uživateli

- Podpora transakčního zpracování
- Údržba integrity databáze

5.1.2 Metodologie analýzy a návrhu databáze – P3A

Návrh databáze vychází z principu tří architektur (**P3A**). Smyslem P3A je postupně upřesňovat datový model tak, aby zcela odpovídal požadavkům využívané databázové technologie.

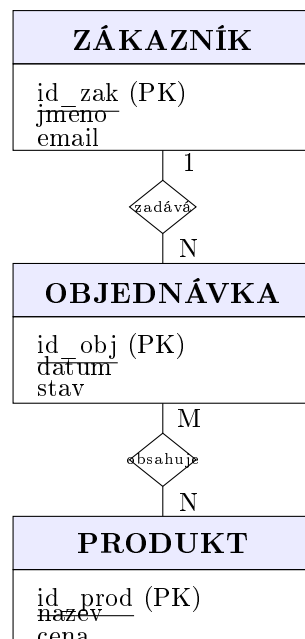
Tři úrovně lidsky: Představ si, že stavíš dům.

- **Konceptuální** = rozhovor s architektem: „chci obývací pokoj, ložnici, kuchyň a koupelnu, ložnice musí být blízko koupelny“. Žádné rozměry, žádné materiály – jen co potřebuješ a jak to spolu souvisí. V databázi: entity (Zákazník, Objednávka) a jejich vztahy – bez datových typů, bez konkrétní databáze.
- **Logický** = technický výkres: místnosti mají rozměry, jsou nakreslené dveře a okna, ale stále bez konkrétního materiálu. V databázi: tabulky, sloupce, primární a cizí klíče, datové typy (INT, VARCHAR) – ale stále nezávislé na tom, zda použiješ Oracle nebo PostgreSQL.
- **Fyzický** = stavební plán: přesný materiál, způsob pokládky, výrobce oken. V databázi: konkrétní SQL skripty pro daný DBMS, indexy, partitioning, optimalizace výkonu.

Příklad prolínající se všemi třemi úrovněmi: systém e-shopu (zákazníci, objednávky, produkty).

1. Konceptuální model (model reality) Model obsahu datové základny na konceptuální úrovni – nezávislý na technologii, bez datových typů.

- **KSR** (Konceptuální schéma reality) – hrubý model; základní entity, vztahy a atributy
- **KSD** (Konceptuální schéma dat) – přesná specifikace; ER diagram (Entity-Relationship)



Obrázek 10: Konceptuální model – ER diagram e-shopu

2. Logický (technologický) model Konceptuální schéma se transformuje do relačních tabulek. Vztah M:N (OBJEDNÁVKA–PRODUKT) se rozloží na vazební tabulku POLOŽKA se dvěma cizími klíči.

- Každá entitní množina → relační tabulka; identifikátory → primární klíč
- Vztah 1:N → cizí klíč (FK) na straně N
- Vztah M:N → vazební tabulka se dvěma cizími klíči
- Generalizace/specializace → varianty: jedna tabulka pro celou hierarchii, nebo tabulky na úrovni specializací

Výsledné schéma (PK podtrženo, FK šipkou):

Tabulka	Atributy
ZÁKAZNÍK	<u>id_zak</u> , jmeno, email
OBJEDNÁVKA	<u>id_obj</u> , id_zak → ZÁKAZNÍK, datum, stav
PRODUKT	<u>id_prod</u> , nazev, cena
POLOŽKA	<u>id_obj</u> → OBJEDNÁVKA, <u>id_prod</u> → PRODUKT, mnozstvi

Cíle logického modelu: věrný obraz reality, interní konzistence, minimalizace redundance, maximalizace stability. Sada normálních forem: 1NF, 2NF, 3NF, BCNF, 4NF, 5NF.

3. Fyzický (implementační) model Logický model se přepíše do konkrétního DDL jazyka cílové databáze – přidají se datové typy, omezení, indexy a optimalizace pro výkon.

- **Způsob uložení dat:** sekvenční (pro zálohy), přímé (hashing – adresa záznamu = $f(\text{PK})$)
- **Způsob přístupu k datům:** sekvenční, přímé, indexy

Listing 14: Fyzický model – CREATE TABLE (PostgreSQL)

```
CREATE TABLE zakaznik (
    id_zak    INT          PRIMARY KEY,
    jmeno     VARCHAR(100) NOT NULL,
    email     VARCHAR(200) UNIQUE NOT NULL
);

CREATE TABLE objednavka (
    id_obj    INT          PRIMARY KEY,
    id_zak    INT          NOT NULL REFERENCES zakaznik(id_zak),
    datum     DATE         NOT NULL DEFAULT CURRENT_DATE,
    stav      VARCHAR(20)  NOT NULL DEFAULT 'nova'
);
-- Index pro rychlé hledání všech objednavek zakaznika
CREATE INDEX idx_obj_zak ON objednavka(id_zak);

CREATE TABLE produkt (
    id_prod   INT          PRIMARY KEY,
    nazev     VARCHAR(200) NOT NULL,
    cena      DECIMAL(10,2) NOT NULL CHECK (cena > 0)
);
-- Vazebni tabulka pro vztah M:N (objednavka <-> produkt)
CREATE TABLE polozka (
    id_obj    INT REFERENCES objednavka(id_obj) ON DELETE CASCADE,
    id_prod   INT REFERENCES produkt(id_prod),
    mnozstvi  INT NOT NULL DEFAULT 1 CHECK (mnozstvi > 0),
    PRIMARY KEY (id_obj, id_prod)    -- slozeny PK
);
```

5.1.3 Normalizace databáze

Normalizace je proces postupného odstraňování redundancí a anomálií z databázového schématu. Každá normální forma buduje na předchozí – pokud je tabulka v 3NF, je automaticky i v 2NF a 1NF.

Proč normalizovat? Bez normalizace vznikají *anomálie*:

- **Vkládací anomálie** – nelze vložit data bez jiných dat (nelze přidat oddělení bez zaměstnance)
- **Aktualizační anomálie** – změna jednoho faktu vyžaduje UPDATE na více řádcích najednou
- **Mazací anomálie** – smazání řádku odstraní nezávislou informaci

Přehled normálních forem:

- **UNF** (Unnormalized Form) – nenormalizovaná databáze; žádná pravidla
- **0NF** – alespoň jeden atribut obsahuje více než jednu hodnotu

- **1NF** – každý atribut obsahuje pouze atomické (nedělitelné) hodnoty
- **2NF** – každý neklíčový atribut je plně závislý na každém kandidátním klíči (neklíčový atribut = atribut, který není součástí žádného kandidátního klíče)
- **3NF** – všechny neklíčové atributy musí být vzájemně nezávislé
- **BCNF** – atributy, které jsou součástí primárního klíče, musí být vzájemně nezávislé
- **4NF** – relace popisuje pouze příčinnou souvislost mezi klíčem a atributy
- **5NF** – relaci již není možno bezztrátově rozložit

1. normální forma (1NF) Pravidlo: Každý atribut obsahuje pouze atomické (nedělitelné) hodnoty – žádné seznamy ani opakující se skupiny v jedné buňce.

Porušení – seznam hodnot v buňce:

id_zam	jmeno	telefony
1	Jan Novák	777 111 222, 777 333 444
2	Eva Černá	603 555 666

Oprava – přesun vícehodnotového atributu do vlastní tabulky:

ZAMESTNANEC		ZAM _ TELEFON	
<u>id_zam</u>	jmeno	<u>id_zam</u>	<u>telefon</u>
1	Jan Novák	1	777 111 222
		1	777 333 444
2	Eva Černá	2	603 555 666

2. normální forma (2NF) Pravidlo: Je v 1NF a každý neklíčový atribut je plně závislý na každém kandidátním klíči – nesmí záviset jen na části složeného klíče. (Relevantní pouze pro tabulky se složeným primárním klíčem.)

Porušení – částečná závislost (navez_prod závisí jen na id_prod, ne na celém PK):

POLOZKA PK = (<u>id_obj</u> , <u>id_prod</u>)				
<u>id_obj</u>	<u>id_prod</u>	mnozství	navez_prod	cena
1	101	2	Kniha Java	599
1	102	1	Myš	299
2	101	1	Kniha Java	599

Problém: “Kniha Java” se opakuje – změna názvu vyžaduje UPDATE na více řádcích.

Oprava – oddělit atributy závislé jen na id_prod:

POLOZKA			PRODUKT		
<u>id_obj</u>	<u>id_prod</u>	mnozství	<u>id_prod</u>	navez_prod	cena
1	101	2	101	Kniha Java	599
1	102	1	102	Myš	299

3. normální forma (3NF) Pravidlo: Je v 2NF a všechny neklíčové atributy musí být vzájemně nezávislé – nesmí existovat tranzitivní závislost neklíčového atributu přes jiný neklíčový atribut.

Co je tranzitivní závislost?

Tranzitivní závislost nastane, když neklíčový atribut závisí na **jiném neklíčovém atributu** místo přímo na PK. Jinými slovy: existuje řetěz $A \rightarrow B \rightarrow C$, kde B není klíč.

Příklad: $id_zam \rightarrow id_oddel \rightarrow nazev_oddel$

$nazev_oddel$ neurčuje přímo PK (id_zam), ale prostřednictvím id_oddel .

Porušení – tranzitivní závislost přes id_oddel :

ZAMESTNANEC			PK = id_zam	
id_zam	jmeno	id_oddel	$nazev_oddel$	budova
1	Jan Novák	10	IT	A
2	Eva Černá	10	IT	A
3	Petr Bílý	20	Finance	B

Závislosti: $id_zam \rightarrow id_oddel$ (přímá, OK) $id_oddel \rightarrow nazev_oddel$, budova (id_oddel **není klíč!**)

Problém: IT přestěhuje z budovy A do C $\Rightarrow$ musím aktualizovat **oba** řádky (Jan i Eva). Pokud zapomenou jeden $\Rightarrow$ nekonzistence.

Oprava – vytáhnout atributy závislé na id_oddel do vlastní tabulky:

ZAMESTNANEC			ODDELENI		
id_zam	jmeno	id_oddel (FK)	id_oddel	$nazev_oddel$	budova
1	Jan Novák	10	10	IT	A
2	Eva Černá	10	20	Finance	B

Přestěhování IT: jediný UPDATE v ODDELENI, žádná redundance.

Boyce-Coddova normální forma (BCNF) Pravidlo: Je v 3NF a atributy, které jsou součástí primárního klíče, musí být vzájemně nezávislé. Formálně: pro každou netriviální funkční závislost $X \rightarrow Y$ musí být X superklíčem. BCNF je přísnější než 3NF.

Porušení – klasický příklad: rozvrh výuky.

ZKOUSENI		
$student$	$predmet$	$ucitel$
Novák	Databáze	Dr. Svátek
Černá	Databáze	Dr. Svátek
Novák	Java	Ing. Vojíš

Funkční závislosti: $(student, predmet) \rightarrow ucitel$ (každý student má jednoho učitele per předmět) a zároveň $ucitel \rightarrow predmet$ (každý učitel učí jen jeden předmět). Kandidátní klíče: $\{student, predmet\}$ nebo $\{student, ucitel\}$. Porušení: $ucitel \rightarrow predmet$, ale $ucitel$ samotný **není superklíč**.

Oprava:

UCITEL _ PREDMET		STUDENT _ UCITEL	
<u>ucitel</u>	predmet	<u>student</u>	<u>ucitel</u>
Dr. Svátek	Databáze	Novák	Dr. Svátek
Ing. Vojíš	Java	Černá	Dr. Svátek
		Novák	Ing. Vojíš

4. normální forma (4NF) Pravidlo: Je v BCNF a relace popisuje pouze příčinnou souvislost mezi klíčem a atributy – neobsahuje netriviální vícehodnotové závislosti (MVD), kdy jeden klíč určuje dvě nezávislé sady hodnot.

Vícehodnotová závislost $A \twoheadrightarrow B$ nastane, když jeden atribut A určuje **nezávislou sadu** hodnot B bez ohledu na ostatní atributy.

Porušení – dovednosti a certifikáty zaměstnance jsou na sobě nezávislé:

ZAM _ DOV _ CERT		
id_zam	dovednost	certifikat
1	Java	ITSM
1	Java	PMP
1	SQL	ITSM
1	SQL	PMP

Jan (id_zam=1) zná Java a SQL, a má certifikáty ITSM a PMP. Tyto fakty jsou **nezávislé** – přidání nové dovednosti (Python) vyžaduje vložit 2 nové řádky (Python+ITSM, Python+PMP).

Oprava – rozdělit na dvě tabulky:

ZAM _ DOVEDNOST		ZAM _ CERTIFIKAT	
<u>id_zam</u>	<u>dovednost</u>	<u>id_zam</u>	<u>certifikat</u>
1	Java	1	ITSM
1	SQL	1	PMP

5. normální forma (5NF – PJNF) Pravidlo: Je v 4NF a relaci již není možno bezztrátově rozložit – neobsahuje závislosti spojení (*join dependency*), které nelze odvodit z kandidátních klíčů.

5NF se uplatňuje u třiciferných (a vícenásobných) vztahů, kde ternární vazbu **nelze** nahradit binárními vztahy bez ztráty informace.

Příklad: dodavatel – produkt – zákazník

dodavatel	produkt	zakaznik
ACME	Tužka	Škola
ACME	Pero	Škola
Beta	Tužka	Firma

Pokud platí obchodní pravidlo: “*jestliže dodavatel D dodává produkt P , zákazník Z odbírá produkt P a D prodává zákazníkovi Z , pak tuple (D,P,Z) musí existovat*”, pak lze

tabulku bezztrátově rozložit do tří binárních projekcí (DODAVATEL_PRODUKT, PRODUKT_ZAKAZNIK, DODAVATEL_ZAKAZNIK) – to je 5NF.

Pokud toto obchodní pravidlo neplatí, ternární tabulka je v 5NF a **nesmí se rozkládat** (rozklad by způsobil spurious tuples při přirozeném joinu).

Poznámka: 4NF a 5NF se v praxi řeší vzácně – většina reálných projektů cíluje na 3NF nebo BCNF.

Denormalizace Záměrné porušování normalizace za účelem zvýšení výkonnosti systému:

- **Výhody:** snižování potřeby joinů, uchování aktuálních hodnot odvozených atributů
- **Nevýhody:** pomalejší UPDATE/INSERT/DELETE, složitější zajištění integrity

5.1.4 Integrita a transakce

Integritní omezení

- **Entitní integrita** – PK musí být UNIQUE a NOT NULL
- **Referenční integrita** – FK odkazuje pouze na existující hodnoty PK
- **Doménová integrita** – atribut obsahuje pouze přípustné hodnoty (CHECK, TRIGGER)

Transakce a vlastnosti ACID

- **A (Atomicity)** – transakce proběhne celá nebo žádná část
- **C (Consistency)** – DB přechází z jednoho konzistentního stavu do jiného
- **I (Isolation)** – souběžné transakce jsou navzájem nezávislé
- **D (Durability)** – efekty potvrzené transakce jsou trvale uloženy

BASE (NoSQL alternativa k ACID) NoSQL systémy upřednostňují dostupnost nad konzistencí (CAP teorém):

- **BA (Basically Available)** – systém vždy odpovídá, i kdyby data nebyla aktuální
- **S (Soft-state)** – stav systému se může měnit i bez vstupu (postupná synchronizace)
- **E (Eventual consistency)** – po dostatečné době bez nových zápisů se systém stane konzistentním

Izolace transakcí a problémy při souběžném přístupu Úrovně izolace (SQL standard) a problémy, které řeší:

- **Dirty read** – čtení nepotvrzených dat jiné transakce

- **Non-repeatable read** – opakované čtení vrátí různé hodnoty (jiná transakce mezitím změnila)
- **Phantom read** – dotaz vrátí jiné řádky při opakování (jiná transakce vložila/smazala)
- **Izolační úrovně** určují, jak moc je jedna transakce „odstíněna“ od změn, které dělají jiné souběžné transakce. Čím vyšší úroveň, tím více izolace a tím méně problémů – ale také nižší výkon (více zamykání). Škála od nejvolnější po nejprísnejší:
 - **READ UNCOMMITTED** – nejvolnější; transakce vidí i *nepotvrzená* data jiné transakce.
Příklad: banka převádí 1000 Kč z účtu A na B; uprostřed převodu (peníze odečteny z A, ale ještě nepřipsány na B) přijde jiná transakce a vidí dočasně „ztracených“ 1000 Kč. Toto je *dirty read* – čtení dat, která možná nikdy nebudou potvrzena.
 - **READ COMMITTED** – transakce vidí pouze *potvrzená* data; dirty read nehrozí. Nejčastější výchozí úroveň (PostgreSQL, Oracle, MS SQL Server).
Příklad: při dvou čteních ve stejné transakci může mezitím jiná transakce data změnit a potvrdit – druhé čtení vrátí jinou hodnotu (*non-repeatable read*).
 - **REPEATABLE READ** – zaručuje, že pokud jednou přečteš řádek, přečteš ho stejně i podruhé ve stejné transakci; jiná transakce ho mezitím nemůže změnit.
Příklad: stále však může jiná transakce *vložit nový řádek* – při opakování dotazu se mohou objevit nové záznamy (*phantom read*).
 - **SERIALIZABLE** – nejprísnejší; transakce se chovají, jako by běžely prísne jedna po druhé (sériově), i když fyzicky běží paralelně; žádný dirty read, non-repeatable read ani phantom read. Platí za to výkonem – více zamykání, možné uváznutí.
Použití: bankovní převody, rezervační systémy – kdekoli je správnost dat důležitější než rychlost.

Úroveň	Dirty read	Non-rep. read	Phantom read	Výkon
READ UNCOMMITTED	možný	možný	možný	nejvyšší
READ COMMITTED	ne	možný	možný	vysoký
REPEATABLE READ	ne	ne	možný	střední
SERIALIZABLE	ne	ne	ne	nejnižší

Uváznutí (Deadlock) Situace, kdy dvě nebo více transakcí čekají na uvolnění zámku navzájem:

- **Prevence** – stanovení pořadí zamykání zdrojů
- **Detekce a obnova** – DBMS detekuje cyklus v grafu čekání a jednu transakci přeruší (victim)
- **Timeout** – transakce čekající příliš dlouho se automaticky ukončí

Obnova po selhání (Recovery)

- **Žurnál (Log)** – záznamy o všech změnách (before-image, after-image)
- **UNDO** – odvolání nepotvrzených transakcí (po pádu systému)
- **REDO** – opakování potvrzených transakcí (obnovení po pádu)
- **Checkpoint** – periodický zápis stavu paměti na disk; zkrátí dobu obnovy
- **OLAP** (Online Analytical Processing) – vícedimenzionální dotazy, datové sklady, historická data
- **OLTP** (Online Transaction Processing) – velké množství malých transakcí (INSERT, UPDATE, DELETE)

5.1.5 Indexy

Indexy slouží ke zrychlení vyhledávání. Každý index zabírá místo v paměti a zpomaluje operace INSERT/UPDATE nad indexovanými sloupci.

Otázky profesorů k tématu: Analýza, návrh a realizace databáze

Komise: Svátek, Chlapek, Zbořil

- Jaký by měl být správný postup analýzy, návrhu a realizace databáze?

Máša Petr, RNDr. Ing., Ph.D.

- Jaké budou problémy aplikace vysoce náročné na data, jak řešit databázi?
- Co se rozumí principem 3NF?

Sládek Pavel, Ing., Ph.D.

- Jaké jsou druhy databází a jak fungují?

Chudán David, Ing., Ph.D.

- Jak se typicky strukturovaná data ukládají? Jaké znáte typy databází?
- Jaké jsou rozdíly mezi formáty CSV, JSON a XML?

Zeman Václav, Ing., Ph.D.

- Jak databáze zajistí konzistenci a integritu řešení?

Strossa Petr, doc. RNDr., CSc.

- V čem se liší NoSQL od relačních databází?
- K čemu slouží indexy v databázi?

Sudžina František, Mgr. et Mgr. Ing., Ph.D.

- Jaké jsou výhody databází v porovnání s tabulkami v Excelu?

Maryška Miloš, doc. Ing., Ph.D.

- Co je normalizace a denormalizace dat?

5.2 Datové modelování

Datové modelování je proces vytváření abstraktního modelu toho, *jaká data systém potřebuje ukládat, jak jsou mezi sebou provázána a jaká omezení na ně platí*. Provádí se **před** samotnou implementací databáze, podobně jako architekt kreslí plán budovy před zahájením stavby.

Proč datové modelování potřebujeme? Bez datového modelu si vývojáři navrhnou tabulky intuitivně – výsledkem bývají redundance, chybějící vazby, problémy s integrací a nutnost drahých refaktoringů. Datový model:

- přesně popisuje co systém o světě *ví* (které entity evidujeme, jaké mají vlastnosti)
- zachytí **pravidla byznysu** jako omezení v databázi (zaměstnanec musí patřit do oddělení)
- slouží jako **společný jazyk** mezi analytiky, vývojáři a zákazníkem
- je vstupem pro normalizaci a fyzický návrh

Co datový model zachycuje?

- **Entity** – věci, o kterých chceme ukládat data (zákazník, produkt, objednávka)
- **Atributy** – vlastnosti entit (jméno zákazníka, cena produktu)
- **Vztahy** – jak jsou entity navzájem propojeny (zákazník *zadává* objednávky)
- **Omezení** – pravidla platná pro data (cena musí být kladná, email musí být unikátní)

Vrstvy datového modelování Datový model existuje ve třech vrstvách abstrakce – od nejobecnější po nejkonkrétnější:

Vrstva	Co zachycuje	Nezávislost	Kdo pracuje
Konceptuální (co potřebujeme)	Entity, vztahy, atributy, omezení; žádné datové typy, žádné tabulky	Nezávislá na DB i technologii	Analytik, zákazník
Logická (jak to uložíme)	Tabulky, sloupce, PK, FK, normalizace; datové typy obecné (INT, VARCHAR)	Nezávislá na konkrétním DBMS	Databázový návrhář
Fyzická (jak to nasadíme)	DDL skripty, indexy, partitioning, storage; specifické pro Oracle / PostgreSQL / MySQL	Vázaná na konkrétní DBMS	DBA, vývojář

Příklad: zákazník zadává objednávky:

- **Konceptuální:** entita ZÁKAZNÍK má atributy {jméno, email}; vztah “zadává” s kardinalitou 1:N na OBJEDNÁVKA
- **Logická:** tabulka zakaznik(id_zak, jmeno VARCHAR, email VARCHAR); tabulka objednavka(id_obj, id_zak FK, datum DATE)

- **Fyzická:** CREATE TABLE zakaznik (id_zak SERIAL PRIMARY KEY, email VARCHAR(200) UNIQUE NOT NULL); + CREATE INDEX idx_obj_zak ON objednavka(id_zak);

Postup datového modelování (ve vztahu k P3A)

1. **Analýza požadavků** – porozumění business doméně; rozhovory s uživateli, analýza dokumentů
2. **Konceptuální model** – ERD nezávislý na technologii; komunikace se zákazníkem
3. **Logický model** – převod ERD na relační schéma; normalizace; nezávislý na konkrétní DB
4. **Fyzický model** – DDL skripty pro konkrétní DBMS; indexy; partitioning; výkon

Kdo datové modelování provádí? V praxi ho dělá **datový architekt** nebo **data-bázový designer**, v menších týmech analytik nebo senior vývojář. Na konci procesu vznikají:

- **ERD diagram** (vizuální konceptuální model; nástroje: ERwin, Lucidchart, draw.io, DbSchema)
- **Datový slovník** (popis každého atributu: datový typ, délka, povinnost, formát)
- **SQL DDL skripty** (fyzická realizace v databázi)

5.2.1 Základní pojmy

- **Entita** – typ objektů důležitých v dané realitě, o nichž se vedou záznamy; tabulka (v DB)
- **Atribut** – vlastnost nebo charakteristika entity; typy: povinný/volitelný, jednoduchý/složený, atomický/vícehodnotový
- **Vztah (relace)** – pojmenovaná spojitost mezi entitami
- **Kardinalita** – četnost vazby: 1:1, 1:N, M:N
- **Parcialita** – volitelnost vztahu (musí/nemusí entita vstupovat do vztahu)
- **Primární klíč (PK)** – atribut(y) jednoznačně identifikující každý záznam (NOT NULL, UNIQUE)
- **Cizí klíč (FK)** – atribut(y) odkazující na PK jiné tabulky; zajišťuje referenční integritu

5.2.2 ER (Entity Relationship) modelování

ER modelování se používá pro abstraktní a konceptuální znázornění dat. Výsledkem je **ERD** (Entity Relationship Diagram).

Kroky konceptuálního návrhu:

1. Identifikace entit
2. Identifikace relací a jejich kardinality
3. Identifikace atributů a jejich spojení s entitami
4. Určení domén atributů
5. Určení kandidátních, primárních a alternativních klíčů
6. Specializace/generalizace entit (volitelný krok)
7. Kontrola redundance v modelu
8. Posouzení s uživateli

5.2.3 Transformace do relačního modelu dat

Konceptuální prvek	Relační prvek
Entity	Tabulky
Atributy	Sloupce
1:N vztahy	Cizí klíč (FK)
M:N vztahy	Asociativní tabulky (2 FK)
Identifikátory	Primární klíče (PK)
Volitelnost	NULL pro FK
Povinnost	NOT NULL pro FK

5.2.4 Modely databázových systémů

- **Hierarchický model** – stromová struktura; vztah rodič/potomek; uzel má právě jednoho rodiče; obtížné vkládání a rušení.
Příklad: organizační struktura firmy – Firma → Oddělení → Zaměstnanec; nebo souborový systém (disk → složka → soubor). Historicky IBM IMS (1960s). Problém: zaměstnanec nemůže patřit do dvou oddělení – hierarchie to neumožňuje.
- **Síťový model** – zobecnění hierarchického; uzel může mít *více rodičů*; vícenásobné vztahy (sety).
Příklad: kusovník (bill of materials) – součástka může být součástí více výrobků zároveň; nebo student zapsaný do více kurzů. Standard CODASYL (1970s), databáze IDMS. Flexibilnější než hierarchický, ale složitá navigace.
- **Relační model** – Dr. Codd 1970; data v tabulkách (řádky/sloupce); vztahy přes cizí klíče; nejčastěji používaný.
Příklad: prakticky vše – e-shopy, bankovníctví, ERP systémy. MySQL, PostgreSQL, Oracle, MS SQL Server. Výhoda: flexibilní dotazy přes SQL, silná teorie (normální formy, transakce ACID).
- **Objektově-orientované DBS (OODBS)** – data jako objekty se stavem (atributy) a chováním (metody); OID (unikátní identifikátor objektu), zapouzdření, dědičnost, polymorfismus.

Příklad: CAD/CAM systémy (komplexní geometrické objekty), multimediální databáze, vědecké aplikace kde objekty mají složitou strukturu (DNA sekvence). db4o, ObjectDB. V praxi méně rozšířené – většina aplikací používá relační DB s ORM.

Princip tří architektur (P3A) Klíčový pojem z otázek profesorů. P3A říká, že každý databázový systém lze popsat na třech úrovních abstrakce – od pohledu uživatele až po fyzické uložení na disku:

1. **Konceptuální** (co systém eviduje) – entity, vztahy, omezení; nezávislé na technologii; ER diagram
2. **Logická** (jak to uložíme) – tabulky, sloupce, PK/FK, normalizace; nezávislé na konkrétním DBMS
3. **Fyzická** (jak to nasadíme) – DDL skripty, indexy, partitioning; specifické pro daný DBMS

Smyslem P3A je **oddělení zodpovědností**: analytik pracuje na konceptuální úrovni se zákazníkem, databázový návrhář na logické, DBA na fyzické. Změna fyzického uložení (přechod z Oracle na PostgreSQL) neovlivní konceptuální model.

Otázky profesorů k tématu: Datové modelování

Řepa Václav, prof. Ing., CSc.

- Najděte ve Vašem relačním modelu potenciální nedostatky. Jaké jsou možné přístupy k jejich eliminaci?
- Jak jsou realizovány vztahy mezi entitami v relačních databázích?
- Jaké architektury návrhu datového modelu znáte (princip 3 architektur)?
- Jaké znáte normální formy?

Pour Jan, doc. Ing., CSc.

- Jaké jsou principy a problémy datového modelování?
- Modelování podnikových systémů – jaké jsou hlavní principy?

Maryška Miloš, doc. Ing., Ph.D.

- Datové modelování a jeho vrstvy.

5.3 Databázové jazyky

5.3.1 Dělení databázových jazyků

Důležité: DDL, DML, DCL a TCL *nejdou* samostatné jazyky – jsou to **kategorie příkazů** v rámci SQL (nebo jiného databázového jazyka). Když napíšete **CREATE TABLE**, píšete stále SQL – konkrétně příkaz kategorie DDL. Když napíšete **SELECT**, píšete SQL kategorie DML.

Kategorie	Příkazy	Co dělá
DDL Data Definition	CREATE, ALTER, DROP, TRUNCATE	Definuje <i>strukturu</i> databáze – tvoří a mění tabulky, indexy, pohledy
DML Data Manipulation	SELECT, INSERT, UPDATE, DELETE	Pracuje s <i>daty</i> v tabulkách – čte, vkládá, mění, maže záznamy
DCL Data Control	GRANT, REVOKE	Řídí <i>přístupová práva</i> – kdo smí co dělat s jakými objekty
TCL Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	Řídí <i>transakce</i> – potvrzení nebo zrušení skupiny změn

Příklad: v jednom SQL skriptu klidně smícháte všechny kategorie:

Listing 15: Příkazy ze všech čtyř kategorií SQL

```
-- DDL: vytvoření struktury
CREATE TABLE projekt (id INT PRIMARY KEY, nazev VARCHAR(100));

-- DCL: přidělení práv
GRANT SELECT, INSERT ON projekt TO uživatel_app;

-- TCL + DML: transakce s daty
BEGIN TRANSACTION;
  INSERT INTO projekt VALUES (1, 'ERP_implementation');
  UPDATE projekt SET nazev = 'ERP_2025' WHERE id = 1;
COMMIT;    -- nebo ROLLBACK pro zrušení
```

5.3.2 Vyhledávací interface pro relační databáze

Vyhledávací interface je způsob, jakým aplikace nebo uživatel *komunikuje* s databází za účelem získání dat. Existují tři úrovně:

1. Úroveň jazyka – SQL SELECT Základním vyhledávacím rozhraním je příkaz **SELECT** ze standardu SQL. Jde o *deklarativní* rozhraní – říkáte co chcete, databázový stroj sám rozhodne jak to načte.

Listing 16: Anatomie SQL SELECT – plný dotaz

```
SELECT  z.jmeno, o.nazev AS oddeleni, COUNT(p.id) AS pocet_projektu
FROM    zamestnanci z
JOIN    oddeleni o    ON z.id_oddel = o.id
LEFT JOIN projekty p  ON p.id_zam  = z.id
WHERE   z.plat > 40000
        AND  z.aktivni = 1
GROUP BY z.jmeno, o.nazev
```

```

HAVING COUNT(p.id) >= 2
ORDER BY pocet_projektu DESC
LIMIT 10;

```

Pořadí vyhodnocení (NE pořadí zápisu): FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Rozšíření SQL vyhledávání:

- **Poddotaz (subquery)** – SELECT uvnitř jiného SELECT; WHERE plat > (SELECT AVG(plat) FROM zamestnanci)
- **VIEW (pohled)** – pojmenovaný uložený dotaz; tváří se jako tabulka; skryje složitost joinu; CREATE VIEW aktivni_zam AS SELECT ...
- **CTE (Common Table Expression)** – dočasný pojmenovaný poddotaz pro čitelnost; WITH TopPlaty AS (SELECT ...) SELECT * FROM TopPlaty

Rozdíl VIEW vs CTE:

- **VIEW** je *trvalý* – uložen v databázi, vidí ho všichni uživatelé, existuje mezi dotazy; lze na něj přidělovat práva (GRANT SELECT ON view TO user); vhodný když stejný složitý dotaz používáš opakovaně z více míst
- **CTE** je *dočasný* – existuje pouze v rámci jednoho dotazu, po jeho skončení zmizí; vhodný pro zpřehlednění složitého dotazu nebo rekurzivní dotazy (hierarchie, stromy); nepotřebuješ CREATE ani DROP

Příklad: Chceš zobrazit zaměstnance s platem nad průměrem. Jako VIEW: CREATE VIEW nadprumerny_plat AS SELECT * FROM zamestnanci WHERE plat > (SELECT AVG(plat) FROM zamestnanci) – použiješ to z deseti různých aplikací. Jako CTE: napíšeš WITH Prumer AS (SELECT AVG(plat) avg FROM zamestnanci) SELECT * FROM zamestnanci, Prumer WHERE plat > avg – jednou, v jednom dotazu, bez nutnosti čehokoli ukládat.

2. Úroveň programového API – JDBC, ODBC, ADO.NET Aplikace (Java, C#, Python...) se s databází nepropojují přímo – používají **standardizované ovladače (drivery)**, které přeloží volání aplikace na síťový protokol databáze.

API	Jazyk	Jak funguje	Příklad
JDBC	Java	Connection	→ PostgreSQL
		PreparedStatement ResultSet	→ JDBC driver
ODBC	C/C++, universal	unifikované C API; driver manager	MS Access, Excel
ADO.NET	C# / .NET	SqlConnection, SqlCommand, SqlDataReader	MS SQL Server
DB-API 2.0	Python	cursor.execute(), fetchall()	psycopg2, sqllite3

Listing 17: JDBC – dotaz z Java aplikace


```
String sql = "SELECT _jmeno, _plat FROM _zamestnanci WHERE _plat > _?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setDouble(1, 40000);           // parametr -- ochrana proti
    SQL injection
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        System.out.println(rs.getString("jmeno") + ": _" + rs.
            getDouble("plat"));
    }
}
```

Tok komunikace: *Aplikace* → *JDBC/ODBC driver* → *síť (TCP/IP)* → *DB server* → zpět výsledek jako *ResultSet*.

3. Úroveň ORM – abstrakce nad SQL ORM (Object-Relational Mapping) mapuje databázové tabulky na objekty v programovacím jazyce. Vývojář nepíše SQL ručně – framework ho generuje automaticky.

ORM	Jazyk	Dotazovací API
Hibernate / JPA	Java	JPQL, Criteria API, <code>entityManager.find()</code>
Entity Framework	C# / .NET	LINQ to Entities
SQLAlchemy	Python	<code>session.query(User).filter(...)</code>
ActiveRecord	Ruby on Rails	<code>User.where(plat: 40000..)</code>

Listing 18: JPA – dotaz bez psaní SQL

```
// JPQL dotaz přes ORM (Hibernate vygeneruje SQL automaticky)
List<Zamestnanec> vysledek = em.createQuery(
    "SELECT _z FROM _Zamestnanec _z WHERE _z.plat > _:minPlat",
    Zamestnanec.class)
    .setParameter("minPlat", 40000.0)
    .getResultList();
```

Výhody ORM: nezávislost na DB (přepnutí Oracle→PostgreSQL = změna konfigurace); objektový přístup; ochrana proti SQL injection. **Nevýhody:** generované SQL může být neefektivní; složité dotazy se píšou obtížně; **N+1 problém** – klasická past ORM: načteš seznam N objednávek (1 dotaz), a pak pro každou objednávku zvlášť načteš zákazníka (N dotazů) = celkem N+1 dotazů do DB místo jednoho JOINu. *Příklad:* 100 objednávek = 1 SELECT na objednávky + 100 SELECT na zákazníky = 101 dotazů. S JOINem by to byl 1 dotaz. Řešení: JOIN FETCH v JPQL nebo `Include()` v Entity Framework – explicitně řekneš ORM, ať načte vše najednou.

5.3.3 SQL (Structured Query Language)

Standardizovaný dotazovací jazyk pro relační databáze. Vznikl z jazyka SEQUEL (IBM, 70. léta). Standardizován organizacemi ISO, IEC a ANSI.

Rok	Standard	Poznámka
1986	SQL-86	První verze
1992	SQL-92	SQL2; komplexní pohled, tři úrovně
1999	SQL:1999	SQL3; regulární výrazy, trigger, OO vlastnosti
2003	SQL:2003	Podpora XML
2008	SQL:2008	Kompletní přepracování

Listing 19: Příklady SQL příkazů

```
-- DDL: vytvoření tabulky
CREATE TABLE zamestnanci (
    ID_zam      INT          PRIMARY KEY,
    jmeno       VARCHAR(50)  NOT NULL,
    plat        DECIMAL(10,2)
);

-- DML: dotaz se spojením tabulek
SELECT z.jmeno, o.nazev AS oddeleni
FROM zamestnanci z
JOIN oddeleni o ON z.oddeleni_id = o.id
WHERE z.plat > 40000;

-- TCL: transakce
BEGIN TRANSACTION;
    UPDATE ucty SET zustatek = zustatek - 1000 WHERE id = 1;
    UPDATE ucty SET zustatek = zustatek + 1000 WHERE id = 2;
COMMIT;
```

5.3.4 PL/SQL

Procedural Language/SQL – procedurální nadstavba SQL od Oracle (bazovaná na Adě). Umožňuje: uložené procedury a funkce, programové balíky, trigger, uživatelsky definované typy.

5.3.5 T-SQL (Transact-SQL)

T-SQL je proprietární rozšíření SQL od společnosti Microsoft, používané v **MS SQL Server** a **Azure SQL Database**. Zatímco standardní SQL je deklarativní (říkáš co chceš), T-SQL přidává procedurální prvky (říkáš jak to udělat krok za krokem).

Co T-SQL přidává oproti standardnímu SQL

- **Proměnné a řídicí struktury** – DECLARE, IF/ELSE, WHILE, BEGIN/END (bloky kódu)
- **Ošetření chyb** – TRY/CATCH/THROW (podobné try-catch v Javě)
- **Uložené procedury a funkce** – logika uložená přímo v databázi

- **Triggery** – automaticky spouštěný kód při INSERT/UPDATE/DELETE
- **Kurzory** – procházení výsledné sady řádek po řádku
- **CTE (Common Table Expressions)** – pojmenované dočasné poddotazy s WITH
- **MERGE** – kombinace INSERT/UPDATE/DELETE v jednom příkazu (UPSERT)

Listing 20: T-SQL – ukázka procedurálních prvků

```
-- Proměnné a IF/ELSE
DECLARE @plat DECIMAL(10,2) = 45000;
DECLARE @zprava VARCHAR(100);

IF @plat > 50000
    SET @zprava = 'Senior';
ELSE IF @plat > 30000
    SET @zprava = 'Junior';
ELSE
    SET @zprava = 'Trainee';

PRINT @zprava;  -- vypise 'Junior'

-- TRY/CATCH pro ošetření chyb
BEGIN TRY
    BEGIN TRANSACTION;
        UPDATE ucty SET zustatek = zustatek - 1000 WHERE id = 1;
        UPDATE ucty SET zustatek = zustatek + 1000 WHERE id = 2;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Chyba:␣' + ERROR_MESSAGE();
END CATCH;

-- CTE (Common Table Expression)
WITH VysokyPlat AS (
    SELECT jmeno, plat
    FROM zamestnanci
    WHERE plat > 50000
)
SELECT * FROM VysokyPlat ORDER BY plat DESC;

-- Uložená procedura
CREATE PROCEDURE ZvysPlatOddeleni
    @id_oddeleni INT,
    @procent DECIMAL(5,2)
AS
BEGIN
    UPDATE zamestnanci
    SET plat = plat * (1 + @procent / 100)
    WHERE id_oddeleni = @id_oddeleni;
END;
```

```
-- Volání procedury
EXEC ZvysPlatOddeleni @id_oddeleni = 10, @procent = 5;
```

T-SQL vs PL/SQL vs PL/pgSQL

Vlastnost	T-SQL	PL/SQL	PL/pgSQL
Platforma	MS SQL Server, Azure	Oracle DB	PostgreSQL
Proměnné	DECLARE @var	v_var TYPE	v_var TYPE
Chyby	TRY/CATCH	EXCEPTION	EXCEPTION
Kurzory	CURSOR FOR	CURSOR	CURSOR
Licence	Proprietární	Proprietární	Open source

5.3.6 OQL (Object Query Language)

Standardizovaný dotazovací jazyk pro objektově-orientované databáze (ODMG). Podobný SQL-92. Neobsahuje příkazy pro definici dat (CREATE TABLE apod.). Kvůli komplexitě neimplementován v úplnosti.

Co je objektově-orientovaná databáze? Zatímco relační DB ukládá data do tabulek (řádky a sloupce), objektově-orientovaná DB ukládá data přímo jako **objekty** – stejně jako objekty v OOP jazycích (Java, C#). Každý objekt má:

- **Atributy** (stav) – jako pole třídy
- **Metody** (chování) – logika přímo u dat
- **OID** (Object Identifier) – unikátní identifikátor objektu v celé DB; na rozdíl od PK v relační DB je OID generován systémem a nemění se
- **Dědičnost** – třída **Student** může dědit z třídy **Osoba**; DB ví o hierarchii tříd

Proč vznikla? V 90. letech narážely aplikace psané v OOP jazycích na *impedance mismatch* (impedanční nesoulad) – objekt v Javě má složitou strukturu (vnořené objekty, kolekce, metody), ale relační DB ho musela rozkrájet do tabulek a zpět skládat přes JOIN. OO databáze tento problém řeší – objekt uložíš a načteš přímo, bez překladu.

Příklad: Objekt **Objednavka** obsahuje seznam objektů **Polozka**, každá **Polozka** obsahuje objekt **Produkt**. V relační DB = 3 tabulky + 2 JOINy. V OO databázi = jeden objekt s vnořenými kolekcemi, uložený a načtený jako celek.

V praxi se OO databáze příliš nerozšířily – většina vývojářů dává přednost relační DB s ORM (Hibernate, Entity Framework), které impedanční nesoulad řeší na aplikační vrstvě. OO DB se používají v CAD/CAM systémech, vědeckých aplikacích nebo všude, kde jsou objekty příliš složité pro relační model.

5.3.7 JPQL (Java Persistence Query Language)

Platformně nezávislý OO dotazovací jazyk, součást JPA specifikace. Dotazuje se na entity uložené v relační DB. Programátor se nestará o konkrétní databázi (Oracle vs MySQL).

5.3.8 XQuery

Dotazovací a funkcionální jazyk pro dotazování na XML data. Používá syntax XPath pro adresování XML dokumentů, doplněnou SQL-podobnými **FLWOR** výrazy: FOR, LET, WHERE, ORDER BY, RETURN.

5.3.9 Datové formáty pro výměnu dat

- **CSV (Comma-Separated Values)** – řádkový textový formát; hodnoty odděleny čárkou (nebo středníkem); první řádek = záhlaví; jednoduchý, podporován všemi nástroji; nevýhoda: žádný datový typ, žádná hierarchie
- **JSON (JavaScript Object Notation)** – textový formát pro strukturovaná data; klíč-hodnota páry, pole, vnořené objekty; nativní pro JavaScript a REST API; čitelnější než XML; nevýhoda: bez schématu
- **XML (eXtensible Markup Language)** – stromová struktura; značky definuje uživatel; validace pomocí DTD nebo XML Schema (XSD); vhodný pro dokumenty a konfiguraci; nevýhoda: verbose, složitější zpracování

Vlastnost	CSV	JSON	XML
Struktura	Plochá tabulka	Hierarchická	Hierarchická
Čitelnost	Vysoká	Střední	Nízká
Datové typy	Ne	Základní	S XSD
Schéma	Ne	JSON Schema	DTD/XSD
Použití	Import/export dat	REST API	Dokumenty, SOAP

5.3.10 SQL Injection

Útok, kdy útočník vloží škodlivý SQL kód do vstupního pole aplikace.

Listing 21: Příklad SQL injection

```
-- Zranitelný dotaz (vstup: ' OR '1'='1')
SELECT * FROM users WHERE username='' OR '1'='1' AND password='';
-- Vrátí všechny záznamy!
```

Ochrana: prepared statements, ORM frameworky, validace vstupů, princip nejmenšího oprávnění, WAF.

Otázky profesorů k tématu: Databázové jazyky

Strossa Petr, doc. RNDr., CSc.

- Jaký je vyhledávací interface pro relační databáze?

Sedláček Jiří, Ing., Ph.D.

- Jaké typy útoků jsou časté při dotazování databází přes webové skriptovací jazyky? Jak se jim bránit?

Komise: Vojř, Pour, Sudzina

- Jaké jsou výhody a nevýhody NoSQL databází?

Maryška Miloš, doc. Ing., Ph.D.

- Popište architekturu datově orientovaného řešení.

6 Datová analytika

6.1 Přidaná hodnota dat pro business

6.1.1 Data jako strategický aktiv

Data jsou v moderním podnikání považována za cenný aktiv – srovnatelný s hmotným majetkem. Přidaná hodnota dat spočívá v podpoře rozhodování na všech úrovních řízení.

Hierarchie dat – DIKW pyramid

- **Data** – nezpracovaná fakta, čísla, symboly bez kontextu
- **Informace** – data s kontextem a strukturou; odpovídají na “kdo, co, kdy, kde”
- **Znalost (Knowledge)** – informace s porozuměním; odpovídá na “jak”; víme *jak* věci fungují a jak je využít
- **Moudrost (Wisdom)** – znalost s hodnotovým soudem; odpovídá na “proč”

Co znamená “znalost s hodnotovým soudem”? Znalost říká *jak* něco udělat. Moudrost navíc říká *jestli to vůbec dělat* – přidává zkušenost, etiku a schopnost posoudit dlouhodobé důsledky. Jde o úsudek, který nelze odvodit jen z dat, ale vyžaduje lidský kontext a hodnoty.

Příklad – tržby firmy klesly o 30 %:

Úroveň	Výrok	Popis
Data	“−30 %, Q3 2024”	Surové číslo bez vysvětlení
Informace	“Tržby klesly kvůli ztrátě 3 klíčových klientů”	Číslo dostalo kontext – víme <i>co</i> se stalo
Znalost	“Víme, že udržení klientů vyžaduje lepší zákaznický servis a flexibilní ceny”	Víme <i>jak</i> problém řešit (ze zkušeností, analýz)
Moudrost	“I přes krátkodobé náklady bychom měli investovat do vztahů se zákazníky – krátkozraké snižování cen by nás poškodilo dlouhodobě”	Hodnotový soud: rozhodnutí co je důležitější (krátkodobý zisk vs. dlouhodobé vztahy); vyžaduje zkušenost a etiku

Jiný příklad – lékař: Data = teplota 38,5 °C. Informace = pacient má horečku. Znalost = horečka se léčí antipyretiky a klidem. Moudrost = u tohoto konkrétního staršího pacienta se srdečním onemocněním je riziko léčby vyšší než přínos – raději budeme monitorovat. **Hodnotový soud** = posouzení co je v dané situaci správné, na základě zkušenosti a hodnot (primum non nocere).

Obchodní hodnota dat

- **Operativní rozhodování** – automatizované, v reálném čase (fraud detection, doporučovací systémy)
- **Taktické rozhodování** – střednědobé; optimalizace procesů, cen, zásob
- **Strategické rozhodování** – dlouhodobé; identifikace trendů, nových trhů
- **Monetizace dat** – přímý prodej dat, datové produkty (API), personalizace

4V dat (Big Data)

- **Volume** – objem; terabajty až petabajty
- **Velocity** – rychlost generování a zpracování
- **Variety** – různorodost formátů (strukturovaná, semi-strukturovaná, nestrukturovaná)
- **Veracity** – spolehlivost a přesnost dat

6.2 Kvalita dat (Data Quality)

6.2.1 Dimenze kvality dat

Kvalita dat je míra, do jaké data splňují požadavky pro svůj zamýšlený účel.

- **Přesnost (Accuracy)** – data odpovídají realitě (věk zákazníka je správný)
- **Úplnost (Completeness)** – chybějící hodnoty (NULL, prázdné pole)
- **Konzistence (Consistency)** – data jsou v souladu v rámci celého systému (stejně jméno v různých tabulkách)
- **Aktuálnost (Timeliness)** – data jsou k dispozici, když jsou potřeba
- **Jedinečnost (Uniqueness)** – žádné duplicitní záznamy
- **Validita (Validity)** – data splňují definovaný formát a rozsah (datum ve správném formátu)
- **Integrita (Integrity)** – referenční integrita zachována (FK odkazuje na existující PK)

6.2.2 Příčiny špatné kvality dat

- Ruční zadávání dat – překlepy, chyby
- Integrace více zdrojů – různé formáty, kódování, konvence
- Zastaralost dat – zákazník se přestěhoval, firma zanikla
- Chybějící validace na vstupu

- Špatně navržené databáze

6.2.3 Data Quality Management

Proces zajišťování a udržování kvality dat:

1. **Profilování dat** – analýza struktury, obsahu a vztahů v datech
2. **Čištění dat (Data Cleansing)** – oprava nebo odstranění nesprávných dat
3. **Standardizace** – jednotný formát (PSČ, telefonní čísla)
4. **Deduplikace** – odstranění duplicit (Record Linkage, fuzzy matching)
5. **Obohacení dat** – doplnění z externích zdrojů
6. **Monitoring** – průběžné sledování kvality dat (KQI – Key Quality Indicators)

Master Data Management (MDM) Centrální správa klíčových podnikových dat (zákazníci, produkty, zaměstnanci) – jeden “zlatý záznam”.

Lidsky: Ve velké firmě má zákazník „Jan Novák” záznam v CRM systému, v ERP systému, v e-shopu a v účetním systému – a v každém systému může být trochu jinak zapsaný (jiný formát adresy, překlep v názvu, jiné IČO). MDM říká: vytvoříme **jeden autoritativní zdroj pravdy** („zlatý záznam”) pro každý klíčový objekt, a všechny systémy se na něj odkazují místo toho, aby si každý vedl vlastní kopii.

Co MDM řeší:

- **Duplicity** – stejný zákazník evidovaný vícekrát pod různými záznamy
- **Nekonzistence** – „Novák s.r.o.” vs „NOVÁK S.R.O.” vs „Novák, s.r.o.” – třikrát ta samá firma
- **Silo efekt** – každé oddělení má svá data, která se neshodují s ostatními

Příklad: Banka má 5 systémů. Zákazník změní adresu na přepážce – bez MDM musí pracovník aktualizovat 5 systémů ručně a v každém může udělat chybu. S MDM aktualizuje jednou v centrálním registru a všechny systémy si změnu stáhnou automaticky.

6.3 Datový sklad, OLAP a Business Intelligence

6.3.1 Datový sklad (Data Warehouse)

Centrální úložiště integrovaných dat z různých zdrojů, optimalizované pro analytické dotazy.

Proč nestačí běžná relační databáze (OLTP)?

Relační databáze (MySQL, PostgreSQL) jsou navrženy pro **OLTP** – tisíce malých rychlých transakcí za sekundu (objednávka, platba, aktualizace skladu). Datový sklad je navržen pro **OLAP** – málo dotazů, ale každý prochází miliony řádků a počítá agregace. Jsou to dva zásadně odlišné případy použití:

Vlastnost	OLTP (relační DB)	DWH / OLAP
Účel	Provozní transakce	Analytika, reporty
Dotazy	Malé, rychlé (ms)	Velké, pomalé (sekundy–minuty)
Objem dat	Aktuální stav	Historická data (roky)
Schéma	Normalizované (3NF)	Denormalizované (hvězda)
Zdroj dat	Jeden systém	Více systémů (ERP, CRM...)
Záписы	Časté INSERT/UPDATE	Dávkové načítání (ETL)
Uložení	Řádkové	Sloupcové (čte jen potřebné sloupce)

Příklad: Dotaz „jaký byl obrat elektroniky po regionech v každém měsíci roku 2023“ prochází miliony řádků a dělá GROUP BY přes 3 dimenze. Na produkční OLTP databázi by:

- **trval minuty** – normalizované schéma vyžaduje mnoho JOINů; OLTP indexy jsou optimalizované pro hledání jednoho záznamu, ne pro skenování milionů
- **zpomalil produkci** – analytický dotaz zamkne tabulky a brání ostatním uživatelům v přidávání objednávek
- **neměl historická data** – OLTP typicky uchovává jen aktuální stav; smazaná nebo přepsaná data jsou pryč
- **neměl data z jiných systémů** – prodeje jsou v e-shopu, náklady v ERP, zákazníci v CRM – OLTP to nedá dohromady

Charakteristiky datového skladu (W. Inmon):

- **Subjektově orientovaný** – data organizována kolem podnikových subjektů (zákazník, produkt), ne procesů
- **Integrovaný** – data z různých zdrojů sjednocena (jednotné formáty, kódování)
- **Časově variantní** – historická data; záznamy obsahují časové razítko
- **Neměnný (Non-volatile)** – data se mazají jen zřídka; přidávají se nová

Architektura datového skladu

- **Staging area** – dočasné uložení surových dat ze zdrojů
- **ETL vrstva** – Extract, Transform, Load
- **Data Warehouse** – centrální úložiště; normalizované (3NF) nebo dimenzionální schéma
- **Data Mart** – podmnožina DW pro konkrétní oddělení (finance, marketing)
- **OLAP engine** – analytické zpracování
- **Reporting/BI nástroje** – vizualizace a reporty

Jak vypadá tok dat při příchodu do datového skladu? Představ si e-shop, který každý den zpracuje tisíce objednávek a chce analyzovat prodeje:

1. **Zdrojové systémy generují data** – transakční databáze (OLTP) e-shopu zapíše novou objednávku; ERP systém aktualizuje sklad; CRM zaznamená kontakt se zákazníkem. Tyto systémy jsou optimalizované na rychlé zápisy, ne na analytiku.
2. **Extract – extrakce ze zdrojů** – ETL/ELT nástroj v pravidelných intervalech (každou noc nebo průběžně) vytáhne nová data ze zdrojů. Buď celou tabulku (full load), nebo jen změny od posledního načtení (inkrementální – CDC).
3. **Staging area – přistávací plocha** – surová data se uloží dočasně beze změn. Vypadají přesně tak, jak přišla ze zdroje – s chybami, duplicitami, různými formáty. Slouží jako bezpečnostní síť: pokud transformace selže, máme původní data k dispozici.
4. **Transform – čištění a přeformátování** – zde se data upraví: opraví se chyby (NULL hodnoty, překlepy), sjednotí formáty (datum z různých systémů do jednoho formátu), odstraní duplicitu, spočítají odvozené hodnoty (marže, sleva).
5. **Load – načtení do DWH** – vyčištěná data se načtou do datového skladu do dimenzionálního schématu (hvězda/sněhová vločka). Fakta (prodeje, transakce) se spojí s dimenzemi (čas, zákazník, produkt, region).
6. **Data Mart – výřezy pro oddělení** – z centrálního DWH se vytvoří specializované pohledy: marketingový data mart (kampaně, konverze), finanční data mart (tržby, náklady), logistický (sklady, dodací doby).
7. **OLAP + BI nástroje – analytika a reporty** – analytici a manažeři se připojí přes Power BI, Tableau nebo Excel a táhnou data. Otázka „jaký byl obrat po regionech v Q3 ve srovnání s loňskem“ se zodpoví během sekund – to by v OLTP databázi trvalo minuty a zatížilo by produkční systém.

Celý tok: OLTP systémy → Staging area → ETL/ELT → Data Warehouse → Data Mart → BI nástroje → manažer

6.3.2 ETL (Extract, Transform, Load)

1. **Extract** – extrakce dat ze zdrojových systémů (OLTP DB, CSV, ERP, CRM, API)
 - Plný výběr (full load) nebo inkrementální (CDC – Change Data Capture)
2. **Transform** – transformace dat:
 - Čištění (oprava chyb, NULL hodnoty)
 - Standardizace formátů
 - Agregace, výpočty odvozených hodnot
 - Deduplikace
 - Obohacení z referenčních dat
3. **Load** – načtení do cílového systému

- Plné přepsání (truncate and reload)
- Inkrementální (pouze nová nebo změněná data)
- SCD (Slowly Changing Dimensions) – správa historických změn dimenzí

ELT – modernější alternativa **ELT (Extract, Load, Transform)** obrátí pořadí: data se nejprve *načtou surová* do cílového datového skladu a teprve pak se tam transformují. Transformaci provádí výpočetní výkon samotného DWH, ne oddělený ETL server.

	ETL	ELT
Tok	Zdroj → ETL server (transformace) → DWH	Zdroj → DWH raw vrstva → DWH (transformace)
Kde se transformuje	Na dedikovaném ETL serveru (mimo DWH)	Přímo v DWH pomocí SQL / dbt
Raw data v DWH	Ne – načítají se až vyčištěná data	Ano – raw data jsou vždy dostupná pro re-transformaci
Výkon	ETL server je úzké hrdlo; škálování je drahé	Cloud DWH (Snowflake, BigQuery) škálují výpočty elasticky
Flexibilita	Změna logiky = přepsání ETL pipeline + reload	Změna logiky = úprava SQL modelu; data jsou stále v DWH
Typické nástroje	Informatica, SSIS, Talend, Pentaho	dbt, Fivetran + dbt, BigQuery, Snowflake
Vhodné pro	On-premise; citlivá data (GDPR – anonymizace před načtením); starší systémy	Cloud DWH; velké objemy; moderní datové platformy

Proč je ELT v moderním prostředí lepší:

1. **Výpočetní výkon cloudu** – DWH jako Snowflake nebo BigQuery mají masivní paralelní zpracování; transformace milionů řádků trvá sekundy; ETL server by byl bottleneck
2. **Raw data jsou vždy k dispozici** – pokud se změní businessová logika, stačí přepsat transformaci a pustit ji znovu; u ETL bychom museli data znovu extrahovat ze zdrojů
3. **Jednoduchost a verzování** – transformace jsou čisté SQL modely; nástroj **dbt** (data build tool) je verzuje v Gitu, testuje a dokumentuje jako kód
4. **Schema-on-read** – schéma výstupních tabulek se definuje při transformaci, ne při načítání; snazší adaptace na změny zdrojových systémů

Kdy stále použít ETL:

- Citlivá data (PII, zdravotní záznamy) – GDPR vyžaduje anonymizaci *před* uložením
- On-premise DWH s omezenou kapacitou (nechceme ukládat surová data)
- Starší infrastruktura s existujícími ETL pipelines (refaktoring by byl dražší než přínos)

6.3.3 Dimenzionální modelování

Dimenzionální modelování (Ralph Kimball, 1990s) je přístup k návrhu datového skladu optimalizovaný pro **analytické dotazy**. Liší se od normalizovaného relačního modelu – záměrně připouští redundanci výměnou za jednoduchost a rychlost čtení.

Základní myšlenka: každý businessový fakt (prodej, klik, platba) se dá popsat dvěma typy informací:

- **Číselná měření** – *co se stalo a za kolik?* (tržba, počet kusů, marže)
- **Kontext** – *kdo, co, kde, kdy?* (zákazník, produkt, pobočka, datum)

Z toho vznikají dva typy tabulek: **tabulka faktů** a **dimenzionální tabulky**.

Tabulka faktů (Fact Table) Uchovává **měřitelné události** – jeden řádek = jeden businessový fakt na dané granularitě.

- Obsahuje **metriky** (číselné hodnoty k agregaci): tržba, množství, cena, marže
- Obsahuje pouze **cizí klíče** na dimenzionální tabulky (popis je v dimenzích, ne zde)
- **Granularita** – nejdůležitější rozhodnutí návrhu: co představuje jeden řádek?
 - Hrubá granularita: jeden řádek = celá objednávka (snadnější, méně řádků)
 - Jemná granularita: jeden řádek = jedna položka na objednávce (flexibilnější, více řádků)

FAKT_PRODEJ (jeden řádek = jedna prodaná položka)						
<u>id_prodeje</u>	id_datum (FK)	id_zakaznik (FK)	id_produkt (FK)	id_pobocka (FK)	mnozstvi	trzba_kc
1	20240315	1042	501	3	2	1 198
2	20240315	1042	207	3	1	299
3	20240316	871	501	1	1	599

Tučně jsou metriky (`mnozstvi`, `trzba_kc`), vše ostatní jsou FK do dimenzí.

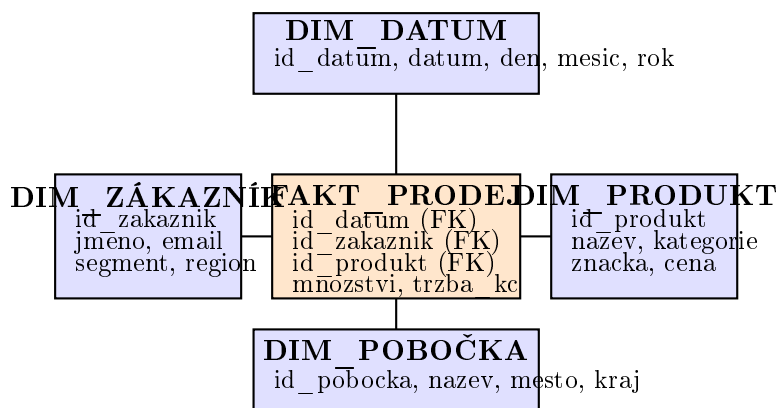
Dimenzionální tabulky (Dimension Tables) Poskytují **kontext a popis** k faktům. Jsou denormalizované – záměrně obsahují redundanci, aby se dotazy psaly jednoduše bez mnoha joinů.

DIM_PRODUKT			
id_produkt	nazev	kategorie	znacka
501	Knihy Java	Knihy	O'Reilly
207	Bezdrátová myš	Periferie	Logitech

DIM_DATUM				
id_datum	datum	den_tydne	mesic	rok
20240315	2024-03-15	Pátek	Březen	2024
20240316	2024-03-16	Sobota	Březen	2024

Typické dimenze: **Čas** (den/týden/měsíc/rok/čtvrtletí), **Zákazník** (jméno/segment/region), **Produkt** (název/kategorie/značka), **Pobočka/Lokalita** (město/kraj/země).

Hvězdicové schéma (Star Schema) Centrální tabulka faktů je obklopena dimenzionálními tabulkami – vizuálně připomíná hvězdu. Dimenze **nejsou normalizovány** (obsahují redundanci).



Obrázek 11: Hvězdicové schéma (Star Schema) – e-shop prodeje

Výhody: jednoduché dotazy (málo joinů), rychlé čtení, srozumitelné pro business uživatele. **Nevýhody:** redundance v dimenzích (jméno zákazníka se opakuje).

Sněhové schéma (Snowflake Schema) Dimenzionální tabulky jsou **normalizovány** – kategorie produktu je v samostatné tabulce, kraj zákazníka v jiné. Vizuálně připomíná sněhovou vločku.

Výhody: méně redundance, menší objem dat. **Nevýhody:** více joinů = pomalejší dotazy; složitější pro analytiku.

V praxi se používá hvězdicové schéma – rychlost dotazů je důležitější než úspora místa.

Listing 22: Tržby podle kategorie a měsíce – typický DWH dotaz
Příklad analytického dotazu nad hvězdicovým schématem

```
SELECT  p.kategorie,
        d.mesic,
        d.rok,
        SUM(f.trzba_kc) AS celkova_trzba,
```

```

COUNT(*) AS pocet_prodanu
FROM FAKT_PRODEJ f
JOIN DIM_PRODUKT p ON f.id_produkt = p.id_produkt
JOIN DIM_DATUM d ON f.id_datum = d.id_datum
WHERE d.rok = 2024
GROUP BY p.kategorie, d.mesic, d.rok
ORDER BY d.mesic, celkova_trzba DESC;

```

6.3.4 OLAP (Online Analytical Processing)

OLAP umožňuje interaktivní analýzu dat z více perspektiv (dimenzí).

Co to znamená „z více perspektiv“? Představ si kostku (OLAP cube) – máš data o tržbách e-shopu a na každé ose kostky je jiná dimenze: *čas* (leden, únor...), *region* (Praha, Brno, Ostrava...), *kategorie produktu* (elektronika, oblečení...). Každá buňka kostky obsahuje metriku (tržba v Kč). OLAP ti umožní kostkou interaktivně otáčet a říznout ji z libovolného úhlu – bez čekání na IT, bez psaní SQL.

Příklad průchodu analytikem:

1. Manažer se podívá na celkové tržby za rok 2024 (nejvyšší úroveň)
2. Vidí, že Q3 je slabý → **drill-down** na měsíce: červenec, srpen, září
3. Září je nejhorší → **slice**: zobrazí jen září, všechny regiony
4. Vidí, že Ostrava výrazně zaostává → **dice**: září + Ostrava + všechny kategorie
5. Zjistí, že elektronika v Ostravě v září klesla → **pivot**: otočí pohled a porovná elektroniku v Ostravě napříč všemi měsíci roku
6. **Roll-up** zpět na kvartály pro prezentaci vedení

Celý tento průchod trvá minuty v Power BI nebo Tableau – bez OLAP by každý krok byl samostatný SQL dotaz přes IT oddělení.

OLAP operace

- **Drill-down** – přechod na nižší úroveň granularity (rok → čtvrtletí → měsíc → den)
- **Roll-up (Drill-up)** – agregace na vyšší úroveň (měsíc → rok)
- **Slice** – výběr jedné hodnoty jedné dimenze (prodeje pouze pro rok 2023); zmenší kostku o jednu dimenzi
- **Dice** – výběr podkubu přes více dimenzí (prodeje pro rok 2023 a region CZ a kategorii elektronika)
- **Pivot (Rotate)** – rotace pohledu; prohodí dimenze na osách (řádky ↔ sloupce)

Typy OLAP

- **MOLAP** – multidimenzionální; data v speciálních kubech; rychlé, ale omezená škálovatelnost

- **ROLAP** – relační; nad standardním DWH; škálovatelné, flexibilní
- **HOLAP** – hybridní; kombinuje MOLAP a ROLAP

6.3.5 KPI, Dimenze, Granularita

- **KPI (Key Performance Indicators)** – klíčové ukazatele výkonnosti; měřitelné hodnoty hodnotící výkonnost organizace vůči cílům
 - Příklady: konverzní poměr, průměrná hodnota objednávky, NPS, churn rate
 - Musí být SMART: Specific, Measurable, Achievable, Relevant, Time-bound
- **Dimenze** – perspektiva, z níž se data analyzují (čas, zákazník, produkt, lokalita)
- **Granularita** – úroveň detailu dat; jemnější granularita = více prostoru, více flexibility
- **Agregace** – shrnutí dat (SUM, AVG, COUNT, MIN, MAX) na vyšší úroveň granularity
- **Metrika/Míra** – numerická hodnota podléhající agregaci (obrat, počet kusů)

6.3.6 Business Intelligence (BI)

BI je soubor technologií, procesů a nástrojů pro transformaci surových dat na srozumitelné, použitelné informace pro obchodní rozhodování.

Složky BI

- Datový sklad + ETL/ELT pipeline
- OLAP engine
- Reporting a dashboardy
- Data mining a prediktivní analytika
- Self-service BI (Power BI, Tableau, Qlik)

Praktický příklad: analýza zákazníků z MSSQL pomocí BI Máš transakční databázi v MS SQL Serveru s tabulkami zákazníků a objednávek. Cíl: pochopit chování zákazníků, odhalit trendy, podpořit rozhodování. Celý tok od zdrojové DB po dashboard vypadá takto:

Krok 1 – Zdrojová data (OLTP databáze v MSSQL)

Transakční DB je navržena pro rychlé zápisy, ne pro analytiku. Dotaz jako “měsíční tržby podle segmentu zákazníka za posledních 5 let” by blokoval produkční server.

Listing 23: Zdrojové tabulky v MSSQL (OLTP)

```
-- Tabulky existující v produkčním MSSQL
SELECT TOP 5 * FROM Customers;    -- id, name, email, segment,
    region, created_at
SELECT TOP 5 * FROM Orders;        -- id, customer_id, order_date,
    total_amount, status
SELECT TOP 5 * FROM OrderItems;    -- id, order_id, product_id, qty,
    unit_price
```

Krok 2 – ETL/ELT: přesun dat do datového skladu

Produkční data se pravidelně (nočně nebo v reálném čase) přesouvají do DWH a transformují do hvězdicového schématu:

Listing 24: Dimenzionální model v DWH (hvězdicové schéma)

```
-- DWH tabulky (Power BI / Azure Synapse / SQL Server DW)
DIM_ZAKAZNIK (id_zak, jmeno, segment, region, datum_registrace)
DIM_CAS      (id_cas, datum, mesic, kvartal, rok, den_tydne)
DIM_PRODUKT  (id_prod, nazev, kategorie, znacka)
FAKT_PRODEJ  (id_zak_fk, id_cas_fk, id_prod_fk, trzba, mnozstvi,
    sleva)
```

Krok 3 – Připojení Power BI k datům

Power BI se připojí přímo k MSSQL nebo k DWH přes konektor. V Power BI Desktop: *Získat data* → *SQL Server* → zadáš server a DB. Power BI automaticky detekuje relace mezi tabulkami (FK).

Krok 4 – Výpočty v Power BI pomocí DAX

DAX (Data Analysis Expressions) je jazyk Power BI pro metriky:

Listing 25: DAX výpočty v Power BI

```
-- Celková tržba
Celkova trzba = SUM(FAKT_PRODEJ[trzba])

-- Tržba za předchozí rok (pro porovnání)
Trzba loni = CALCULATE([Celkova trzba], SAMEPERIODLASTYEAR(DIM_CAS[
    datum]))

-- Meziroční růst v %
Rust YoY % = DIVIDE([Celkova trzba] - [Trzba loni], [Trzba loni]) *
    100

-- Počet aktivních zákazníků (alespoň 1 nákup za posledních 90 dní)
Aktivni zakaznici =
    CALCULATE(
        DISTINCTCOUNT(FAKT_PRODEJ[id_zak_fk]),
        DATESINPERIOD(DIM_CAS[datum], TODAY(), -90, DAY)
    )
```

Krok 5 – Vizualizace a dashboard

V Power BI sestavíš dashboard z vizuálů:

Vizuál	Co zobrazuje	Businessová otázka
Sloupcový graf	Tržby podle měsíce + loni	Rosteme nebo klesáme YoY?
Mapa	Tržby podle regionu	Kde jsou naši nejceněnější zákazníci?
Koláčový/pruhový	Podíl segmentů (Premium/Standard/Budget)	Který segment nás živí?
KPI karta	Celková tržba, aktivní zákazníci, průměrná objednávka	Rychlý přehled klíčových čísel
Tabulka	Top 10 zákazníků (RFM analýza)	Na koho se zaměřit?
Průběhový graf	Churn rate (% zákazníků bez nákupu 6 měsíců)	Přicházíme o zákazníky?

Krok 6 – Filtry (Slicery) a drill-through

Uživatel si může dashboard sám filtrovat:

- **Slicer** – filtr na období (rok/čtvrtletí), region, segment zákazníka
- **Drill-through** – klik na region na mapě → přejde na detailní stránku tohoto regionu
- **Cross-filtering** – klik na sloupec v grafu automaticky filtruje všechny ostatní vizuály

Publikace a sdílení – hotový report se publikuje do Power BI Service (cloud); manažeři ho vidí v prohlížeči nebo mobilní aplikaci; reporty se mohou automaticky aktualizovat.

Celkový tok od dat k rozhodnutí:

MSSQL (zákazníci, objednávky) $\xrightarrow{\text{ETL/ELT}}$ Datový sklad (hvězda) $\xrightarrow{\text{Power BI konektor}}$ DAX
 metriky $\xrightarrow{\text{vizualizace}}$ Dashboard $\xrightarrow{\text{publish}}$ Manažerské rozhodování

6.3.7 Balanced Scorecard (BSC)

BSC (Kaplan & Norton, 1992) je strategický nástroj pro měření a řízení výkonnosti organizace z více perspektiv – překlenuje propast mezi strategií a operativou.

4 perspektivy BSC

- **Finanční perspektiva** – jak nás vnímají akcionáři? KPI: ROI, ROE, zisk, náklady
- **Zákaznická perspektiva** – jak nás vnímají zákazníci? KPI: spokojenost, NPS, retence, podíl na trhu
- **Perspektiva interních procesů** – v čem musíme excelovat? KPI: čas cyklu, kvalita, inovace

- **Perspektiva učení a růstu** – jak rozvíjíme schopnosti? KPI: školení, fluktuace, spokojenost zaměstnanců

Každá perspektiva obsahuje: strategický záměr → cíle → metriky → iniciativy. BSC propojuje KPI s firemní strategií – nestačí sledovat jen finanční ukazatele.

Otázky profesorů k tématu: Datová analytika

Pour Jan, doc. Ing., CSc.

- Co je datový sklad a jaké jsou jeho charakteristiky?
- Jaký je rozdíl mezi OLTP a OLAP?
- Co je ETL a jaké jsou jeho fáze?

Maryška Miloš, doc. Ing., Ph.D.

- Jaké jsou dimenze kvality dat?
- Co je KPI a jak se definuje?
- Vysvětlete pojem granularita v kontextu datového skladu.

Sudzina František, Mgr. et Mgr. Ing., Ph.D.

- Jaká jsou V velkých dat (Big Data)?
- Co je Business Intelligence a jaké nástroje znáte?

Chudán David, Ing., Ph.D.

- Jaké jsou OLAP operace? Vysvětlete drill-down a roll-up.
- Co je hvězdicové a sněhové schéma?

7 Zpracování informací

7.1 Strojové učení a dolování z dat (Data Mining)

7.1.1 Základní pojmy

Umělá inteligence (AI) AI je obor informatiky zabývající se tvorbou systémů schopných vykonávat úkoly, které by normálně vyžadovaly lidskou inteligenci – jako je vnímání, uvažování, učení, rozhodování nebo porozumění přirozenému jazyku.

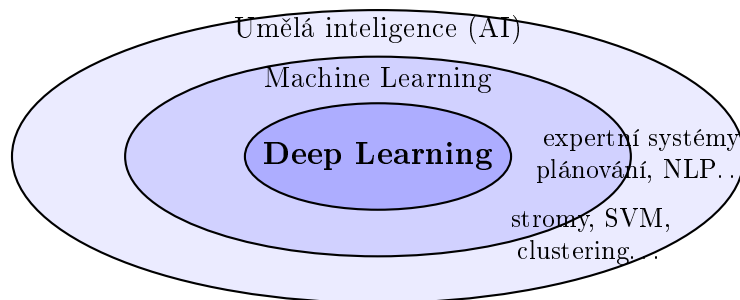
Starší definice “simulace lidského myšlení” je nepřesná – moderní AI systémy dosahují inteligentního chování i metodami zcela odlišnými od lidské kognice (např. matice vah v neuronové síti se nijak nepodobá lidskému mozku). Přesnější je říct:

“AI = systém, který vnímá prostředí, zpracovává informace a jedná tak, aby dosáhl cíle.”

Rozdělení AI podle obecnosti

- **ANI (Narrow AI)** – *slabá AI*; zvládá jeden konkrétní úkol lépe než člověk; příklady: AlphaGo (Go), GPT (text), DALL-E (obrázky), spam filtr; **vše co dnes reálně existuje je ANI**
- **AGI (Artificial General Intelligence)** – *silná AI*; hypotetický systém schopný řešit libovolný intelektuální úkol jako člověk; dosud neexistuje
- **ASI (Artificial Super Intelligence)** – hypotetická AI překonávající člověka ve všech oblastech; pouze spekulativní

Hierarchie pojmů: $AI \supset ML \supset \text{Deep Learning}$



- **Strojové učení (ML)** – podoblast AI; algoritmy, které se *učí ze zkušenosti* (dat) bez explicitního programování pravidel; model sám odhalí vzory v datech
- **Hluboké učení (Deep Learning)** – podoblast ML; vícevrstvé neuronové sítě; automaticky extrahuje příznaky (features) z raw dat (obrázků, textu, zvuku); příklady: CNN (počítačové vidění), Transformer (GPT, BERT)
- **Dolování dat (Data Mining)** – průnik ML, statistiky a databázových technologií; zaměřen na odkrývání vzorů, korelací a trendů v (zejména databázových) datech; výsledky jsou interpretovatelné pro business

Typy strojového učení

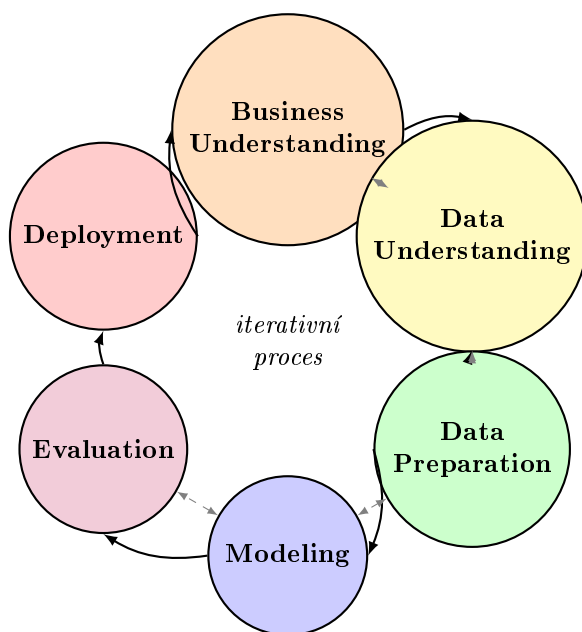
- **Supervised learning (učení s učitelem)** – trénovací data obsahují pár vstup-výstup (labeled); model se naučí mapování; *příklady*: klasifikace spamu, predikce ceny bytu, rozpoznávání obrazu
- **Unsupervised learning (bez učitele)** – data bez labelů; model hledá strukturu sám; *příklady*: segmentace zákazníků (clustering), detekce anomálií, doporučovací systémy
- **Reinforcement learning (posilované učení)** – agent se učí interakcí s prostředím; dostává odměny za správné akce a tresty za špatné; *příklady*: AlphaGo, autonomní řízení, optimalizace reklam

7.1.2 CRISP-DM

CRISP-DM (Cross-Industry Standard Process for Data Mining, 1999) je standardní metodika popisující jak správně vést ML/data mining projekt od zadání po nasazení.

Proč ji potřebujeme? Bez struktury většina ML projektů ztroskotá na tom, že tým rovnou skočí na modelování, zatímco 80 % práce tvoří příprava dat a pochopení problému. CRISP-DM toto řeší – říká “nejdřív pochop problém, pak teprve modeluj”.

Klíčová vlastnost: metodika není lineární – fáze se opakují a navzájem ovlivňují. Typicky se mezi nimi iteruje vícekrát (např. po Evaluation zjistíme, že chybí data → zpět na Data Understanding).



Obrázek 12: CRISP-DM – šest iterativních fází

Praktický příklad: e-shop chce predikovat, který zákazník v příštím měsíci odejde (churn).

1. Business Understanding – co chceme dosáhnout?

- Businessový cíl: snížit churn rate o 20 % cílenými retencními nabídkami

- Převod na ML problém: binární klasifikace – zákazník odejde (1) nebo ne (0)
- Kritérium úspěchu: $\text{precision} \geq 70\%$ (nechceme zbytečně posílat slevy neodcházejícím)

2. Data Understanding – jaká data máme?

- Zdrojová data: transakční DB (MS SQL), CRM systém, log webového chování
- EDA (Exploratory Data Analysis): distribuce proměnných, korelace, missing values
- Zjištění: 15 % zákazníků nemá vyplněný věk; churn je nevyvážený (5 % pozitivních)

3. Data Preparation – příprava pro model (80 % práce)

- **Výběr příznaků (features):** počet nákupů za 3 měsíce, průměrná útrata, dny od posledního nákupu, počet stížností, kategorie produktů
- **Čištění:** imputace chybějícího věku (mediánem), odstranění duplicit
- **Transformace:** normalizace číselných hodnot, one-hot encoding kategorií
- **Řešení nebalance:** SMOTE (syntetické generování minority class)

4. Modeling – trénování modelu

- Vyzkoušeny: logistická regrese (baseline), Random Forest, XGBoost
- Rozdělení dat: 70 % trénink / 15 % validace / 15 % test (nebo 10-fold CV)
- Hyperparameter tuning: Grid Search nebo Bayesian Optimization
- Nejlepší výsledek: XGBoost, $\text{AUC} = 0.87$

5. Evaluation – odpovídá model businessovým cílům?

- Technická evaluace: $\text{Precision} = 0.74$, $\text{Recall} = 0.61$, $\text{F1} = 0.67$, $\text{AUC} = 0.87$
- Businessová evaluace: identifikuje 61 % skutečných churnerů; splňuje cíl $\text{precision} \geq 70\%$
- Rozhodnutí: model nasadit do pilotu pro segment Premium zákazníků
- *Zpětná smyčka:* recall je nízký – vrátíme se na Data Preparation a přidáme features z webového logu (navštívené stránky, čas na webu)

6. Deployment – model jde do produkce

- Model zabalíme jako REST API (Python/Flask nebo FastAPI)
- Každý den se spustí pipeline: nová data $\rightarrow$ model $\rightarrow$ seznam zákazníků k oslovení
- CRM systém automaticky vytvoří kampaň s personalizovanou nabídkou
- Monitoring: sleduje se data drift (mění se vstupní data?) a model drift (klesá přesnost?)

7.1.3 Rozhodovací stromy (Decision Trees)

Hierarchická stromová struktura pro klasifikaci nebo regresi.

Struktura

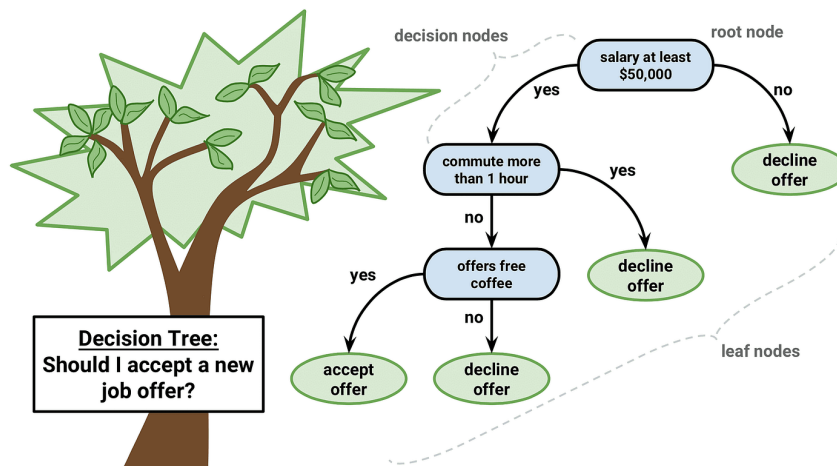
- **Kořenový uzel** – vstupní bod; první atribut pro větvení
- **Vnitřní uzly** – testy podmínek na attributech
- **Větvě** – možné výsledky testu
- **Listy** – výsledná třída nebo hodnota

Algoritmy

- **ID3, C4.5** – kritérium informační gain (entropie)
- **CART** – Gini impurity; produkuje binární stromy
- **Prořezávání (Pruning)** – redukce přeučení (overfitting) odstraněním nerelevantních větví

Výhody a nevýhody

- **Výhody:** interpretovatelné, nevyžadují normalizaci, zvládají numerická i kategorická data
- **Nevýhody:** náchylné na přeučení, nestabilní (malá změna dat = jiný strom)



Obrázek 13: Ukázka rozhodovacího stromu

Random Forest Ansamblová metoda; mnoho rozhodovacích stromů trénovaných na náhodných podmnožinách dat a atributů. Výsledek: hlasování (klasifikace) nebo průměr (regrese). Redukuje overfitting.

- **Klasifikace** – každý strom hlasuje pro třídu (např. spam/ne-spam); výsledkem je třída s nejvíce hlasy.

- **Regrese** – každý strom vrátí číselnou hodnotu (např. cenu nemovitosti); výsledkem je průměr všech předpovědí.

Proč redukuje overfitting: jeden strom se může přetrénovat na konkrétní data; průměrování/hlasování přes mnoho různorodých stromů tyto chyby vyruší.

7.1.4 Clustering (Shlukování)

Nesupervizovaná metoda; seskupení podobných objektů do skupin (clusterů).

K-Means

1. Inicializuj k centroidů (náhodně nebo K-Means++)
2. Přiřaď každý bod nejbližšímu centroidu (Euklidovská vzdálenost)
3. Přepočítej centroidy jako průměr přiřazených bodů
4. Opakuj 2–3 dokud se centroidy nepřestanou měnit

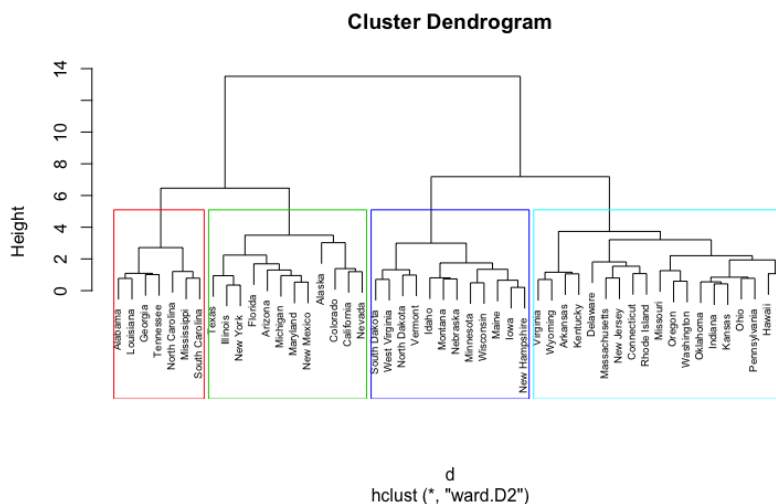
Problém: nutno předem zvolit k ; citlivý na odlehlé hodnoty; předpokládá kulové clustery.



Obrázek 14: Ukázka K-Means clusteringu

Hierarchické shlukování

- **Agglomerativní** – bottom-up; každý bod začíná jako vlastní cluster, postupně se spojují
- **Divisivní** – top-down; začíná jedním clusterem, postupně se dělí
- Výstup: dendrogram (stromová struktura hierarchie)



Obrázek 15: Dendrogram hierarchického shlukování

DBSCAN Density-Based Spatial Clustering; clusterly jako oblasti vysoké hustoty; zvládá libovolné tvary; identifikuje outliers.

7.1.5 Asociační pravidla

Asociační pravidla hledají vztahy tvaru “**jestliže zákazník koupí A, koupí také B**” v transakčních datech. Klasická aplikace: **analýza tržního košíku** (market basket analysis).

Příklad transakční databáze (5 nákupů):

Transakce	Zakoupené položky
T1	Pivo, Pleny, Chips
T2	Pivo, Pleny
T3	Pivo, Chips
T4	Pleny, Chips, Džus
T5	Pivo, Pleny, Chips, Džus

Klíčové metriky – intuitivně **1. Podpora (Support)** – *Jak běžná je tato kombinace celkově?*

$$\text{supp}(A \Rightarrow B) = \frac{\text{počet transakcí obsahujících A i B}}{\text{celkový počet transakcí}}$$

Příklad: Pivo a Pleny se vyskytují společně v T1, T2, T5 – tedy ve 3 z 5 transakcí:

$$\text{supp}(\text{Pivo} \Rightarrow \text{Pleny}) = \frac{3}{5} = 60\%$$

Nízká podpora = pravidlo platí jen zřídka (nezajímavé pro business).

2. Důvěra (Confidence) – Jak spolehlivé je pravidlo? Když zákazník koupí A , jak často koupí i B ?

$$\text{conf}(A \Rightarrow B) = \frac{\text{supp}(A \cup B)}{\text{supp}(A)} = P(B \mid A)$$

Příklad: Pivo se vyskytuje v T1, T2, T3, T5 (4 transakce). Z nich Pleny jsou v T1, T2, T5 (3 transakce):

$$\text{conf}(\text{Pivo} \Rightarrow \text{Pleny}) = \frac{3}{4} = 75\%$$

Když zákazník koupí Pivo, v 75 % případů koupí také Pleny. Vysoká důvěra **sama nestačí** – pokud zákazníci kupují Pleny vždy (100 % podpora), pak 75 % důvěra je vlastně *horší* než náhodný výběr.

3. Lift – Je tato asociace opravdu zajímavá, nebo je to jen náhoda?

$$\text{lift}(A \Rightarrow B) = \frac{\text{conf}(A \Rightarrow B)}{\text{supp}(B)}$$

Lift říká: o kolik více zákazníků koupí B díky tomu, že koupili A , oproti náhodnému výběru?

- **Lift** > 1 – pozitivní asociace; A skutečně pomáhá predikovat B ; pravidlo je užitečné
- **Lift** = 1 – A a B jsou nezávislé; znalost A nepomáhá predikovat B
- **Lift** < 1 – negativní asociace; zákazníci kupující A méně pravděpodobně kupují B

Příklad: Pleny jsou v T1, T2, T4, T5 – $\text{supp}(\text{Pleny}) = 4/5 = 80\%$:

$$\text{lift}(\text{Pivo} \Rightarrow \text{Pleny}) = \frac{75\%}{80\%} = 0,94 < 1$$

Lift < 1 – zákazníci kupující Pivo kupují Pleny *méně* než průměr. Tato asociace není užitečná!

Pravidlo	Support	Confidence	Lift	Závěr
Pivo $\Rightarrow$ Chips	60 %	75 %	1.25	Užitečné (lift > 1)
Pivo $\Rightarrow$ Pleny	60 %	75 %	0.94	Nezajímavé (lift < 1)
Pleny $\Rightarrow$ Chips	60 %	75 %	1.25	Užitečné

Algoritmus Apriori – k čemu slouží a jak funguje **Problém:** pro n různých položek existuje 2^n možných kombinací – pro supermarket s 10 000 produkty je to astronomické číslo. Nelze vyzkoušet všechny.

Apriori tento problém řeší chytrou zkratkou pomocí **anti-monotonní vlastnosti**:

Anti-monotonní vlastnost (klíčová myšlenka Apriori)

Pokud je kombinace $\{A, B\}$ **nefrekventovaná** (podpora < minSupport), pak **každá nadmnožina** $\{A, B, C\}$, $\{A, B, D\}$, ... bude také nefrekventovaná.
 $\Rightarrow$ Celé větve prostoru kombinací lze **odříznout bez prohledávání**.

Postup algoritmu:

1. Najdi všechny frekventované položky ($\text{supp} \geq \text{minSupport}$): {Pivo}, {Pleny}, {Chips}...
2. Kombinuj dvojice a ověř jejich support: {Pivo, Pleny}, {Pivo, Chips}...; **odřízní** nefrekventované
3. Kombinuj trojice z frekventovaných dvojic...; opakuj dokud nejsou žádné nové kombinace
4. Z frekventovaných množin vygeneruj pravidla a vypočítej Confidence a Lift

K čemu je Apriori v praxi dobrý:

- **Merchandising / rozmístění zboží** – zjistíš, že Pleny a Pivo se kupují dohromady → dej je do stejné uličky (nebo naopak do jiných, aby zákazník prošel celým obchodem)
- **Doporučovací systémy** – “zákazníci, kteří koupili X, také koupili Y” (Amazon, Netflix, Spotify to v základu takto dělají)
- **Křížový prodej (cross-selling)** – banka zjistí, že klienti s hypotékou si brzy sjednají pojištění → proaktivní nabídka
- **Detekce podvodů** – neobvyklé kombinace transakcí mohou signalizovat podvod
- **Analýza webového chování** – které stránky se navštěvují spolu? → optimalizace navigace

7.1.6 Hodnocení klasifikačních modelů

Matice záměn (Confusion Matrix)

	Predikováno: ANO	Predikováno: NE
Skutečnost: ANO	TP (True Positive)	FN (False Negative)
Skutečnost: NE	FP (False Positive)	TN (True Negative)

- **Accuracy** – $\frac{TP+TN}{TP+TN+FP+FN}$ – celková přesnost; kolik procent všech případů model klasifikoval správně. *Pozor: zavádějící u nevyvážených dat – model který vždy řekne „není spam“ dosáhne 99% accuracy, pokud je 99 % e-mailů legitimních.*
- **Precision** – $\frac{TP}{TP+FP}$ – z všech které model označil jako pozitivní, kolik jich skutečně pozitivní bylo.
Příklad: spamový filtr označil 100 e-mailů jako spam; z toho 90 bylo skutečně spam a 10 byly legitimní e-maily (FP). Precision = $90/100 = 90\%$.
Kdy je klíčová: když je drahé mít falešný poplach – nechceš posílat legitimní e-maily do spamu.
- **Recall (Sensitivity)** – $\frac{TP}{TP+FN}$ – z všech skutečně pozitivních případů, kolik jich model našel.
Příklad: v e-mailech bylo 200 spamů; filtr zachytil 180 a 20 prošlo (FN). Recall = $180/200 = 90\%$.
Kdy je klíčová: když je drahé zmeškat pozitivní případ – diagnóza rakoviny (nechceš zmeškat ani jeden případ).

- **Precision vs Recall – trade-off:** zvýšením prahu (model říká „pozitivní“ jen když je velmi jistý) roste Precision ale klesá Recall a naopak.
- **F1 score** – $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ – harmonický průměr Precision a Recall; vyvážená metrika když ti záleží na obou; hodnota 0–1, čím vyšší tím lepší.
- **Specificity** – $\frac{TN}{TN+FP}$ – z reálných negativ, kolik bylo správně klasifikováno jako negativní.

ROC křivka a AUC **ROC křivka** (Receiver Operating Characteristic) zobrazuje, jak se mění Recall (TPR) vs falešně pozitivní míra ($FPR = \frac{FP}{FP+TN}$) při postupném snižování rozhodovacího prahu modelu. Každý bod na křivce odpovídá jednomu nastavení prahu.

AUC (Area Under Curve) – plocha pod ROC křivkou; čím větší plocha, tím lepší model:

- **AUC = 1.0** – perfektní model; rozlišuje všechny pozitivní od negativních
- **AUC = 0.5** – náhodný model (přímka); model neví nic
- **AUC = 0.8–0.9** – dobrý model v praxi

Výhoda AUC: nezávisí na konkrétním prahu – hodnotí celkovou schopnost modelu rozlišovat třídy, ne jen jedno nastavení.

Metody ověřování modelů

- **Hold-out** – rozdělení na trénovací (70–80%) a testovací (20–30%) množinu
- **K-fold cross-validation** – data rozdělena na k stejných částí; model natrénován na $k - 1$ částech, testován na zbývajících; opakováno k krát; výsledek = průměr; typicky $k = 10$
- **Leave-one-out (LOO)** – speciální případ k-fold kde $k = n$; nákladné, ale nejmenší bias
- **Bootstrap** – opakované vzorkování s vracením; odhad variance modelu

Overfitting a underfitting

- **Overfitting (přeučení)** – model se příliš přizpůsobil trénovacím datům; nízká chyba na trénování, vysoká na testování; řešení: regularizace, pruning, více dat, cross-validation
- **Underfitting (podučení)** – model je příliš jednoduchý; vysoká chyba i na trénovacích datech; řešení: složitější model, více příznaků

7.2 Ukládání a vyhledávání textových informací

7.2.1 Information Retrieval (IR)

Disciplína zabývající se vyhledáváním nestrukturovaných informací (textu) v rozsáhlých kolekcích. Klíčový pojem: **relevance** – jak dobře dokument odpovídá dotazu.

Hodnocení IR systémů

- **Precision** – z vrácených dokumentů, kolik je relevantních: $P = \frac{TP}{TP+FP}$
- **Recall** – z relevantních dokumentů, kolik bylo vráceno: $R = \frac{TP}{TP+FN}$
- **F-measure**: $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$ – harmonický průměr precision a recall

7.2.2 Booleovský model vyhledávání

Nejstarší IR model. Dokumenty a dotazy reprezentovány jako množiny termů.

- Dotaz je booleovský výraz: `informatika AND databáze AND NOT NoSQL`
- Dokument je relevantní, pokud splňuje booleovský výraz (1/0 – binární relevance)
- **Výhody**: jednoduchost, přesnost formulace dotazu
- **Nevýhody**: žádné pořadí výsledků, výsledek je příliš citlivý na formulaci, žádná dílčí shoda

Rozšíření Booleovského modelu

- **Fuzzy model** – stupně členství termů v dokumentu (0–1); termy s váhami
- **Geometrický model** – dokumenty a dotazy jako vektory; relevance = vzdálenost

7.2.3 Vektorový model (Vector Space Model)

Každý dokument i každý dotaz se převede na **číselný vektor** – jedno číslo (váha) pro každé slovo v celém slovníku. Relevance dokumentu pro dotaz = **podobnost jejich vektorů**.

Pracovní příklad – 3 dokumenty a 1 dotaz:

ID	Text
D1	“databáze databáze SQL”
D2	“databáze NoSQL”
D3	“Python strojové učení”
Q	dotaz: “databáze SQL”

TF-IDF váhování Váha termu t v dokumentu d říká: *jak moc je toto slovo charakteristické pro tento dokument?*

$$w(t, d) = \underbrace{\text{TF}(t, d)}_{\text{četnost v doc.}} \cdot \underbrace{\text{IDF}(t)}_{\text{vzácnost ve sbírce}}$$

- **TF (Term Frequency)** – kolikrát se slovo vyskytuje v dokumentu; “databáze” v D1 = 2
- **IDF (Inverse Document Frequency)** = $\log \frac{N}{df(t)}$, kde N = počet dokumentů, $df(t)$ = v kolika dokumentech se term vyskytuje.

- Čím více dokumentů slovo obsahuje, tím větší je $df(t)$, tím menší je podíl $N/df(t)$ a tím nižší je IDF – slovo není příliš rozlišovací.
- Běžná slova jako “a”, “the” se vyskytují skoro ve všech dokumentech $\Rightarrow df(t) \approx N \Rightarrow \log(N/N) = \log(1) = 0 \Rightarrow$ váha blízká nule (filtrují se automaticky, bez stop-listu).
- Vzácné odborné slovo je třeba jen v jednom dokumentu $\Rightarrow df(t) = 1 \Rightarrow \log(N)$ – maximální IDF, slovo je velmi charakteristické.
- Logaritmus tlumí extrémní rozdíly: sbírka s milionem dokumentů by bez něj dávala obrovská čísla; log je škáluje do rozumného rozsahu.

Výpočet IDF pro náš příklad ($N = 3$):

Term	$df(t)$	IDF = $\log(3/df)$	Interpretace
databáze	2	$\log(1,5) = 0,41$	Střední vzácnost (ve 2/3 dokumentů)
SQL	1	$\log(3,0) = 1,10$	Vzácný (jen v D1)
NoSQL	1	$\log(3,0) = 1,10$	Vzácný (jen v D2)
Python	1	$\log(3,0) = 1,10$	Vzácný (jen v D3)

TF-IDF váhové vektory (slovník = {databáze, SQL, NoSQL, Python, strojové, učení}):

	databáze	SQL	NoSQL	Python	strojové	učení
D1	$2 \times 0,41 = 0,82$	$1 \times 1,10 = 1,10$	0	0	0	0
D2	$1 \times 0,41 = 0,41$	0	$1 \times 1,10 = 1,10$	0	0	0
D3	0	0	0	1,10	1,10	1,10
Q	0,41	1,10	0	0	0	0

Cosine similarity – výpočet relevance

$$\cos(\vec{d}, \vec{q}) = \frac{\vec{d} \cdot \vec{q}}{|\vec{d}| \cdot |\vec{q}|}$$

Hodnota 1 = vektory míří stejným směrem (maximální shoda); 0 = kolmé (žádná shoda).

Výpočet pro dotaz $Q = \text{“databáze SQL”}$:

$$\cos(\vec{D1}, \vec{Q}) = \frac{(0,82 \times 0,41) + (1,10 \times 1,10)}{|\vec{D1}| \cdot |\vec{Q}|} = \frac{0,34 + 1,21}{\sqrt{0,82^2 + 1,10^2} \cdot \sqrt{0,41^2 + 1,10^2}} = \frac{1,55}{1,37 \times 1,17} \approx \mathbf{0,97}$$

$$\cos(\vec{D2}, \vec{Q}) = \frac{(0,41 \times 0,41) + 0}{\dots} \approx \mathbf{0,30}$$

$$\cos(\vec{D3}, \vec{Q}) = \frac{0 + 0 + \dots}{\dots} = \mathbf{0,00}$$

Výsledek: D1 (0,97) $\gg$ D2 (0,30) $\gg$ D3 (0,00) – vyhledávač vrátí dokumenty v tomto pořadí.

Invertovaný index Bez invertovaného indexu bychom museli při každém dotazu projít všechny dokumenty – pro miliardy webových stránek nepřijatelné. Invertovaný index to obrátí: pro každé slovo předpočítáme seznam dokumentů, kde se vyskytuje.

Struktura:

Term	Posting list (dokumenty + pozice)
databáze	D1 (pozice: 1, 2), D2 (pozice: 1)
SQL	D1 (pozice: 3)
NoSQL	D2 (pozice: 2)
Python	D3 (pozice: 1)
strojové	D3 (pozice: 2)
učení	D3 (pozice: 3)

Jak se zpracuje dotaz “databáze SQL”:

1. Najdi v indexu posting list pro “databáze”: $\{D1, D2\}$
2. Najdi posting list pro “SQL”: $\{D1\}$
3. Průnik nebo skórování: D1 obsahuje obě slova $\rightarrow$ nejvyšší skóre
4. Výsledek seřaď podle TF-IDF / cosine similarity

Výhoda: dotaz se zodpoví v čase $O(\text{velikost posting listu})$, ne $O(\text{počet dokumentů})$. To je důvod, proč Google odpoví na dotaz za milisekundy i přes 50 miliard indexovaných stránek. Elasticsearch a Apache Lucene jsou postaveny přesně na tomto principu.

7.2.4 PageRank

Algoritmus pro hodnocení webových stránek (Google, Brin & Page, 1998). **Základní myšlenka:** stránka je důležitá, pokud na ni odkazuje mnoho důležitých stránek.

$$PR(A) = \frac{1-d}{N} + d \cdot \sum_{i \in B_A} \frac{PR(B_i)}{L(B_i)}$$

kde d je dampening factor (0.85), B_A jsou stránky odkazující na A, $L(B_i)$ počet odchozích odkazů z B_i .

- Počítá se iterativně, dokud se hodnoty nestabilizují
- **Dampening factor** – modeluje pravděpodobnost, že uživatel pokračuje v procházení
- Moderní vyhledávače kombinují PageRank s mnoha dalšími signály (obsah, rychlost, mobilní optimalizace)

7.3 Regulární výrazy

7.3.1 Základy regulárních výrazů

Regulární výraz (regex) je vzor popisující množinu řetězců. Používají se pro vyhledávání, validaci a nahrazování textu.

Základní symboly

Symbol	Význam
.	Libovolný znak (kromě newline)
*	0 nebo více výskytů
+	1 nebo více výskytů
?	0 nebo 1 výskyt
^	Začátek řádku
\$	Konec řádku
[abc]	Znak ze skupiny
[a-z]	Znak z rozsahu
[^abc]	Libovolný znak mimo skupinu
()	Skupina (capturing group)
	Alternativa (nebo)
\d	Číslice [0-9]
\w	Alfanumerický znak [a-zA-Z0-9_]
\s	Bílý znak (mezera, tab, newline)
{n,m}	n až m výskytů

Příklady

- E-mail: `^[\\w.+%\\-]+@[\\w\\-]+\\. [\\w\\-\\.]+$`
- PSČ (CZ): `\\d{3}\\s?\\d{2}`
- IP adresa: `(\\d{1,3}\\.){3}\\d{1,3}`
- Datum: `\\d{4}-\\d{2}-\\d{2}`

Greedy vs. Lazy

- **Greedy (výchozí):** `.*` – zachytí co nejvíce znaků
- **Lazy:** `.*?` – zachytí co nejméně znaků

Otázky profesorů k tématu: Zpracování informací

Sklenák Vilém, prof. Ing., CSc.

- Co je vektorový model vyhledávání informací?
- Jak funguje PageRank?
- Jaký je rozdíl mezi Booleovským a vektorovým modelem IR?

Vojtěch Stanislav, Ing., Ph.D.

- Co je data mining? Jaké jsou jeho základní úlohy?
- Vysvětlete algoritmus K-Means clustering.
- Co jsou asociační pravidla a jak se měří jejich kvalita?

Svátek Vojtěch, prof. Ing., Ph.D.

- Co je CRISP-DM a jaké jsou jeho fáze?
- Vysvětlete overfitting a jak mu předcházet.

Chudán David, Ing., Ph.D.

- Jaký je rozdíl mezi klasifikací a regresí v ML?

- Co je TF-IDF a k čemu se používá?

8 IT Governance

8.1 Governance vs. Management

8.1.1 Základní rozdíl

Governance (Správa/Řízení)	Management (Řízení)
Zajišťuje hodnocení potřeb, podmínek a možností zainteresovaných stran	Plánuje, buduje, provozuje a monitoruje aktivity
Stanovuje směr přes priority a rozhodování	Odpovídá za realizaci strategie
Monitoruje výkonnost a shodu	Řídí každodenní provoz
Zodpovídá rada (Board, ředitelé)	Zodpovídá management (manažeři, vedoucí)
“Kdo a co”	“Jak a kdo konkrétně”
Hodnotí (Evaluate) – Řídí (Direct) – Monitoruje (Monitor)	Plánuje (Plan) – Buduje (Build) – Provozuje (Run) – Monitoruje (Monitor)

Governance stanovuje *co* a *proč* – určuje cíle, priority a pravidla hry; zodpovídá za ni rada či vedení organizace. **Management** řeší *jak* – plánuje, realizuje a provozuje konkrétní aktivity, aby těchto cílů dosáhl; zodpovídají za něj manažeři a vedoucí týmů. Governance je tedy nadřazená vrstva: vymezuje prostor, ve kterém management pracuje.

Enterprise Governance of IT (EGIT) = zajistí, že IT strategie je v souladu s podnikovou strategií a že IT přináší hodnotu.

8.1.2 Klíčové oblasti EGIT

- **Soulad se strategií (Strategic Alignment)** – IT plány jsou propojeny s business plány
- **Tvorba hodnoty (Value Delivery)** – IT investice přinášejí příslibené benefity
- **Řízení zdrojů (Resource Management)** – optimální využití IT zdrojů (lidé, technologie, data)
- **Řízení rizik (Risk Management)** – identifikace a zvládání IT rizik
- **Měření výkonnosti (Performance Measurement)** – sledování a monitorování IT výkonnosti (BSC, KPI)

8.2 COBIT (Control Objectives for Information Technology)

COBIT je komplexní rámec pro správu a řízení podnikové informatiky vydaný organizací ISACA. Aktuální verze je **COBIT 2019** (vydána listopad 2018); předchozí verze COBIT 5 (2012) je stále rozšířená a v literatuře hojně citovaná.

Pro koho je COBIT určen? Primárně pro **management a vedení organizace** – CIO, IT ředitele, interní auditory a compliance manažery. Zajímá je otázka „*Děláme správné věci a dosahujeme cílů?*” – tedy governance pohled na IT. Typicky ho používají větší organizace, banky, pojišťovny a firmy podléhající regulaci (SOX, Basel).

Vztah COBIT a ITIL: COBIT a ITIL se doplňují – nejsou konkurenti.

- **COBIT** říká *co* musí governance a management IT pokrývat a jak měřit, zda je to splněno; je to zastřešující rámec.
- **ITIL** říká *jak* konkrétně provozovat a dodávat IT služby; je to provozní návod pro IT tým.
- Prakticky: COBIT definuje cíl (např. “Zajistit dodávku kvalitních služeb”), ITIL poskytuje procesy, jak ho naplnit (Incident Management, Change Management...).
- COBIT sám na ITIL odkazuje jako na jeden z doporučených standardů pro oblast správy IT služeb.

Vlastnost	COBIT 5 (2012)	COBIT 2019
Verze	5	2019 (rok vydání, dále ročníky)
Počet cílů	37 procesů	40 governance/management objectives
Terminologie	“Procesy”	“Governance & Management Objectives”
Přizpůsobení	Jeden přístup pro všechny	Design Factors – rámec se přizpůsobí organizaci
Nové koncepty	–	Focus Areas (DevSecOps, cloud, kyberb.)
Performance model	PAM (6 úrovní)	Aktualizovaný, více v souladu s CMMI
Open source	Částečně	Více materiálů zdarma

8.2.1 Klíčové novinky COBIT 2019

Design Factors (Faktory návrhu) COBIT 2019 zavádí 11 **design factors** – organizace si rámec *přizpůsobí* podle svého kontextu místo aplikace jednoho univerzálního přístupu:

- Typ organizace (soukromá, veřejná, nezisková)
- Velikost organizace

- IT strategie (nákladová optimalizace vs. inovace)
- Model sourcing IT (in-house, outsourcing, cloud)
- Metody vývoje IT (Agile, Waterfall, DevOps)
- Požadavky na compliance (GDPR, DORA, ISO 27001)
- Rizikový profil organizace a další

Příklad: startupová fintech firma s 50 zaměstnanci a 100 % cloud infrastrukturou použije COBIT 2019 odlišně než velká banka s on-premise legacy systémy a regulatorními požadavky ČNB.

Focus Areas (Oblast zaměření) Tematické rozšíření COBIT 2019 pro specifické domény:

- **Small and Medium Enterprises** – zjednodušené pokyny pro menší organizace
- **Cybersecurity** – řízení kybernetické bezpečnosti v souladu s NIST, ISO 27001
- **Digital Transformation** – governance digitální transformace
- **DevSecOps** – integrace bezpečnosti do CI/CD pipeline
- **Cloud Governance** – řízení cloudových služeb

8.2.2 Pět principů COBIT 5

Principy COBIT 5 jsou stále relevantní – COBIT 2019 je rozšiřuje, ne nahrazuje.

Princip 1: Naplňování potřeb zainteresovaných stran Cílem je tvorba hodnoty pro zainteresované strany. COBIT pracuje s kaskádou cílů: Potřeby zainteresovaných stran → Podnikové cíle → IT-related cíle → Cíle facilitátorů.

Princip 2: Pokrytí organizace od začátku do konce (End-to-End) COBIT zahrnuje celé řízení podnikových IT aktiv:

- Zahrnuje všechny IT funkce a procesy v organizaci
- Pokrývá interní i externí IT providery
- Integruje governance a management

Princip 3: Aplikace jediného integrovaného rámce COBIT je zastřešující rámec, který lze kombinovat s jinými standardy:

- Soulad s ISO/IEC 38500 (IT Governance)
- Integrace s ITIL (správa IT služeb), ISO 27001 (bezpečnost), PMBOK (projekty)
- Jeden rámec namísto fragmentace

Princip 4: Uplatnění holistického přístupu **Facilitátor (enabler)** je cokoliv, co pomáhá organizaci dosáhnout svých cílů v oblasti governance a managementu IT – tedy zdroje, nástroje a mechanismy, které governance *umožňují* fungovat. COBIT definuje 7 takových facilitátorů:

1. Principy, politiky a rámce
2. Procesy
3. Organizační struktury
4. Kultura, etika a chování
5. Informace
6. Služby, infrastruktura a aplikace
7. Lidé, dovednosti a kompetence

Princip 5: Oddělení governance od managementu Viz tabulka výše – COBIT jasně odděluje oblast governance (EDM domény) od oblasti managementu (APO, BAI, DSS, MEA domény).

8.2.3 Domény procesů COBIT 5

Governance (EDM – Evaluate, Direct, Monitor)

- EDM01 – Zajišťování nastavení governance rámce
- EDM02 – Zajišťování tvorby hodnot
- EDM03 – Zajišťování optimalizace rizik
- EDM04 – Zajišťování optimalizace zdrojů
- EDM05 – Zajišťování transparentnosti k zainteresovaným stranám

Management (4 domény, 32 procesů)

- **APO (Align, Plan, Organise)** – 13 procesů; strategie, architektura, HR, bezpečnost
- **BAI (Build, Acquire, Implement)** – 10 procesů; projekty, změny, kapacity
- **DSS (Deliver, Service, Support)** – 6 procesů; provoz, správa incidentů a problémů
- **MEA (Monitor, Evaluate, Assess)** – 3 procesy; výkonnost, shoda, IT governance

8.2.4 Model procesní vyspělosti COBIT (PAM)

COBIT definuje 6 úrovní vyspělosti (zralosti) procesu:

0. **Incomplete** – proces neexistuje nebo nedosahuje svého účelu

1. **Performed** – proces existuje a dosahuje svého účelu (ad-hoc)
2. **Managed** – proces je plánovaný, monitorovaný a přizpůsobovaný
3. **Established** – proces je definovaný a standardizovaný v organizaci
4. **Predictable** – proces je statisticky měřen a předvídatelný
5. **Optimizing** – proces se průběžně zlepšuje; inovace a optimalizace

Hodnocení vychází z ISO/IEC 15504 (SPICE). Cílová úroveň závisí na strategickém významu procesu.

8.2.5 COBIT v praxi – příklad z bankovníctví

Scénář: Česká banka zavádí COBIT jako governance rámec. Jak se jednotlivé domény projevují v reálném provozu?

Doména	Kdo	Co dělá	konkrétně	Příklad v bance
EDM (Governance)	Představenstvo, CFO	Schvaluje strategii, sleduje výsledky, zajišťuje soulad s regulací	IT	Představenstvo schválí investici 50 mil. Kč do modernizace core banking; stanoví rizikový apetit pro cloud
APO (Align, Plan)	IT ředitel, architekti	Tvoří IT strategii, plánuje portfolio projektů, řídí bezpečnost a architekturu		IT strategie na 3 roky: migrace do cloudu, zavedení open banking (PSD2), cyber security program
BAI (Build, Acquire)	Projektoví manažeři, vývojáři	Realizuje projekty, řídí změny, testuje a nasazuje nová řešení		Projekt nové mobilní bankovní aplikace: PMP, change management, UAT testování, rollout
DSS (Deliver, Support)	IT operace, service desk	Provozuje systémy, řeší incidenty, zajišťuje dostupnost		Sobotní výpadek platebního systému: P1 incident, eskalace, 2-hodinová obnova, komunikace klientům
MEA (Monitor, Assess)	Interní audit, CISO	Měří shodu, hodnotí výkonnost, reportuje regulátorovi (ČNB)		Čtvrtletní audit: IT shoda s DORA regulací, měření SLA, audit kyberbezpečnosti pro ČNB

Maturity model v praxi: Banka si nechá udělat COBIT assessment. Zjistí, že:

- Incident management (DSS02) je na úrovni **3 – Established** (procesy definovány, ale ne měřeny)

- Change management (BAI06) je na úrovni **2 – Managed** (ad-hoc přístupy, bez jednotného procesu)
- Cíl: oba procesy na úroveň **4 – Predictable** do 18 měsíců (vyžaduje regulátor)

8.3 ITIL (IT Infrastructure Library)

ITIL je sada doporučených postupů pro správu IT služeb (IT Service Management – ITSM). Zaměřuje se na soulad IT služeb s potřebami businessu.

8.3.1 ITIL 3 (v3, 2007/2011)

Organizovaný kolem 5 knih odpovídajících životnímu cyklu IT služby:

1. **Service Strategy (Strategie služeb)** – definování trhu a zákazníků; finanční řízení IT
2. **Service Design (Návrh služeb)** – návrh nových nebo změněných služeb; SLA, kapacita, dostupnost
3. **Service Transition (Přechod služeb)** – bezpečný přechod do provozu; change management, CMDB
4. **Service Operation (Provoz služeb)** – každodenní provoz; incident management, problem management, service desk
5. **Continual Service Improvement (Kontinuální zlepšování)** – PDCA cyklus; metriky, CSF, KPI

8.3.2 ITIL v praxi – klíčové procesy s příklady

Incident Management vs. Problem Management – rozdíl na příkladu Toto je nejdůležitější rozlišení v ITIL:

	Incident Management	Problem Management
Cíl	Co nejrychleji obnovit službu	Najít a odstranit kořenovou příčinu
Otázka	“Jak to opravit teď?”	“Proč to padlo a jak tomu příště zabránit?”
Výstup	Obnovená služba	Trvalá oprava / workaround
Priorita	Rychlost	Důkladnost

Scénář: v pondělí ráno padl produkční web e-shopu.

Incident Management (okamžitá reakce):

1. **Detekce:** monitorovací nástroj (Zabbix/Datadog) detekuje výpadek a vytvoří ticket v JIRA/ServiceNow; závažnost P1
2. **Záznam:** service desk zaznamená: čas vzniku, dotčené systémy, dopad (5 000 uživatelů nemůže nakupovat)

3. **Klasifikace a eskalace:** P1 = eskalace na L2/L3 tým; přidání on-call inženýrů
4. **Diagnostika:** tým zjistí, že databázový server má plný disk (logs) → aplikace nemohou zapisovat
5. **Řešení:** smazání starých logů, restart aplikačního serveru → web funguje po 47 minutách
6. **Uzavření:** ticket uzavřen; uživatelé notifikováni; post-incident report do 24 hodin

⇒ Incident vyřešen rychle, ale **příčina stále existuje** – logy rostou dál.

Problem Management (týden po incidentu):

1. **Analýza:** tým dělá Root Cause Analysis (RCA); zjistí, že logy nejsou rotovány (logrotate selhal po upgradu)
2. **Known Error:** zaznamenají Known Error do databáze (KEDB): “logy rostou bez omezení po upgrade na v2.5”
3. **Trvalá oprava:** opraví konfiguraci logrotate na všech serverech; přidají monitoring volného místa s alertem při <15 %
4. **Prevence:** aktualizují checklist pro budoucí upgrady – ověřit logrotate konfiguraci

⇒ Kořenová příčina odstraněna; tento typ incidentu se nebude opakovat.

Change Management – jak projde změna systémem Každá změna v produkci musí projít procesem, aby se předešlo novým incidentům:

RFC $\xrightarrow{\text{posouzení}}$ CAB schválení $\xrightarrow{\text{nasazení}}$ PIR hodnocení

- **RFC (Request for Change)** – žádost o změnu; popis co, proč, rizika, rollback plán
- **CAB (Change Advisory Board)** – komise (IT manažeři, zástupci businessu, bezpečnost) schvaluje rizikové změny; standardní/nouzové změny mají zjednodušený proces
- **PIR (Post-Implementation Review)** – po nasazení: fungovalo to podle plánu? Co příště zlepšit?

Příklad: vývojář chce nasadit novou verzi platebního modulu v pátek večer. CAB zamítne – v pátek se změny nenasazují (výpadek přes víkend = největší dopad). Nasazení přeplánováno na úterý ráno s definovaným rollback postupem.

8.3.3 ITIL 4 (2019)

Modernizovaná verze reagující na DevOps, Agile, Cloud a digitální transformaci.

Klíčové koncepty ITIL 4

- **Service Value System (SVS)** – holistický pohled na to, jak všechny komponenty a aktivity organizace spolupracují na vytváření hodnoty

- **Service Value Chain (SVC)** – 6 aktivit: Plan, Improve, Engage, Design & Transition, Obtain/Build, Deliver & Support
- **4 dimenze správy služeb** – každou službu je nutné zvážit ze čtyř úhlů pohledu, aby nic nebylo opomenuto:
 1. **Organizace a lidé** – kdo službu zajišťuje; role, zodpovědnosti, kultura a kompetence týmu
 2. **Informace a technologie** – jaká data, znalosti a nástroje jsou ke službě potřeba
 3. **Partneři a dodavatelé** – kdo dodává zvenčí; smlouvy, závislosti na třetích stranách
 4. **Procesy a hodnotové toky** – jak práce protéká organizací od požadavku k výsledku; aktivity a jejich návaznosti
- **Guiding principles (7 průvodních principů):**
 1. Zaměřit se na hodnotu (Focus on value)
 2. Začínat od stávajícího stavu (Start where you are)
 3. Postupovat iterativně s využitím zpětné vazby
 4. Spolupracovat a podporovat viditelnost
 5. Přemýšlet a pracovat holisticky
 6. Zachovávat jednoduchost a praktičnost
 7. Optimalizovat a automatizovat

ITIL 4 – Management Practices ITIL 4 nepoužívá termín “procesy”, ale “practices” (34 celkem):

- **General management** – 14 praktik (risk management, strategie, SLM, bezpečnostní management)
- **Service management** – 17 praktik (incident management, change enablement, service desk)
- **Technical management** – 3 praktiky (deployment management, infrastructure management)

ITIL 4 Specializační moduly (2020–2021) Po vydání základního ITIL 4 Foundation vydala AXELOS série specializačních publikací:

- **Create, Deliver & Support (CDS)** – jak navrhovat, budovat a provozovat IT produkty a služby; zaměřen na DevOps, CI/CD, service desk a incident management
- **Drive Stakeholder Value (DSV)** – jak budovat hodnotné vztahy se zákazníky a uživateli; Customer Journey, SLA vyjednávání, onboarding

- **High-velocity IT (HVIT)** – jak dosáhnout rychlých iterací v digitálním prostředí; soulad s DevOps, Lean, Agile a Site Reliability Engineering (SRE)
- **Direct, Plan & Improve (DPI)** – jak řídit a zlepšovat IT organizaci na strategické úrovni; propojení s COBIT a dalšími governance rámci
- **Digital & IT Strategy (DITS)** – pro C-level; jak formulovat IT/digitální strategii; soulad businessu a IT; digitální transformace

Praktický dopad: ITIL 4 Foundation + jeden nebo více specializačních modulů tvoří certifikační cestu **ITIL 4 Managing Professional** nebo **ITIL 4 Strategic Leader**. Firmy jako O2, Vodafone nebo ČEZ vyžadují ITIL certifikaci pro IT service management role.

ITIL 4 v praxi – Service Value Chain na příkladu nové funkce *Zákazník (product manager) požaduje novou funkci: “zákazník dostane push notifikaci, když mu přijde platba.”*

SVC aktivita	Co se konkrétně děje
Plan	IT a business definují prioritu funkce v product backlogu; odhadnou ROI a technickou složitost
Engage	Product owner upřesní požadavky se zákazníky (user research); dohodne se s dodavatelem push notifikací (Firebase)
Design & Transition	Vývojáři navrhnu architekturu (event-driven: platba → Kafka → notification service); připraví deployment pipeline; security review
Obtain/Build	Sprint: vývojáři kódují, QA testuje, code review; integrace s Firebase; load testing
Deliver & Support	Feature nasazena (canary deploy – nejprve 5 % uživatelů); monitoring latence notifikací; service desk dostane FAQ pro případné dotazy zákazníků
Improve	Po 2 týdnech: analýza dat – 78 % uživatelů notifikace otevře; latence 3s je příliš vysoká; optimalizace fronty zpráv; plánuje se batched delivery

Proč je ITIL 4 lepší než ITIL 3 pro tento příklad? V ITIL 3 by funkce šla přes striktní životní cyklus (Strategy → Design → Transition → Operation) – trvalo by měsíce. ITIL 4 umožňuje iterativní přístup (Agile/DevOps): funkce jde do produkce za 2 týdny, zpětná vazba je okamžitá, zlepšení probíhá průběžně.

8.3.4 Srovnání ITIL 3 vs ITIL 4

Aspekt	ITIL v3	ITIL 4
Organizační jednotka	Procesy	Practices (praktiky)
Struktura	5 knih/fází životního cyklu	Service Value System (SVS)
Kontejner	ITIL v3 Service Lifecycle	Service Value Chain
Integrace	Silo-přístup	Agile, DevOps, Lean
Zaměření	IT operace	Tvorba hodnoty pro business
Certifikace	Foundation, Practitioner...	Foundation, Specialist...

Otázky profesorů k tématu: IT Governance

Pour Jan, doc. Ing., CSc.

- Co je IT Governance a jaký je rozdíl od IT Managementu?
- Jaké jsou klíčové oblasti Enterprise Governance of IT?

Maryška Miloš, doc. Ing., Ph.D.

- Popište 5 principů COBIT 5.
- Jaké jsou domény procesů v COBIT 5 a co obsahují?

Chudán David, Ing., Ph.D.

- Jaký je rozdíl mezi ITIL 3 a ITIL 4?
- Co je Service Value System v ITIL 4?

Řepa Václav, prof. Ing., CSc.

- Co je COBIT a pro koho je určen?
- Jaký je vztah mezi COBIT a ITIL?

Zeman Václav, Ing., Ph.D.

- Vyjmenujte 7 průvodních principů ITIL 4.
- Co je incident management a problem management?

9 Podnikové informační systémy

9.1 Typy podnikových IS

9.1.1 ERP (Enterprise Resource Planning)

ERP je integrovaný podnikový IS pokrývající všechny hlavní podnikové procesy.

Typické moduly ERP

- **Finance a účetnictví** – hlavní kniha, pohledávky, závazky, controlling
- **Personalistika (HR)** – správa zaměstnanců, mzdy, docházka
- **Výroba (PP – Production Planning)** – plánování výroby, kapacit, kusovníky (BOM)
- **Zásobování (MM – Materials Management)** – nákup, sklady, inventury
- **Prodej a distribuce (SD – Sales & Distribution)** – objednávky, expedice, fakturace
- **Projektový management** – řízení projektů, alokace zdrojů

Hlavní přínosy ERP

- Integrace – jeden systém, sdílená data; eliminace informačních sil
- Standardizace procesů
- Reálné informace o stavu firmy
- Automatizace rutinních procesů

Příklady ERP systémů SAP ERP (S/4HANA), Oracle ERP Cloud, Microsoft Dynamics 365, Infor, Epicor, Helios, Money S5.

9.1.2 CRM (Customer Relationship Management)

IS zaměřený na správu vztahů se zákazníky; 360° pohled na zákazníka.

Oblasti CRM

- **Operativní CRM** – automatizace prodeje (SFA), marketingu (MA) a zákaznické podpory
- **Analytické CRM** – analýza zákaznických dat; segmentace, CLV (Customer Lifetime Value), churn
- **Kolaborativní CRM** – sdílení zákaznických informací napříč kanály a odděleními

Příklady: Salesforce, Microsoft Dynamics CRM, HubSpot, SAP CRM.

9.1.3 BI (Business Intelligence)

Viz kapitola Datová analytika. BI pokrývá sběr, integraci, analýzu a prezentaci dat pro podporu rozhodování. Zahrnuje: datový sklad, ETL, OLAP, reporting, dashboardy.

9.1.4 SCM (Supply Chain Management)

IS pro správu dodavatelského řetězce – od dodavatelů surovin přes výrobu až k finálním zákazníkům.

Funkce SCM

- **Plánování** – prognózy poptávky, plánování výroby, optimalizace zásob
- **Zásobování (Procurement)** – správa dodavatelů, nákupní procesy
- **Logistika** – přeprava, skladování, distribuce
- **Zpětný tok** – reklamace, vrácení zboží

Příklady: SAP SCM, Oracle SCM Cloud, Manhattan Associates.

9.1.5 MES (Manufacturing Execution System)

MES je systém na výrobní úrovni – propojuje plánování (ERP) s fyzickým výrobním procesem.

Funkce MES

- Sledování výroby v reálném čase
- Správa výrobních zakázek
- Kontrola kvality (SPC – Statistical Process Control)
- Sledovatelnost výrobků (traceability)
- Správa výrobního zařízení (OEE – Overall Equipment Effectiveness)

Příklady: Siemens Opcenter, SAP ME, Rockwell Automation FactoryTalk, GE Digital Proficy, Wonderware (AVEVA).

9.2 Implementace podnikových IS

9.2.1 4 etapy implementace podnikového IS

1. Výběr systému a dodavatele

- Analýza požadavků, funkční a technická specifikace (RFP – Request for Proposal)
- Hodnocení nabídek (scoring model)
- Výběr z variant: hotový systém (standardní), zakázkový vývoj, kombinace

2. Uzavření smlouvy

- SLA (Service Level Agreement) – definice úrovně služeb
- Licenční podmínky, maintenance, podpora

3. Implementace a nasazení

- Projektové řízení (PM metodika)
- Konfigurace a customizace systému
- Migrace dat ze starých systémů
- Integrace s ostatními systémy
- Testování (UAT – User Acceptance Testing)
- Přejít do produkce (Big Bang nebo postupné nasazení)
- Školení uživatelů

4. Provoz a rozvoj

- Helpdesk a podpora uživatelů
- Monitoring a správa
- Upgrady a rozvoj funkcionalit
- Kontinuální zlepšování

9.2.2 IASW vs. TASW

IASW (Individual Application Software) – Individuální software

- Vytvářen specificky pro potřeby konkrétní organizace (zakázkový vývoj)
- **Výhody:** přesně odpovídá požadavkům, konkurenční výhoda, plná kontrola
- **Nevýhody:** vysoké náklady a čas vývoje, riziko projektu, závislost na dodavateli

TASW (Typical Application Software) – Typový/krabicový software

- Standardní SW vyvinutý pro obecné použití (SAP, MS Dynamics, Oracle)
- **Výhody:** nižší cena, rychlejší nasazení, best practices, stabilita, ekosystém
- **Nevýhody:** nutnost přizpůsobit procesy systému, omezená flexibilita, licenční poplatky

Hybridní přístup Typový základ + individuální rozšíření a integrace. Nejčastější v praxi.

9.3 Trendy a přínosy podnikových IS

9.3.1 Cloud Computing

Servisní modely

- **IaaS (Infrastructure as a Service)** – pronájem infrastruktury (VM, storage, síť); zákazník spravuje OS a aplikace. Příklady: AWS EC2, Azure VM, Google Compute Engine.
- **PaaS (Platform as a Service)** – pronájem platformy pro vývoj a nasazení aplikací; zákazník spravuje jen aplikace. Příklady: Azure App Service, Google App Engine, Heroku.
- **SaaS (Software as a Service)** – pronájem hotové aplikace přes internet; zákazník jen používá SW. Příklady: Salesforce, Office 365, Google Workspace, SAP S/4HANA Cloud.

Modely nasazení

- **Public Cloud** – sdílená infrastruktura, provozovaná poskytovatelem (AWS, Azure, GCP)
- **Private Cloud** – dedikovaná infrastruktura pro jednu organizaci
- **Hybrid Cloud** – kombinace public a private cloud
- **Multi-cloud** – využití více cloudových poskytovatelů

Výhody cloudových IS

- **OPEX** (Operating Expenditure) místo **CAPEX** (Capital Expenditure) – místo jednorázové velké investice do hardware a licencí (CAPEX) firma platí pravidelný provozní poplatek za službu (OPEX); náklady jsou předvídatelné a snáze schválné v rozpočtu
- Škálovatelnost – rychlé přidání kapacity
- Odpadá nutnost správy infrastruktury
- Vždy aktuální verze softwaru
- Globální dostupnost

9.3.2 SOA (Service-Oriented Architecture)

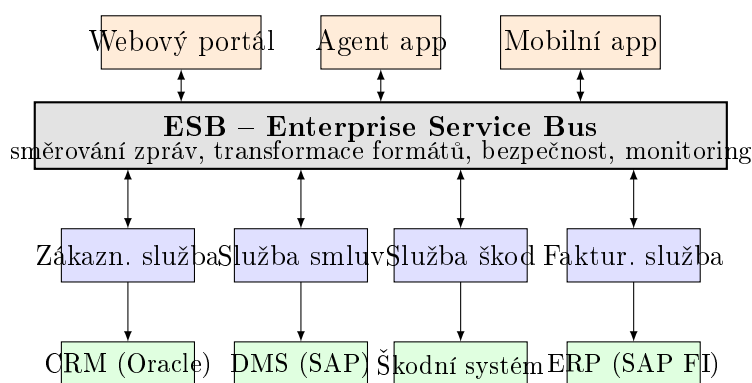
Architekturální přístup, kde jsou aplikační funkce poskytovány jako **volně propojené, opakovaně použitelné služby** komunikující přes standardizovaná rozhraní. Každá služba

dělá jednu věc dobře a je nezávislá na ostatních.

Klíčové principy SOA

- **Volné propojení (Loose Coupling)** – služby jsou nezávislé; změna jedné neovlivní ostatní
- **Znovupoužitelnost** – jedna služba slouží více aplikacím/procesům
- **Abstrakce** – konzument služby nevidí interní implementaci (zná jen rozhraní)
- **Standardizovaná rozhraní** – SOAP/WSDL (klasické SOA) nebo REST/JSON (moderní)
 - **SOAP** (Simple Object Access Protocol) – protokol pro výměnu zpráv mezi službami; zprávy jsou formátovány v **XML** a přenášeny typicky přes HTTP; striktní, silně typovaný, vhodný pro enterprise prostředí s požadavky na bezpečnost a transakce
 - **WSDL** (Web Services Description Language) – XML dokument popisující rozhraní služby: jaké operace nabízí, jaké parametry přijímá a co vrací; slouží jako “kontrakt” – konzument si načte WSDL a přesně ví, jak službu volat
- **ESB (Enterprise Service Bus)** – centrální sběrnice; zajišťuje směrování, transformaci formátů, bezpečnost a monitoring zpráv mezi službami

Praktický příklad: SOA v pojišťovně *Scénář: pojišťovna má oddělené systémy pro zákazníky, pojistné smlouvy, škody a fakturaci. Bez SOA by každý systém volal ostatní přímo – „špagetová architektura”. SOA tyto systémy integruje přes standardizované služby a ESB.*



Obrázek 16: SOA architektura pojišťovny – ESB integruje heterogenní systémy

Scénář – zákazník nahlásí škodu přes webový portál:

1. Webový portál pošle požadavek **na ESB**: “Nová škodní událost, zákazník ID 4821”
2. ESB **ověří autentizaci** a přesměruje dotaz na **Zákaznickou službu**: “Existuje zákazník 4821? Má aktivní smlouvu?”
3. ESB souběžně volá **Službu smluv**: “Vrať detail smlouvy zákazníka 4821”

4. ESB předá data **Službě škod**: “Vytvoř novou škodní událost č. SKD-2024-8871”
5. Služba škod zavolá **Fakturační službu**: “Zablokuj pojistné plnění do vyřešení”
6. ESB vrátí odpověď webovému portálu: “Škoda přijata, číslo SKD-2024-8871”

Přitom webový portál, mobilní aplikace i agentská aplikace **všechny sdílí stejné služby** – žádný duplicitní kód. Fakturační systém je SAP, CRM je Oracle – ESB překládá formáty.

SOA vs. Mikroservisy

Vlastnost	SOA	Mikroservisy
Komunikace	ESB (centralizovaná)	REST/gRPC (přímá, decentralizovaná)
Granularita	Větší, sdílené služby	Malé, samostatně nasaditelné
Technologie	SOAP, WSDL, XML	REST, JSON, gRPC
Nasazení	Monolitické bloky	Docker, Kubernetes (kontejnery)
Typické použití	Enterprise integrace (ERP, CRM)	Cloud-native aplikace, startupy
Příklad	Pojišťovna, banka	Netflix, Amazon, Spotify

SOA a mikroservisy sdílejí základní myšlenku (volné propojení, znovupoužitelnost), ale liší se v granularitě a způsobu komunikace. Mikroservisy jsou v podstatě SOA bez ESB centralizace.

9.3.3 Přínosy a rizika podnikových IS

Přínosy

- Zvýšení efektivity procesů a produktivity
- Lepší dostupnost a přesnost informací
- Podpora strategického rozhodování
- Standardizace a optimalizace procesů
- Snížení provozních nákladů

Rizika implementace

- Překročení rozpočtu a harmonogramu (průměrně 30-40% IT projektů)
- Odpor uživatelů ke změně (change management)
- Nekvalitní migrace dat
- Nevyhovující přizpůsobení procesů
- Závislost na dodavateli (vendor lock-in)

9.3.4 Ekonomické hodnocení IS investic

Statické metody (bez zohlednění času)

- **ROI (Return on Investment)** – $\frac{\text{Čistý zisk}}{\text{Investice}} \times 100\%$; rychlé porovnání variant
- **TCO (Total Cost of Ownership)** – celkové náklady vlastnictví: pořizovací náklady + provozní náklady za dobu životnosti (licence, HW, podpora, školení, migrace)
- **Payback Period** – doba, za kterou se investice vrátí z úspor nebo výnosů

Dynamické metody (zohledňují časovou hodnotu peněz)

- **NPV (Net Present Value)** – čistá současná hodnota; diskontované budoucí toky minus investice; $\text{NPV} > 0$ = projekt je výhodný
- **IRR (Internal Rate of Return)** – vnitřní výnosové procento; diskontní sazba, při které $\text{NPV} = 0$
- **PI (Profitability Index)** – $\frac{\text{PV budoucích toků}}{\text{Investice}}$; $\text{PI} > 1$ = výhodné

ABC (Activity-Based Costing) Kalkulace nákladů dle aktivit – přiřazuje náklady ke konkrétním IT aktivitám a službám; umožňuje přesněji vyhodnotit nákladovost jednotlivých IS komponent nebo procesů.

Benchmarking IS Porovnání výkonnosti IS s odvětvovými standardy nebo konkurencí (Gartner IT benchmarks, ITIL benchmarking).

Otázky profesorů k tématu: Podnikové IS

Pour Jan, doc. Ing., CSc.

- Co je ERP a jaké jsou jeho typické moduly?
- Jaké jsou 4 etapy implementace podnikového IS?
- Jaký je rozdíl mezi CRM a ERP?

Řepa Václav, prof. Ing., CSc.

- Jaký je rozdíl mezi IASW a TASW? Kdy co použít?
- Jaké jsou přínosy a rizika implementace podnikového IS?

Chudán David, Ing., Ph.D.

- Co je SaaS a jak se liší od IaaS a PaaS?
- Co je SOA a jaké jsou jeho výhody?

Maryška Miloš, doc. Ing., Ph.D.

- Co je MES a jak se liší od ERP?
- Jaký je rozdíl mezi Public, Private a Hybrid cloud?

Sudzina František, Mgr. et Mgr. Ing., Ph.D.

- Co je SCM a jaké jsou jeho hlavní funkce?
- Jaké jsou výhody cloudových IS oproti on-premise řešením?

10 Analýza a návrh informačních systémů

10.1 Propojení IT a businessu

10.1.1 Proč je propojení IT a businessu důležité

IT a business musejí být vzájemně sladěny, aby IT investice přinášely skutečnou hodnotu. Bez souladu vznikají situace, kdy IT dodává technologie, které business nepotřebuje, nebo business má potřeby, které IT nedokáže podpořit.

Business/IT alignment (soulad IT a businessu) = míra, do jaké IT strategie, cíle, aktivity a procesy podporují podnikovou strategii, cíle, aktivity a potřeby.

10.1.2 SAM – Strategic Alignment Model

SAM (Henderson & Venkatraman, 1993) je nejznámější model pro popis vztahu mezi podnikovou a IT strategií. Pracuje se **čtyřmi doménami**:

Co je strategická úroveň? Strategická úroveň odpovídá na otázku “**Kam chceme jít a proč?**”. Zabývá se dlouhodobými záměry, konkurenčním postavením a rozhodnutími, která určují směr celé organizace. Rozhodují zde **vrcholoví manažeři a představenstvo** – horizontem jsou roky.

- **Business strategie** – “Budeme e-shop číslo 1 v ČR do roku 2027; vstoupíme na slovenský trh; zavedeme prémiové členství” – odpovídá na: v jakém businessu chceme být? Jak chceme konkurovat? Co je naše hodnotová nabídka?
- **IT strategie** – “Přejdeme do cloudu (AWS); zavedeme microservisy; investujeme do AI personalizace” – odpovídá na: jaké technologie a IT schopnosti potřebujeme k podpoře business strategie v dlouhodobém horizontu?

Co je operační úroveň? Operační úroveň odpovídá na otázku “**Jak to každý den funguje?**”. Zabývá se každodenním provozem, procesy a infrastrukturou, která strategii realizuje. Rozhodují zde **manažeři středního řízení a IT provoz** – horizontem jsou týdny až měsíce.

- **Organizační infrastruktura a procesy** – jak jsou organizovány týmy, jaké jsou podnikové procesy, role a zodpovědnosti; “Zákaznický servis funguje 24/7, máme call centrum o 50 lidech, objednávkový proces trvá 3 kroky”
- **IT infrastruktura a procesy** – servery, sítě, aplikace, databáze, helpdesk, SLA, monitoring; “Máme 3 produkční servery v AWS, automatický deployment přes CI/CD, databáze PostgreSQL, service desk reaguje do 4 hodin”

	Business	IT
Strategická (“Kam jdeme?”)	Business strategie <i>Příklad: vstup na SK trh, prémiové členství</i>	IT strategie <i>Příklad: migrace do AWS, AI personalizace</i>
Operační (“Jak to běží?”)	Org. infrastruktura a procesy <i>Příklad: call centrum 50 lidí, 3-krokový objednávkový proces</i>	IT infrastruktura a procesy <i>Příklad: PostgreSQL, CI/CD, 4h SLA odezva</i>

Klíčový rozdíl: Strategie říká *co a proč*, operativa říká *jak konkrétně*. Bez souladu mezi těmito čtyřmi doménami se stává, že IT koupí drahý cloud (IT strategie), ale business procesy jsou stále papírové (org. infrastruktura) – investice nepřinese hodnotu.

Dvě osy souladu:

- **Strategická shoda (Strategic Fit)** – *vertikální*; soulad strategie s operativou v rámci jedné strany (business strategie musí být realizovatelná v org. procesech; IT strategie musí odpovídat IT infrastruktuře)
- **Funkční integrace (Functional Integration)** – *horizontální*; soulad business a IT na stejné úrovni (business strategie a IT strategie musejí jít stejným směrem; stejně tak business procesy a IT systémy)

4 perspektivy dosažení alignment dle SAM Každá perspektiva říká, **kdo táhne** změnu a **kdo se přizpůsobuje**. V perspektivách 1 a 2 táhne *business* a IT reaguje; v perspektivách 3 a 4 táhne *IT* a business reaguje.

1. Strategy Execution (Business táhne, IT následuje)

Business strategie → Org. infrastruktura → IT infrastruktura. Management definuje strategii a org. procesy; IT pak pouze doplní potřebné systémy. Nejběžnější pohled – IT je zde *nástrojem*.

Příklad: „Expandujeme na slovenský trh” → „Otevřeme nové obchodní procesy a call centrum” → „IT nasadí vícejazyčný CRM.”

2. Technology Transformation (Business táhne, IT buduje strategii)

Business strategie → IT strategie → IT infrastruktura. Business opět určuje směr, ale IT si samostatně odvodí vlastní strategii a pak buduje infrastrukturu. Rozdíl od Strategy Execution: IT strategii *aktivně formuluje*, nejen implementuje.

Příklad: „Chceme být e-shop č. 1” → „Potřebujeme AI personalizaci a cloudovou škálovatelnost” → „Stavíme mikroservisy na AWS.”

3. Competitive Potential (IT táhne, business reaguje)

IT strategie → Business strategie → Org. infrastruktura. Nová technologická schopnost *otevře nové business příležitosti* – IT inovuje a business se přizpůsobí.

Příklad: „Máme AI doporučovací engine” → „Můžeme nabídnout personalizované předplatné” → „Restrukturujeme obchodní tým kolem subscription modelu.”

4. **Service Level** (IT táhne, business přizpůsobuje procesy)

IT strategie → IT infrastruktura → Org. infrastruktura. IT buduje excelentní servisní platformu a business procesy se pak přizpůsobí jejím možnostem. IT je zde *interní servisní organizací* – důraz na kvalitu a spolehlivost IT služeb.

Příklad: „Máme nejlepší cloudovou platformu“ → „Optimalizujeme infrastrukturu“ → „Obchodní procesy jsou redesignovány, aby využily naše efektivní IT prostředí.“

10.1.3 IS/IT Strategie

IS/IT strategie je plán, jak organizace využije IS/IT k dosažení podnikových cílů.

Složky IS/IT strategie

- **IS strategie** – *co* potřebujeme (aplikace, data, informace pro podporu businessu)
- **IT strategie** – *jak* to dodáme (technologie, infrastruktura, platformy)
- **IM strategie** – správa informačních zdrojů, datová governance

Tvorba IS/IT strategie

1. Analýza podnikového prostředí (SWOT, PEST, Porter 5 sil)
2. Pochopení podnikové strategie a cílů
3. Analýza stávajícího IS/IT (portfolio aplikací, GAP analýza)
4. Identifikace příležitostí pro IS/IT
5. Formulace IS/IT strategie (cíle, priority, roadmapa)

Analýza portfolia aplikací – McFarlanův strategic grid Aplikace jsou zařazeny dle strategické důležitosti:

- **Strategic** – klíčové pro budoucí strategii i dnešní provoz
- **Key Operational** – kritické pro chod firmy dnes, ale bez strategického potenciálu
- **High Potential** – experimentální, inovativní; vysoký budoucí potenciál
- **Support** – nutné, ale ne strategické; kandidát na outsourcing

10.1.4 Podniková architektura (Enterprise Architecture)

PA je holistický popis organizace z různých pohledů – propojuje business potřeby s IT řešením.

Vrstvy podnikové architektury

1. **Business architektura** – organizační struktura, procesy, role, cíle
2. **Datová architektura** – datové modely, datové toky, master data
3. **Aplikační architektura** – aplikace, jejich funkce a integrace
4. **Technologická architektura** – hardware, síť, platformy, middleware

Co je architektonický rámec a k čemu slouží? Představ si, že firma roste a má desítky aplikací, stovky procesů, vlastní datová centra a stovky zaměstnanců. Každé oddělení si pořizuje systémy po svém, nikdo neví jak do sebe všechno zapadá, výsledkem je chaos a duplicita.

Architektonický rámec je *strukturovaná metodika, která říká jak popsat, naplánovat a řídit celou podnikovou architekturu* – tedy jak firma funguje (procesy), co uchovává (data), jaké systémy používá (aplikace) a na jaké infrastruktuře (technologie). Rámec poskytuje společný jazyk a šablony pro všechny zúčastněné – architekty, manažery i vývojáře.

Architektonické rámce TOGAF (The Open Group Architecture Framework)

Nejrozšířenější architektonický rámec na světě. Vydává ho The Open Group; volně dostupný po registraci.

- **Co řeší:** kompletní proces tvorby a údržby podnikové architektury od začátku do konce
- **Klíčový nástroj: ADM (Architecture Development Method)** – iterativní cyklus 9 fází:
 - **Preliminary** – příprava; definice principů architektury
 - **A – Architecture Vision** – co chceme dosáhnout? Scope, stakeholderi
 - **B – Business Architecture** – jak funguje business (procesy, role, cíle)
 - **C – Information Systems Architecture** – data a aplikace
 - **D – Technology Architecture** – infrastruktura, síť, hardware
 - **E–H** – plánování přechodu, řízení implementace a governance
- **Výstup:** Architecture Repository – katalog aplikací, datových toků, komponent
- **Příklad použití:** velká banka chce migrovat do cloudu; TOGAF ADM provede tým od analýzy stávajícího stavu (“as-is”) přes návrh cílového stavu (“to-be”) až po plán migrace

Zachmanův rámec (Zachman Framework)

K čemu slouží: Zachmanův rámec zajišťuje, že IS je popsán *úplně* – z pohledu každého stakeholdera a ze všech relevantních úhlů. Různí lidé vidí systém jinak: manažer řeší strategii, architekt logický návrh, vývojář fyzickou implementaci. Rámec říká: *každý stakeholder musí umět odpovědět na všech 6 otázek* – jinak architektura obsahuje slepá

místa. Neříká *jak* architektonické práce dělat (na rozdíl od TOGAF), ale *co* musí být popsáno.

Co vyjadřuje buňka tabulky: Každá buňka je průsečíkem *pohledu* (kdo se dívá) a *otázky* (na co odpovídá). Popisuje konkrétní artefakt, který daný stakeholder potřebuje pro danou dimenzi. Například: Owner × Co? = sémantický datový model (co business považuje za klíčové entity); Builder × Co? = fyzický datový model v databázi.

Pohled Otázka	\ Co? (data)	Jak? (funkce)	Kde? (sítě)	Kdo? (lidé)	Kdy? (čas)	Proč? (motivace)
Planner (manažer)	Obchodní entity	Procesy	Lokality	Org. jednotky	Události	Strategie
Owner (vlastník)	Sémantický model	Biz. procesy	Biz. logistika	Org. workflow	Hlavní plány	Biz. plán
Designer (architekt)	Logický datový model	Aplikační funkce	Distribuovaná arch.	ITW architektura	Procesní struktura	Pravidla
Builder (vývojář)	Fyzický datový model	Systémový design	Systémová architektura	Prezentace	Řízení	Pravidla DB
...	...	...	...	...	...	...

- *Příklad:* sloupec “Co?” na řádce “Planner” = seznam klíčových obchodních entit (zákazník, produkt, smlouva); na řádce “Builder” = fyzický datový model v databázi
- Zachman říká: každý úhel pohledu (manažer, architekt, vývojář) musí umět zodpovědět všech 6 otázek – jinak architektura není úplná

ARIS (Architecture of Integrated Information Systems)

Rámec a nástroj od Prof. Augusta-Wilhelma Scheera (Saarlandská univerzita, 1992). Zaměřen zejména na **modelování podnikových procesů** a jejich propojení s IT.

• 5 pohledů ARIS:

- **Organizační pohled** – organizační struktura, role, zodpovědnosti
- **Funkční pohled** – funkce a jejich hierarchie (co systém/proces dělá)
- **Datový pohled** – datové objekty, stavy, události (vstup/výstup funkcí)
- **Procesní pohled** – propojení všech pohledů; modeluje se pomocí **EPC diagramů**
- **Výkonový pohled** – metriky, KPIs, náklady a výkonnost procesů
- **Nástroj ARIS Toolset** – software pro modelování EPC a dalších diagramů; hojně využívaný v SAP implementacích (SAP používá ARIS pro dokumentaci svých procesů)
- *Příklad:* e-shop popisuje proces “Vyřízení objednávky” v ARIS: organizační pohled (role: skladník, účetní), datový pohled (objednávka, faktura), procesní pohled (EPC diagram s tokem událostí a funkcí)

Rámec	Silná stránka	Typické použití	Výstup
TOGAF	Komplexní proces (ADM); mezinárodní standard	Velké firmy; IT transformace; cloud migrace	Architecture Repository
Zachman	Ucelené pohledy; nic neopomenout	Akademické prostředí; audit úplnosti architektury	Matice pohledů
ARIS	Procesní modelování; propojení s SAP	Procesní reengineering; SAP implementace	EPC diagramy, procesní dokumentace

10.1.5 Digitální transformace

IT přestává být jen podpůrnou funkcí (“správa serverů”) – stává se strategickým partnerem businessu, který aktivně vytváří nové produkty, zákaznické zkušenosti a obchodní modely.

Shadow IT – co to je a proč vzniká? Shadow IT jsou IT systémy, aplikace nebo služby, které zaměstnanci nebo oddělení **pořizují a používají bez vědomí nebo schválení IT oddělení.**

Jak vzniká:

- Marketingové oddělení potřebuje rychle spustit kampaň – IT říká “za 3 měsíce”; marketing si sám zaregistruje Mailchimp nebo HubSpot a platí z vlastního rozpočtu
- Obchodníci si nainstalují Dropbox nebo Google Drive, protože firemní SharePoint “je pomalý a komplikovaný”
- Vývojáři používají vlastní GitHub účty místo firemního GitLabu
- Účetní tvoří klíčové reporty v Excelu, které nikdo jiný nevidí ani nezalohuje

Proč to zaměstnanci dělají	Rizika pro firmu
IT je pomalé a byrokratické	Firemní data mimo kontrolu (GDPR!)
Firemní nástroje jsou zastaralé	Bezpečnostní zranitelnosti bez IT dohledu
Jednodušší SaaS nástroje jsou dostupné	Ztráta dat při odchodu zaměstnance
Vlastní oddělení chce mít kontrolu	Duplicitní data, nekonzistentní informace
	Porušení licenčních podmínek

Jak shadow IT řešit:

- **Poznat ho** – audit nástrojů (nástroje jako Netskope, Casb) odhalí cloudové aplikace v síti
- **Pochopit proč vzniká** – není to vzpoura; je to signál, že IT neslouží businessu dobře

- **Řešení:** zrychlit IT procesy; schvalovat ověřené SaaS nástroje do “katalogu”; zavést self-service portál kde si oddělení mohou vybrat schválené nástroje sami

Bimodální IT – co to je a proč vzniklo? Tradiční IT oddělení funguje jako jedno těleso – stejní lidé a stejné procesy obsluhují jak kritické bankovní systémy (kde chyba = katastrofa) tak nové mobilní aplikace (kde rychlost = konkurenční výhoda). Tyto dva světy mají protikladné požadavky.

Bimodální IT (Gartner, 2014) navrhuje rozdělit IT do **dvou oddělených módů**:

Vlastnost	Mode 1 – “Klasický”	Mode 2 – “Agile”
Cíl	Spolehlivost, stabilita, bezpečnost	Rychlost, inovace, experimenty
Metodika	Waterfall, ITIL, kontrolované změny	Agile, Scrum, DevOps, CI/CD
Cyklus	Měsíce až roky	Dny až týdny
Riziko	Velmi nízké (nulová tolerance chyb)	Vyšší (fail fast, learn fast)
Příklady systémů	Core banking, ERP, mzdy, billing	Mobilní app, web, AI produkty, IoT
Tým	Zkušení specialisté, stabilní tým	Malé cross-functional týmy, startupy

Konkrétní příklad – Česká spořitelna:

- **Mode 1:** Core banking systém (SAP/Murex) zpracovává miliony transakcí denně; jakákoliv chyba = finanční ztráty a regulační problémy; nasazení nové verze probíhá jednou za čtvrtletí s rozsáhlým testováním
- **Mode 2:** Mobilní aplikace George; nová funkce (push notifikace, FaceID login) se vydá každé 2 týdny; chyba = špatné recenze, ale žádná systémová katastrofa; tým experimentuje a rychle reaguje na feedback uživatelů

Kritika bimodálního IT: Někteří odborníci (zejména z DevOps komunity) bimodální IT kritizují – oddělení prý vytváří “dvě rychlosti” které se navzájem nespojí. Preferují místo toho **jednotnou DevOps kulturu** kde i core systémy lze nasazovat rychle díky automatizaci a testování. V praxi ale většina velkých korporací bimodální přístup de facto uplatňuje, i když ho tak nenazývá.

CDO (Chief Digital Officer) Relativně nová C-level role zodpovědná za digitální transformaci organizace. CDO propojuje business a technologie – na rozdíl od CIO (který řídí IT operace) se CDO zaměřuje na *vytváření nových digitálních produktů, zákaznických zkušeností a obchodních modelů*. *Příklad:* CDO v retailu zavede omnichannel strategii (e-shop + kamenné obchody + mobilní app jako jeden integrovaný celek) a řídí datové analytické projekty pro personalizaci.

10.2 MMDIS – Multidimenzionální metoda pro IS

MMDIS (Multidimensional Method for IS – Multidimenzionální metoda pro IS) je komplexní metodický rámec pro analýzu, návrh a rozvoj informačních systémů, vyvinutý na VŠE Praha.

10.2.1 11 principů MMDIS

1. **Princip systémové integrace** – IS je chápán jako celek, ne jako sumu dílčích systémů
2. **Princip architektonického pohledu** – různé pohledy na IS (procesní, datový, aplikační, technologický)
3. **Princip opakovaného použití** – znovupoužití existujících komponent a řešení
4. **Princip otevřenosti** – IS musí být otevřený pro integraci a rozšíření
5. **Princip konzistence** – konzistence napříč všemi pohledy a úrovněmi abstrakce
6. **Princip komplexnosti** – úplné pokrytí všech aspektů IS
7. **Princip stupňování abstrakce** – postupné zpřesnění od strategické po implementační úroveň
8. **Princip zapojení uživatelů** – aktivní účast uživatelů v celém procesu
9. **Princip iterativního přístupu** – opakování a zpřesňování
10. **Princip dokumentace** – průběžná dokumentace artefaktů
11. **Princip nezávislosti metody** – metoda je nezávislá na konkrétní technologii

10.2.2 9 fází MMDIS (skupiny GST-VY)

Fáze jsou rozděleny do tří skupin: **G**lobal analysis, **S**olution design, **T**ransformation.

Skupina G – Globální analýza (strategická úroveň)

- **G1 – Vymezení a analýza prostředí** – analýza organizačního prostředí, business cílů a strategií
- **G2 – Formulace IS/ICT strategie** – definice cílů, priorit a směřování IS/ICT rozvoje
- **G3 – Analýza existujícího IS** – posouzení stávajícího stavu IS/ICT (GAP analýza)

Skupina S – Systémová analýza a návrh

- **S1 – Analýza procesů** – modelování a optimalizace podnikových procesů (EPC, BPMN)

- **S2 – Analýza dat** – konceptuální datový model (ER diagram, UML class)
- **S3 – Analýza aplikací** – požadavky na aplikační podporu, use case diagramy
- **S4 – Analýza technologií** – technologická infrastruktura, platformy

Skupina T – Transformace

- **T1 – Výběr dodavatele/řešení** – výběrové řízení, hodnocení nabídek
- **T2 – Implementace a nasazení** – projekt implementace, migrace, školení

Pozn.: VY = výsledky (ne fáze); každá fáze produkuje **artefakty** – konkrétní dokumenty nebo modely, které jsou výstupem fáze a vstupem do fáze následující (např. procesní mapa, ER diagram, funkční specifikace, testovací plán).

10.2.3 3 skupiny pohledů na IS

MMDIS pracuje s třemi základními pohledy (dimenzemi):

1. **Procesní pohled** – business procesy, workflow, aktivity a jejich vazby; odpovídá na otázku „*Co se děje a v jakém pořadí?*“; nástroje: EPC, BPMN, vývojové diagramy
2. **Datový pohled** – datové modely, entity, atributy, vztahy; odpovídá na otázku „*S jakými daty pracujeme a jak jsou strukturována?*“; nástroje: ER diagram, UML class diagram
3. **Aplikační (funkční) pohled** – aplikace, jejich funkce a integrace; odpovídá na otázku „*Jaký software to podporuje?*“

Rozdíl procesního a datového pohledu: Procesní pohled zachycuje *dynamiku* – jak informace tečou, kdo co kdy dělá. Datový pohled zachycuje *strukturu* – jaká data existují, jaké mají atributy a jak spolu souvisejí. Oba pohledy jsou nutné: procesní pohled bez datového neví, s čím proces pracuje; datový pohled bez procesního neví, kdy a jak se data mění.

Doplňující pohled: **Technologický pohled** – hardware, software, síť, bezpečnost.

10.3 Procesní modelování

10.3.1 Základní pojmy

- **Podnikový proces** – soubor vzájemně propojených nebo vzájemně působících činností, který přeměňuje vstupy na výstupy (ISO 9000)
- **Instance procesu** – konkrétní průběh procesu (jeden případ, case)
- **Aktivita (Activity)** – elementární část procesu; akce nebo úkol vykonávaný v procesu
- **Trigger (spouštěč)** – událost zahajující proces
- **Swimlane** – grafické oddělení zodpovědností v diagramu procesu

10.3.2 EPC (Event-driven Process Chain)

EPC je modelovací jazyk pro podnikové procesy; vyvinut v rámci frameworku ARIS (Prof. Scheer). Hojně využívaný v SAP projektech.

Elementy EPC

- **Událost (Event)** – stav nebo výskyt iniciující nebo výsledek aktivity; tvar hexagonu
 - Spouštěcí událost – zahajuje aktivitu (př.: „Objednávka přijata“)
 - Výsledná událost – ukončuje aktivitu (př.: „Faktura odeslána“)
- **Funkce (Function)** – aktivita/úkol; tvar zaoblené obdélníku (př.: „Zkontrolovat dostupnost zboží“, „Vystavit fakturu“)
- **Logické spojky (Connectors):**
 - **AND** – všechny větve aktivní (paralelní zpracování) (př.: *po přijetí objednávky se zároveň připraví zboží i vystaví faktura*)
 - **OR** – jedna nebo více větví (inkluzivní) (př.: *zákazník může dostat e-mail, SMS nebo obojí*)
 - **XOR** – právě jedna větev (exkluzivní, alternativa) (př.: *objednávka je buď schválena, nebo zamítnuta – nikdy obojí*)
- **Organizační jednotka** – kdo vykonává funkci (př.: „Účetní oddělení“, „Skladník“, „Systém ERP“)
- **Datový/informační objekt** – data vstupující do nebo výstupující z funkce (př.: „Objednávkový formulář“ jako vstup funkce „Zpracovat objednávku“)

Pravidla EPC

- Proces musí začínat a končit událostí
- Alternují se události a funkce (ne dvě funkce nebo dvě události za sebou)
- Spojky mohou větvit nebo slučovat toky

10.3.3 BPMN (Business Process Model and Notation)

BPMN 2.0 je standard OMG pro grafické znázornění podnikových procesů. Nahrazuje EPC jako de facto standard.

Základní kategorie elementů BPMN Flow Objects (Objekty toku):

- **Events (Události)** – kruhy; zahajují, přerušují nebo ukončují proces:
 - Start Event (prázdný kruh) (př.: „Zákazník podal reklamaci“)
 - End Event (plný kruh) (př.: „Reklamace vyřešena“)

- Intermediate Event (dvojitý kruh) – timer, zpráva, chyba (př.: *timer – čekej 3 dny na odpověď zákazníka; zpráva – přišel e-mail s potvrzením*)
- **Activities (Aktivity)** – zaoblené obdélníky:
 - Task – elementární úkol (př.: *„Zkontrolovat záruční dobu“*)
 - Sub-Process – rozvinutelná aktivita obsahující vlastní proces (př.: *„Zpracovat platbu“ – uvnitř má vlastní kroky: ověření karty, strhnutí částky, potvrzení*)
 - Call Activity – volání globálně definovaného procesu (př.: *sdílený proces „Ověření totožnosti“ používaný ve více různých diagramech*)
- **Gateways (Brány)** – kosočtverce; větvení a slučování toku:
 - **Exclusive (XOR)** – právě jedna větev (př.: *je zboží skladem? Ano → expeduj; Ne → objednej od dodavatele*)
 - **Parallel (AND)** – všechny větve (+) (př.: *po schválení objednávky se paralelně spustí příprava balíčku i odeslání potvrzovacího e-mailu*)
 - **Inclusive (OR)** – jedna nebo více větví (př.: *zákazník si může zvolit dopravu, pojištění nebo obojí*)
 - **Event-based** – větvení na základě příchozí události (př.: *čeká se, co přijde dříve – platba nebo storno objednávky*)

Connecting Objects:

- **Sequence Flow** – plná šipka; tok řízení *vnitř jednoho poolu* – určuje pořadí aktivit (př.: *šipka z „Ověřit objednávku“ na „Vystavit fakturu“*)
- **Message Flow** – přerušovaná šipka s prázdnou hlavičkou; komunikace *mezi pooly* (různými účastníky/organizacemi) (př.: *zákazník posílá objednávku do poolu e-shopu*)
- **Association** – tečkovaná čára; propojení datových objektů s aktivitami (př.: *dokument „Smlouva“ asociovaný s aktivitou „Podepsat smlouvu“*)

Rozdíl Sequence Flow vs. Message Flow: Sequence Flow říká *co přijde po čem* – je to časová a logická návaznost kroků v rámci jednoho procesu. Message Flow říká *kdo s kým komunikuje* – přenáší zprávu přes hranici mezi dvěma samostatnými účastníky. Sequence Flow nikdy nepřekračuje hranici poolu; Message Flow naopak *vždy* propojuje různé pooly.

Paralelní zpracování v BPMN: Modeluje se pomocí **Parallel Gateway (AND, symbol +)**. Rozbíhací AND gateway spustí *všechny* odchozí větve najednou – žádná podmínka se nevyhodnocuje. Sbíhací AND gateway čeká, dokud *nepřijdou všechny* příchozí větve, pak pokračuje. *Příklad:* po schválení objednávky rozbíhací AND spustí paralelně „Příprav zásilku“ i „Odešli potvrzovací e-mail“; sbíhací AND počká na oba, pak přejde na „Uzavři objednávku“.

Swimlanes:

- **Pool** – celý proces nebo participant (organizace, systém) (př.: *jeden pool „E-shop“, druhý pool „Zákazník“ – každý má vlastní tok*)

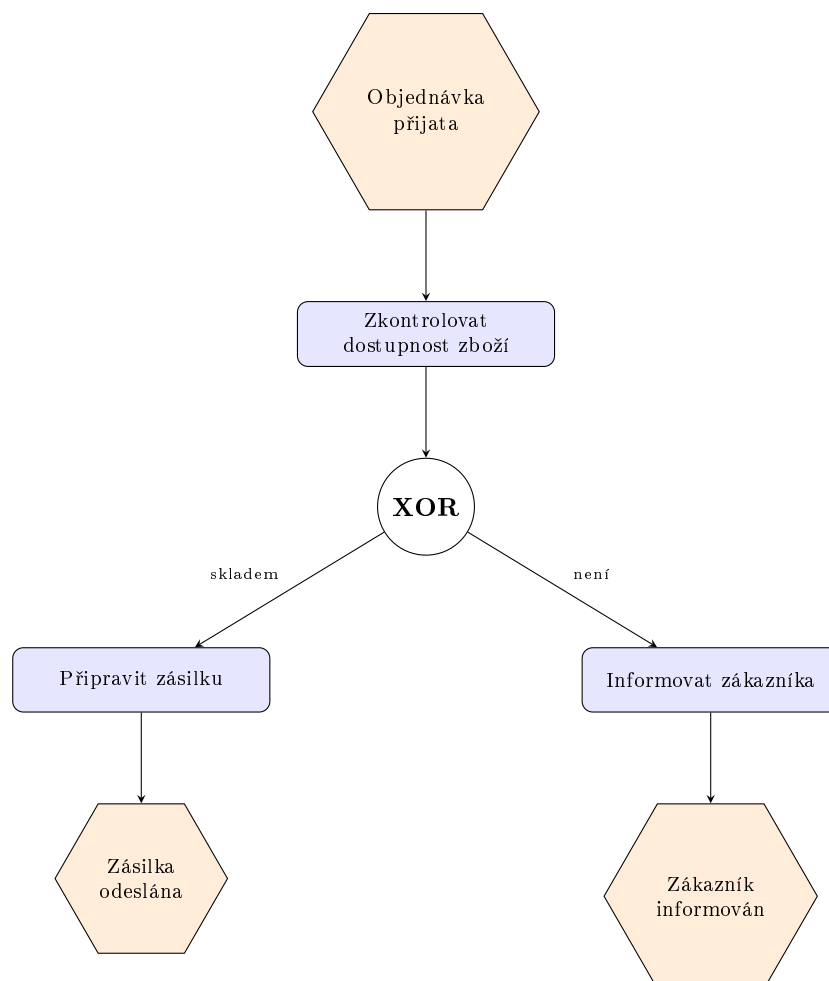
- **Lane** – plavecká dráha v rámci poolu; odděluje zodpovědnosti rolí (př.: v poolu „E-shop“ jsou lanes „Obchodní oddělení“, „Sklad“, „Účetnictví“)

Artifacts:

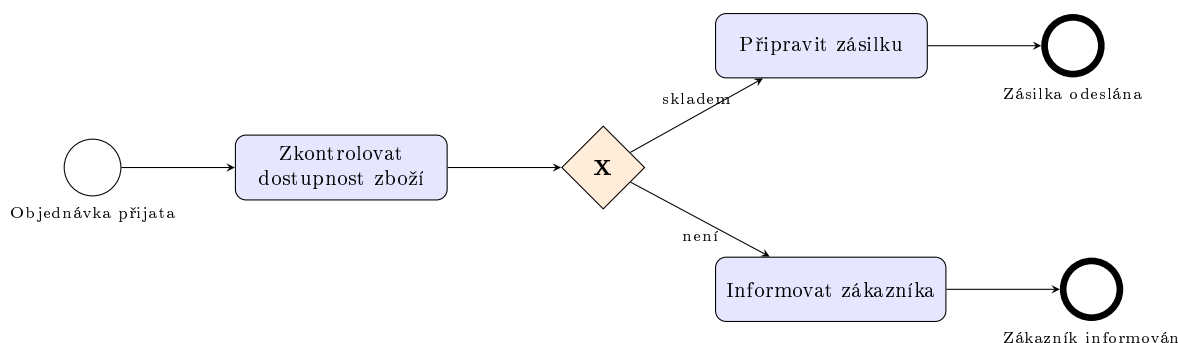
- **Data Object** – konkrétní dokument nebo datová položka (př.: „Faktura.pdf“ přiřazená aktivitě „Odeslat fakturu“)
- **Data Store** – trvalé úložiště dat (př.: databáze zákazníků, do které se zapisuje po registraci)
- **Text Annotation** – vysvětlující poznámka v diagramu (př.: „SLA: vyřídit do 24 hod.“ připojená k aktivitě „Zpracovat reklamaci“)

Příklad: Zpracování objednávky – EPC vs. BPMN Stejný proces zapsaný v obou notacích: zákazník podá objednávku, systém ověří dostupnost zboží; je-li skladem, zásilka se připraví a odešle, jinak zákazník dostane informaci o nedostupnosti.

EPC – orientace shora dolů, události střídají funkce:



BPMN – orientace zleva doprava, explicitní Start/End Event:



Co diagramy ilustrují:

- **Start a konec** – EPC nemá speciální symbol; první a poslední hexagon jsou implicitně začátek a konec. BPMN má explicitní Start Event (tenký kroužek) a End Event (tučný kroužek).
- **Větvení** – EPC: kroužek s textem *XOR*; BPMN: diamant s *X* uvnitř – vizuálně odlišné, ale stejná sémantika (právě jedna větev).
- **Povinné střídání** – v EPC musí po každé funkci (modrý obdélník) následovat událost (oranžový hexagon) – proto jsou dva koncové hexagony. V BPMN po úkolu může rovnou následovat End Event bez mezivýsledkových událostí.
- **Orientace** – EPC konvence: shora dolů; BPMN konvence: zleva doprava (přirozené pro swimlanes).

Srovnání EPC vs BPMN

Aspekt	EPC	BPMN
Standard	ARIS (Prof. Scheer)	OMG standard
Použití	SAP projekty, ARIS tool	Obecný standard
Exekuce	Obtížná	BPMN 2.0 je spustitelný (BPEL)
Složitost	Jednodušší	Bohatší sémantika
Adopce	Klesá	Rostě, de facto standard

10.3.4 KBPR (Key Business Process Requirements)

Klíčové požadavky na podnikové procesy – rámec pro hodnocení procesů:

- Efektivnost (Effectiveness) – proces dosahuje zamýšlených výsledků
- Eficiencia (Efficiency) – minimální spotřeba zdrojů
- Flexibilita – schopnost adaptace na změny
- Bezpečnost – ochrana informací a zdrojů
- Měřitelnost – jasné metriky výkonnosti

10.4 Architektura IS a způsoby zpracování

10.4.1 MDA (Model Driven Architecture)

OMG (Object Management Group) – mezinárodní nezisková standardizační organizace pro IT; vydává standardy jako UML, BPMN, MDA. Říká se jim OMG standardy, protože za nimi stojí tato organizace.

MDA (OMG standard) – přístup k vývoji SW, kde je **model** primárním artefaktem, nikoliv kód. Myšlenka: nejprve popište systém abstraktně (co má dělat, bez technologie), pak automaticky transformujte na platformně specifický model a nakonec na kód.

- **CIM (Computation Independent Model)** – *byznys pohled*; popisuje, co systém dělá z pohledu byznysu, bez jakéhokoli zmínění technologie nebo implementace; typicky BPMN diagram nebo textový popis požadavků; “model” zde znamená formalizovaný popis byznysu
- **PIM (Platform Independent Model)** – model systému nezávislý na konkrétní platformě; popsán v UML; zachycuje logiku a strukturu bez technologických detailů
- **PSM (Platform Specific Model)** – model přizpůsobený konkrétní platformě (Java EE, .NET, Web); generován transformací z PIM
- Z PSM lze automaticky generovat kód

Transformace: $CIM \rightarrow PIM \rightarrow PSM \rightarrow Code$; každá transformace je řízenou, opakovatelnou operací.

10.4.2 Způsoby zpracování dat v IS

- **Dávkové zpracování (Batch Processing)** – data se shromáždí a zpracují najednou v určený čas (noční mzdové výpočty, měsíční výpisy); nízká latence požadavku, vysoká propustnost
- **Interaktivní zpracování (Online/Interactive)** – uživatel přímo interaguje se systémem v reálném čase; odpověď do sekund (bankovní přepážka, rezervační systém)
- **Distribuované zpracování** – výpočet rozložen na více uzlů/serverů; škálovatelnost, odolnost proti výpadkům; příklady: cloud, mikroservisy, Hadoop
- **Zpracování řízené událostmi (Event-driven)** – komponenty reagují na události (events); volná vazba; příklady: IoT, mikroservisy s message brokery, GUI
- **Zpracování v reálném čase (Real-time)** – striktní časové požadavky; zpracování musí proběhnout v definovaném čase; kritické systémy (řízení letadel, průmyslové stroje, streaming)

Centralizované vs. decentralizované zpracování

- **Centralizované** – mainframe/server; jednoduchá správa, single point of failure
- **Decentralizované** – výpočty u klienta; autonomie, ale složitá synchronizace

10.4.3 Typy IS/ICT služeb

- **Informační služby** – poskytování informací a dat (reporty, dashboardy, notifikace)
- **Aplikační služby** – softwarové funkce zpřístupněné uživatelům nebo systémům (ERP, CRM, API)
- **Infrastrukturní služby** – síť, servery, storage, cloud (IaaS); zajišťuje IT provoz
- **Podpůrné služby** – helpdesk, service desk, školení, konzultace

SLA (Service Level Agreement) Formální smlouva definující úroveň poskytované služby:

- **Dostupnost** – např. 99.9% uptime (8.7 hodin výpadku/rok)
- **Čas odezvy** – jak rychle systém odpoví na požadavek
- **Čas obnovy** – RTO (Recovery Time Objective), RPO (Recovery Point Objective)
- **Podpora** – reakční doba helpdesku (P1: 1h, P2: 4h, P3: 1 pracovní den)

Otázky profesorů k tématu: Analýza a návrh IS

Řepa Václav, prof. Ing., CSc.

- Co je MMDIS a jaké jsou jeho hlavní principy?
- Jaký je rozdíl mezi procesním a datovým pohledem na IS?
- Popište elementy EPC diagramu.

Pour Jan, doc. Ing., CSc.

- Jaké jsou výhody BPMN oproti EPC?
- Co je podnikový proces a jak ho modelujeme?
- Jaké jsou fáze analýzy IS v MMDIS?

Chudán David, Ing., Ph.D.

- Jaký je rozdíl mezi Sequence Flow a Message Flow v BPMN?
- Vysvětlete BPMN gateway typy.

Svátek Vojtěch, prof. Ing., Ph.D.

- Co je swimlane a k čemu slouží?
- Jak se modeluje paralelní zpracování v BPMN a EPC?

11 Bezpečnost informačních systémů

11.1 Kryptografie a kryptografická primitiva

11.1.1 Základní pojmy

- **Kryptologie** = Kryptografie + Kryptoanalýza
- **Kryptografie** – věda zabývající se důvěrností komunikace přes nezabezpečený kanál; způsoby, jak zašifrovat odesílaná data, aby jim nepovolaná osoba neporozuměla
- **Kryptoanalýza** – zabývá se slabinami šifrovacích systémů; způsoby dešifrování bez znalosti klíčů

Cíle kryptografie

- **Důvěrnost (Confidentiality)** – obsah zprávy znají pouze oprávněné strany; šifrování
- **Autentizace (Authentication)** – ověření identity; digitální podpis, certifikát
- **Integrita (Integrity)** – zpráva nebyla po cestě změněna; hash, MAC, digitální podpis
- **Neodmítnutelnost (Non-repudiation)** – odesílatel nemůže popřít odeslání; digitální podpis

11.1.2 Kryptografická primitiva

Algoritmy se základními kryptografickými vlastnostmi, ze kterých se skládají kryptografické protokoly:

- **Bez klíče:** náhodná čísla, hašovací funkce (*př.: SHA-256 – z libovolné zprávy vyrobí 256bitový otisk; ověření integrity souboru*)
- **Se sdíleným klíčem:** symetrické šifry (blokové, proudové), MAC (*př.: AES – šifrování dat na disku; HMAC – ověření authenticity cookie na webu*)
- **S veřejným klíčem:** digitální podpisy, šifrování s veřejným klíčem, výměna klíčů (KEX) (*př.: RSA – zašifrování AES klíče při navázání TLS spojení; ECDSA – digitální podpis PDF dokumentu*)

11.1.3 Symetrická kryptografie

Stejný klíč pro šifrování i dešifrování. Rychlá, vhodná pro velká data.

Proudové šifry (Stream Ciphers)

- Generátor proudu bitů; XOR s otevřeným textem → šifrovaný text
- Klíč se nesmí opakovat (každá zpráva musí mít nový klíč)
- **Vernamova šifra (One Time Pad)** – klíč stejně dlouhý jako zpráva; teoreticky perfektní
- Příklady: Salsa20, ChaCha20, RC4 (zastaralý)

Příklad šifrování (zjednodušeně na bajtech):

	Binárně	Hex	Poznámka
Zpráva (plaintext)	01001000	48	znak 'H'
Proud klíče (keystream)	10110011	B3	generováno ze sdíleného klíče
Šifrovaný text (XOR)	11111011	FB	výsledek XOR

Dešifrování: XOR šifrovaného textu se stejným proudem klíče vrátí původní zprávu (FB XOR B3 = 48 = 'H'). Bezpečnost závisí na nepředvídatelnosti proudu klíče – proto nesmí být tentýž klíč použit dvakrát.

Blokové šifry (Block Ciphers)

- Otevřený text se rozdělí do bloků n bitů, každý blok se šifruje samostatně
- Substitučně-permutační síť; rundy
- **DES** – 70. léta; 56-bit klíč, 64-bit blok; prolomeno v roce 2000 (zastaralý)
- **AES (Advanced Encryption Standard)** – nástupce DES; 128-bit blok; klíč 128/192/256 bitů; 10 rund; standard

Módy blokových šifer

- **ECB (Electronic Codebook)** – každý blok šifrován nezávisle; **nepoužívat** – vzory v šifrovaném textu
- **CBC (Cipher Block Chaining)** – XOR s předchozím šifrovaným blokem + IV; dlouho populární, ale má bezpečnostní slabiny
- **CTR (Counter Mode)** – z blokové šifry vytvoří proudovou; bloky šifrovatelné paralelně; základ CTR_DRBG
- **GCM (Galois/Counter Mode)** – CTR + MAC (AEAD); aktuálně preferovaný mód; SSL/TLS 1.3
 - **MAC (Message Authentication Code)** – kryptografický otisk zprávy vytvořený se sdíleným klíčem; ověřuje zároveň *integritu* (zpráva nebyla změněna) i *autentizaci* (zprávu vytvořil někdo se správným klíčem). Na rozdíl od prostého hashe útočník bez klíče MAC nezfalšuje. V GCM se MAC počítá přes Galoisovo tělové násobení → výsledkem je **authentication tag** (obvykle 128 bitů) přiložený k šifrovanému textu.

Inicializační vektor (IV) – náhodná hodnota kombinovaná s prvním blokem; zabrání útokům opakováním.

11.1.4 Asymetrická kryptografie

Dva matematicky svázané klíče: **veřejný (public key)** a **soukromý (private key)**. Veřejný klíč lze sdílet; soukromý musí zůstat tajný.

Typy algoritmů

- **RSA** – bezpečnost stojí na *obtížnosti faktorizace velkých čísel* ($n = p \cdot q$); klíče 2048+ bitů. *Princip šifrování*: veřejný klíč (n, e) , šifrování $c = m^e \bmod n$; dešifrování $m = c^d \bmod n$ soukromým klíčem d . Odvodit d z veřejného (n, e) vyžaduje znát p a q , což je pro velká n výpočetně neřešitelné. *Použití*: šifrování AES klíče (hybridní šifrování), digitální podpis.
- **Diffie-Hellman (DH)** – bezpečnost stojí na *obtížnosti diskrétního logaritmu*; výměna klíčů bez přenosu tajemství. Obě strany se dohodnou na veřejných parametrech p, g ; každá zvolí soukromé číslo, vypočte veřejnou hodnotu a pošle ji druhé straně; ze dvou veřejných hodnot obě strany odvodí *stejně* sdílené tajemství, aniž ho kdykoliv přenesly po síti. *Použití*: výměna klíčů v TLS (ECDHE pro Perfect Forward Secrecy).
- **Eliptické křivky (EC)** – stejná matematická idea jako DH, ale operace probíhají na eliptické křivce nad konečným tělesem. Výsledek: **výrazně kratší klíče** pro stejnou bezpečnost (256 bitů EC $\approx$ 3072 bitů RSA). Varianty: ECDH (výměna klíčů), ECDSA / Ed25519 (digitální podpis). *Použití*: TLS 1.3 (ECDHE), certifikáty, SSH klíče.

Diffie-Hellman výměna klíčů

1. Dohodnou se na veřejných číslech p (prvočíslo) a g (generátor)
2. Alice zvolí soukromé a , spočítá $A = g^a \bmod p$ a pošle Bobovi
3. Bob zvolí soukromé b , spočítá $B = g^b \bmod p$ a pošle Alici
4. Sdílené tajemství: $A^b \bmod p = B^a \bmod p = g^{ab} \bmod p$

Odposlechem lze získat A a B , ale výpočet a nebo b je prakticky nemožný (diskrétní logaritmus).

Limitace asymetrické kryptografie

- Pomalejší než symetrická (složitě matematické operace)
- Omezená délka zprávy, kterou lze přímo zašifrovat
- V praxi: asymetrická kryptografie pro výměnu klíčů $\rightarrow$ symetrická pro data (TLS, PGP)

11.2 Hash, MAC, Digitální podpis

11.2.1 Hašovací funkce (Hash)

- **Fixní délka výstupu** – nezávisle na délce vstupu (SHA-256 $\rightarrow$ 256 bitů)
- **Jednosměrná** – nelze z hashe rekonstruovat vstup
- **Deterministická** – stejný vstup $\rightarrow$ vždy stejný hash
- **Lavinový efekt** – malá změna vstupu $\rightarrow$ zcela jiný hash
- Algoritmy: SHA-256, SHA-3, bcrypt (pro hesla), argon2 (pro hesla)

Hash zajišťuje pouze integritu, ne autentizaci ani neodmítnutelnost.

11.2.2 MAC (Message Authentication Code)

- Ověří, že zpráva byla vygenerována někým, kdo zná sdílený klíč (symetrická autentizace)
- Zajišťuje integritu i autentizaci původu
- Nejčastěji v cookies: Login + heslo $\rightarrow$ cookie s MAC; server ověří při každém požadavku
- **HMAC** – Hash-based MAC; $HMAC(k, m) = H((k \oplus \text{opad}) || H((k \oplus \text{ipad}) || m))$

11.2.3 AEAD – Authenticated Encryption with Associated Data

Současné šifrování i výpočet MAC – efektivnější než kombinace šifra + MAC:

- **MAC-then-Encrypt** – SSL/TLS < 1.3; zranitelný na padding oracle
- **Encrypt-then-MAC** – IPsec, SSH; bezpečnější
- **Encrypt-and-MAC** – SSH; zranitelný (MAC odhaluje info o plaintextu)
- **GCM mode** – AES-GCM je AEAD; nejbezpečnější přístup

Ukázka AES-GCM (nejpoužívanější AEAD):

1. Vstup: klíč (256 bit), nonce/IV (96 bit), plaintext, Associated Data (AD – nešifrovaná, ale chráněná data, např. HTTP hlavičky)
2. CTR mód zašifruje plaintext $\rightarrow$ ciphertext
3. GHASH (Galoisovo tělové násobení) spočítá authentication tag (128 bit) přes ciphertext + AD
4. Výstup: ciphertext + authentication tag
5. Příjemce: dešifruje CTR módem, přepočítá tag; pokud se neshoduje $\rightarrow$ zpráva zamítnuta (integrita + autentizace)

Klíčová vlastnost: Associated Data (AD) jsou *chráněna tagem, ale nešifrována* – příjemce ověří, že AD nebyla změněna, ale může je číst v plaintextu. Typické použití: HTTP/2 hlavičky v TLS 1.3.

11.2.4 Digitální podpis

Asymetrický ekvivalent MAC – zajišťuje autentizaci i neodmítnutelnost.

Princip

1. Z textu se vytvoří hash
2. Hash se zašifruje **soukromým klíčem** odesílatele → digitální podpis
3. Příjemce dešifruje podpis **veřejným klíčem** odesílatele → původní hash
4. Příjemce spočítá hash z přijatého textu a porovná – shoda = integrita + autentizace

Primitiv	Integrita	Autentizace	Neodmítnutelnost
Hash	Ano	Ne	Ne
MAC	Ano	Ano	Ne
Digitální podpis	Ano	Ano	Ano

11.3 Certifikáty, eIDAS

11.3.1 Digitální certifikát

Certifikát je datová struktura obsahující:

- Identifikační údaje (CN, O, OU, C)
- Platnost certifikátu (od – do)
- Veřejný klíč
- Rozšiřující údaje (SAN, Key Usage)
- Digitální podpis certifikační autority (CA)

Infrastruktura veřejných klíčů (PKI) PKI (Public Key Infrastructure) je soustava technologií, rolí a procesů, která umožňuje bezpečné používání asymetrické kryptografie ve velkém měřítku. Základní problém: jak víte, že veřejný klíč, který jste dostali, skutečně patří tomu, za koho se dotyčný vydává? PKI to řeší pomocí **certifikátů** – důvěryhodná třetí strana (CA) podepíše veřejný klíč spolu s identitou vlastníka.

- **CA (Certification Authority)** – vydává a podepisuje certifikáty; ověřuje identity žadatelů; *př.: Let's Encrypt, DigiCert, I.CA*
- **RA (Registration Authority)** – ověřuje identitu žadatele jménem CA (fyzická kontrola dokladů); CA pak certifikát vydá

- **Řetězec důvěry** – Root CA → Intermediate CA → koncový certifikát; prohlížeč důvěřuje Root CA (předinstalované); ověří celý řetězec až ke koncovému certifikátu
- **CRL (Certificate Revocation List)** – seznam zrušených certifikátů; nevýhoda: může být zastaralý
- **OCSP** – online dotaz na CA v reálném čase; odpověď: valid / revoked / unknown

11.3.2 eIDAS – Elektronická identifikace a podpisy

eIDAS – nařízení EU č. 910/2014 o elektronické identifikaci a důvěryhodných službách. Vztah z hlediska komunikace “osoba – stát”.

Typy elektronických podpisů dle eIDAS

Druh el. podpisu	Kvalita certifikátu	Uložení priv. klíče
Zaručený el. podpis	Žádný požadavek	Žádný požadavek
Uznávaný el. podpis	Kvalifikovaný certifikát	Žádný požadavek
Kvalifikovaný el. podpis	Kvalifikovaný certifikát	Kvalifikovaný prostředek (HW)

- **Zaručený el. podpis** – splňuje eIDAS požadavky pro zaručený podpis; vydávají např. CESNET, I.CA
- **Uznávaný el. podpis** – český právní pojem (zákon č. 297/2016 Sb.); = zaručený el. podpis + kvalifikovaný certifikát; soukromý klíč může být uložen softwarově (na disku); *jako samostatný produkt* ho poskytovatelé nenabízejí – v praxi získáte kvalifikovaný certifikát a použijete ho buď se SW klíčem (uznávaný podpis) nebo s HW tokenem (kvalifikovaný podpis)
- **Kvalifikovaný el. podpis** – privátní klíč na kvalifikovaném prostředku (čipová karta, USB token); nelze vyexportovat; nejvyšší právní síla

11.4 Identifikace, Autentizace, Vícefaktorová autentizace

11.4.1 Identifikace vs. Autentizace vs. Autorizace

- **Identifikace** – entita předloží identifikátor (uživatelské jméno); systém pozná, kdo to je
- **Autentizace** – ověření, že entita je skutečně tím, za koho se vydává (heslo, certifikát)
- **Autorizace** – udělení práv; co smí autentizovaný uživatel dělat
- **Auditování** – sledování činnosti a vytváření záznamů; zajišťuje odpovědnost

11.4.2 Hesla

Ukládání hesel:

- Hesla se ukládají jako hash, **nikdy v plain textu**
- **Salt** – náhodný řetězec přidaný k heslu před hashováním; zabrání rainbow table útokům
- **Pepper** – tajná hodnota přidaná k heslu; uložena odděleně od databáze
- Moderní funkce: **argon2** (doporučený), **bcrypt** – záměrně pomalé; parametrizovatelné

Formát bcrypt hashe: \$2a\$12\$salt(22 znaků)hash(31 znaků)

- \$2a\$ – typ algoritmu (Bcrypt)
- 12\$ – cost faktor ($2^{12} = 4096$ iterací)

11.4.3 Vícefaktorová autentizace (MFA)

Kombinace více faktorů z různých kategorií:

- **Něco co vím** – heslo, PIN, bezpečnostní otázka
- **Něco co mám** – telefon (OTP), hardware token, čipová karta
- **Něco co jsem** – biometrika (otisk prstu, sken sítnice, hlas)

2FA (Two-Factor Authentication) – přesně 2 různé faktory. **2SV (Two-Step Verification)** – 2 kroky ověření (mohou být ze stejné kategorie, např. heslo + OTP SMS). V praxi se pojmy 2FA a 2SV nerozlišují.

TOTP (Time-based One-time Password) – Jednorázové heslo generované na základě aktuálního času a sdíleného tajemství. Platné zpravidla 30 sekund. Standard RFC 6238.

FIDO2 / WebAuthn – Autentizační systém pro webové aplikace (W3C + FIDO Alliance). Vstupní brána do SSO (Single Sign On). Eliminuje hesla – autentizace pomocí HW klíče, biometrie nebo PIN s kryptografickým důkazem.

11.5 Segmentace sítě a bezpečnostní technologie

11.5.1 Segmentace sítě

Segmentace sítě = vytvoření podsítí oddělujících různé části sítě. Omezuje šíření malware a zvyšuje výkon.

11.5.2 Firewall

Síťové zařízení blokující nebo povolující provoz na základě pravidel.

Techniky blokování

- **drop** – paket zahozen; odesílatel nedostane žádnou odpověď
- **reject** – paket zahozen; odesílatel dostane ICMP zprávu o nedostupnosti
- **tcp reset** – pošle TCP RST

Typy firewallů

- **Packet filter** – filtrování na základě hlaviček paketů; pracuje na vrstvách L3 a L4 síťového modelu: **L3 (síťová vrstva)** = zdrojová/cílová IP adresa, TTL; **L4 (transportní vrstva)** = TCP/UDP port, příznaky (SYN, ACK); nezkoumá obsah paketu; rychlý, ale nedokáže odhalit aplikační hrozby
- **Stateful inspection** – sleduje stav TCP spojení
- **Application layer (WAF)** – filtrování na aplikační vrstvě (HTTP, HTTPS)

11.5.3 Proxy servery

Proxy server funguje jako prostředník mezi klientem a cílovým serverem.

- **Forward Proxy** – u klienta; maskuje identitu klienta; caching; content filtering
- **Reverse Proxy** – u serveru; load balancing; SSL termination; ochrana interních serverů

11.5.4 NAT (Network Address Translation)

Překlad IP adres v záhlaví paketů:

- Skrývá interní síťovou strukturu
- Šetří veřejné IPv4 adresy (privátní → veřejná)
- **SNAT (Source NAT)** – mění zdrojovou IP (pro odchozí provoz)

Příklad: PC s IP 192.168.1.10 odešle HTTP požadavek; router přepíše zdrojovou IP na svou veřejnou 1.2.3.4; vzdálený server odpovídá na 1.2.3.4; router přeloží zpět na 192.168.1.10.

- **DNAT (Destination NAT)** – mění cílovou IP (port forwarding)

Příklad: Na veřejné IP 1.2.3.4:80 běží webový server ve vnitřní síti na 192.168.1.20:80; příchozí paket na 1.2.3.4:80 je routerem přeložen a doručen na 192.168.1.20:80.

- **PAT (Port Address Translation) = NAPT** – rozšíření SNAT; *mnoho privátních IP sdílí jednu veřejnou IP* pomocí různých portů

Příklad: Tři PC (192.168.1.10, .11, .12) naváží každé HTTP spojení; router přiřadí každému jiný zdrojový port na veřejné IP:

Interní	→	Veřejné (po PAT)
192.168.1.10:54001	→	1.2.3.4:60001
192.168.1.11:54001	→	1.2.3.4:60002
192.168.1.12:54001	→	1.2.3.4:60003

Router udržuje NAT tabulku a při odpovědi přeloží zpět. Díky PAT může celá domácí síť (254 zařízení) sdílet jednu veřejnou IPv4 adresu.

11.5.5 IDS a IPS

IDS (Intrusion Detection System)

- Detekuje neobvyklé aktivity; generuje varování (alert)
- **NIDS** – sleduje celou síť; **HIDS** – sleduje jeden host
- Metody detekce: signaturová, anomálievá (ML)

IPS (Intrusion Prevention System)

- IDS + aktivní blokování detekované hrozby
- Zastaví útok v reálném čase
- Kombinace IDS/IPS je běžná

11.5.6 TLS (Transport Layer Security)

Kryptografický protokol zajišťující bezpečnou komunikaci přes síť. Nástupce SSL.

TLS 1.2 handshake

1. Client Hello (cipher suites, random)
2. Server Hello (certifikát, cipher suite)
3. Key exchange (RSA nebo DH)
4. Change Cipher Spec, Finished (obě strany)
5. Šifrovaná komunikace

TLS 1.3 (aktuální standard) **RTT (Round Trip Time)** = čas jedné výměny zpráv tam a zpět (klient → server → klient). TLS 1.2 potřeboval **2 RTT** před začátkem šifrované komunikace (dva úplné oběhy). TLS 1.3 to zvládne za **1 RTT** – klient rovnou v prvním Client Hello pošle i Key Share (ECDHE parametry); server může odpovědět šifrovaně a ihned přiložit certifikát i Finished zprávu.

Princip TLS 1.3 handshake (1-RTT):

1. **Client Hello** – klient pošle: podporované cipher suites, náhodné číslo, *Key Share* (veřejné ECDHE parametry)

2. **Server Hello + šifrovaná data** – server vybere cipher suite, pošle svůj Key Share; oba odvodí sdílené tajemství; server hned pošle certifikát a Finished (šifrovaně)
3. **Client Finished** – klient ověří certifikát, pošle Finished; spojení je navázáno
4. Výsledek: šifrovaná komunikace začíná po **jednom obousměrném výměnu** (1 RTT)
 - Pouze bezpečné cipher suites; odstraněny staré (RC4, DES, MD5, RSA key exchange)
 - Povinný Perfect Forward Secrecy (ECDHE) – kompromitace dlouhodobého klíče neohrožuje minulé relace
 - **0-RTT** pro opakovaná připojení – klient pošle data hned v prvním paketu pomocí session ticketu; pozor na replay attacks (odeslaná data lze zopakovat)

11.5.7 VPN (Virtual Private Network)

- VPN klient vytvoří virtuální síťové rozhraní na úrovni OS
- Veškerý provoz tunelován přes VPN server (šifrovaně)
- VPN server aplikuje NAT – cílový server vidí IP VPN serveru
- **Remote Access VPN** – vzdálený přístup pracovníků do firemní sítě
- **Site-to-Site VPN** – propojení dvou firemních lokalit
- Protokoly: OpenVPN, WireGuard, IKEv2/IPsec, L2TP/IPsec

11.5.8 Honeypot a Web Application Firewall

Honeypot

- Systém záměrně navržený jako lákadlo pro útočníky
- Detekuje a zaznamenává útočnickou činnost
- Honeynety = sítě honeypotů

WAF (Web Application Firewall)

- Chrání webové aplikace před OWASP Top 10 (SQLi, XSS, CSRF, DoS)
- Funguje jako obousměrný proxy pro HTTP/HTTPS
- Analyzuje obsah požadavků, blokuje škodlivé vzory
- ~70% hlášených útoků cílí na aplikační vrstvu

Zeman Václav, Ing., Ph.D.

- Vysvětlete rozdíl mezi symetrickou a asymetrickou kryptografií.
- Jak funguje digitální podpis?
- Jaký je rozdíl mezi hashem, MAC a digitálním podpisem z hlediska bezpečnostních vlastností?
- Co je TLS a jak probíhá TLS handshake?

Sedláček Jiří, Ing., Ph.D.

- Co je eIDAS a jaké typy elektronických podpisů definuje?
- Jak funguje PKI a certifikáty?
- Vysvětlete vícefaktorovou autentizaci a její faktory.

Komise: Svátek, Chlapek, Zbořil

- Jaké jsou technologie pro segmentaci sítě?
- Co je IDS a jak se liší od IPS?
- Jaký je rozdíl mezi forward a reverse proxy?

Chudán David, Ing., Ph.D.

- Jak se správně ukládají hesla v databázi?
- Co je salt a proč je důležitý?
- Jak funguje NAT?

Vojtěch Stanislav, Ing., Ph.D.

- Co jsou AEAD schémata a proč jsou důležitá?
- Jaký je bezpečnostní problém s ECB módem blokové šifry?